Tugas Besar 1 IF3070 Dasar Inteligensi Artifisial

Optimasi Penempatan Kendaraan pada Geladak Kapal Kargo Menggunakan Algoritma Local Search



Kelompok 38:

Yumna Fathonah Kautsar	18224063
Rafi Putra Nugraha	18224079
Illona Nasywa Hannum	18224081

Dosen Pengampu :
Ir. Windy Gambetta, MBA

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
INSTITUT TEKNOLOGI BANDUNG
2025

DAFTAR ISI

DAFTAR ISI **2**
DAFTAR TABEL **5**
I. Deskripsi Persoalan **6**
II. Pembahasan **7**
2.1 Pemilihan Objective Function 7
2.2 Implementasi Model Algoritma 8
2.2.1 Kelas dan Fungsi pada Model.py 11
2.2.3 Kelas dan Fungsi pada Modul neighbors.py 13
2.2.4 Kelas dan Fungsi pada Modul objective.py 17
2.2.5 Kelas dan Fungsi pada Modul loader.py 20
2.2.6 Kelas dan Fungsi pada Modul initial.py 21
2.3 Implementasi Algoritma Local Search 23
2.3.1 Algoritma Steepest Ascent Hill-Climbing 23
Tabel 2.3.1 Algoritma Steepest Ascent Hill-Climbing 23
2.3.2 Algoritma Sideways Move Hill-Climbing 28
Tabel 2.3.2 Algoritma Sideways Move Hill-Climbing 28
2.3.3 Algoritma Random Restart Hill-Climbing 32
Tabel 2.3.3 Algoritma Random Restart Hill-Climbing 33
2.3.4 Algoritma Stochastic Hill-Climbing 36
Tabel 2.3.4 Algoritma Stochastic Hill-Climbing 37
2.3.5 Algoritma Simulated Annealing 40
Tabel 2.3.5.1 Algoritma Simulated Annealing 41
2.3.6 Algoritma Genetic 49
Tabel 2.3.6.1 Genetic Algorithm 49
III. Analisis dan Hasil Eksperimen **58**
3.1 Algoritma Steepeset Ascent Hill Climbing 58
3.1.1 Eksperimen Pertama 58
3.1.2 Eksperimen Kedua 59
3.1.3 Eksperimen Ketiga 60
3.1.4 Analisis Singkat 61
3.2 Algoritma Sideways Move Hill Climbing 62
3.2.1 Eksperimen Pertama 62

3.2.2 Eksperimen Kedua..... 63
3.2.3 Eksperimen Ketiga..... 64
3.2.4 Analisis Singkat.....65
3.3 Algoritma Random Restart Hill Climbing..... 66
3.3.1 Eksperimen Pertama.....66
3.3.2 Eksperimen Kedua.....67
3.3.3 Eksperimen Ketiga..... 68
3.3.4 Analisis Singkat.....69
3.4 Algoritma Stochastic Hill Cimbing..... 70
3.4.1 Eksperimen Pertama.....70
3.4.2 Eksperimen Kedua..... 71
3.4.3 Eksperimen Ketiga..... 72
3.4.4 Analisis Singkat.....73
3.5 Algoritma Simulated Annealing..... 74
3.1.1 Eksperimen Pertama..... 74
3.1.2 Eksperimen Kedua..... 75
3.1.3 Eksperimen Ketiga..... 76
3.6 Algoritma Genetic..... 82
3.6.1 Eksperimen Pertama.....82
3.1.2 Eksperimen Kedua..... 83
3.1.3 Eksperimen Ketiga.....84
3.1.4 Eksperimen Keempat..... 85
3.1.5 Eksperimen Kelima.....86
3.1.6 Eksperimen Keenam..... 87
3.1.7 Eksperimen Ketujuh..... 88
3.1.8 Eksperimen Kedelapan.....89
3.1.9 Eksperimen Kesembilan..... 90
3.1.10 Eksperimen Kesepuluh.....91
3.1.11 Eksperimen Kesebelas..... 92
3.1.12 Eksperimen Kedua belas..... 93
3.1.13 Eksperimen Ketiga belas..... 94
3.1.14 Eksperimen Keempat belas.....95
3.1.15 Eksperimen Kelima belas.....96
3.1.16 Eksperimen Keenam belas.....97
3.1.17 Eksperimen Ketujuh belas.....98

3.1.18 Eksperimen Kedelapan belas..... 99
3.7 Analisis Keseluruhan.....101
IV. Kesimpulan dan Saran.....101
V. Pembagian Tugas.....101
VI. Referensi.....102

I. Deskripsi Persoalan

Permasalahan yang dikaji dalam laporan ini adalah bagaimana mengoptimalkan penempatan kendaraan pada geladak kapal kargo yang bertujuan untuk menentukan susunan kendaraan agar keuntungan pengangkutan kendaraan yang diperoleh dapat maksimal. Laporan ini disusun untuk memenuhi Tugas Besar 1 pada mata kuliah **IF3070 Dasar Intelegensi Artifisial**.

Setiap kendaraan memiliki karakteristik berupa dimensi panjang & lebar, berat, orientasi dan *shipping fee*. Sementara itu, geladak kapal memiliki keterbatasan dimensi dan kapasitas berat yang harus dipenuhi. Oleh karena itu, permasalahan ini berfokus pada pencarian konfigurasi penempatan kendaraan yang optimal dengan mempertimbangkan berbagai batasan yang ada. Konfigurasi mencakup pemilihan kendaraan yang akan diangkut, posisi koordinat setiap kendaraan, serta orientasinya. Kendaraan yang tidak diangkut tetap diperhitungkan sebagai bagian dari kendaraan, tetapi ditempatkan di luar geladak kapal

Untuk menyelesaikan permasalahan ini, digunakan pendekatan local search dengan tiga algoritma, yaitu *Hill-Climbing, Simulated Annealing*, dan *Genetic Algorithm*. Ketiga algoritma tersebut digunakan untuk mencari konfigurasi penempatan kendaraan yang memberikan nilai objektif tinggi melalui mekanisme pencarian yang berbeda. Hasil eksperimen kemudian dibandingkan berdasarkan nilai objektif yang diperoleh, durasi pencarian, dan konsistensi hasil untuk mengevaluasi efektivitas masing-masing algoritma dalam menyelesaikan persoalan penempatan kendaraan pada geladak kapal kargo.

Selain Itu, hasil dari pengujian ini divisualisasikan untuk menunjukkan *state* awal dan akhir dari koordinat titik penempatan kendaraan serta perubahan nilai *objective function* selama iterasi. Hal tersebut dilakukan untuk memberikan pengetahuan mengenai efektivitas dan efisiensi berbagai algoritma *local search*, serta faktor-faktor yang mempengaruhi kinerja pencarian solusi untuk penempatan kendaraan pada geladak kapal kargo.

II. Pembahasan

2.1 Pemilihan *Objective Function*

Fungsi objektif dirancang untuk menghasilkan nilai yang mencerminkan seberapa bagus suatu susunan kendaraan di kapal. State susunan ini direpresentasikan sebagai dictionary dengan key vehicle_id dan valuenya berupa object Placement. Placement berisi koordinat serta orientasi kendaraan. Kendaraan yang tidak masuk ke dalam kapal tetap masuk ke dalam entry dictionary State, hanya saja nilai koordinatnya berupa None. Objektif dalam kasus ini adalah untuk memaksimalkan keuntungan yang didapat dari fee shipping, sehingga ini menjadi patokan untuk fungsi objektif.

Komponen utamanya adalah fee, penalty overlap, penalty out of bounds, dan penalty overweight. Keempat komponen ini memiliki fungsi helper untuk menghitung total nilai keempat komponen sehingga perhitungan fungsi objektif lebih mudah.

Konstanta Penalty dihitung melalui fungsi penalty_weights(), yang terdiri dari dua konstanta dasar, yaitu base dan rate. Base dihitung dari fee maksimum dari seluruh kendaraan ditambah 1, sedangkan rate dihitung dari rasio fee:berat tertinggi. Base akan digunakan untuk menjadi multiplier saat menghitung nilai total penalti overlap, out of bounds, dan overweight. Rate akan digunakan untuk menghitung faktor tambahan untuk penalty overweight untuk memastikan semakin banyak overweight, penalty akan bertambah r per unit overweight.

Dengan komponen di atas, fungsi objektif dapat didefinisikan sebagai berikut:

$$f(s) = \sum_{v \in \mathrm{In}(s)} \mathrm{fee}_v - \Big(P \cdot \mathrm{overlap}(s) + P \cdot \mathrm{oob}(s) + [\,\mathrm{overweight}(s) > 0\,] \cdot \big(P + r \cdot \mathrm{overweight}(s)\big) \Big)$$

dengan P sebagai base penalty dan r sebagai rate coeffcient.

BONUS: terdapat suatu atribut yang tidak dipakai, yaitu ETA. Dengan atribut ini, fungsi objektif kedua bisa dibuat. The idea is, kita dapat mengimplementasikan fungsi objektif kedua dengan menambahkan ETA sebagai tie-breaker untuk konfigurasi shipping fee yang sama. Konsep dari ETA ialah semakin kecil ETA, semakin tinggi valuenya. Objective functionnya tidak diimplement dalam codebase karena keterbatasan waktu, namun ada function untuk menghitung total_eta.

2.2 Implementasi Model Algoritma

Sebelum membahas implementasi algoritma local search, terlebih dahulu dibangun beberapa modul pendukung yang menangani pemodelan data, pembacaan data masukan, pembentukan solusi tetangga, serta evaluasi kualitas solusi. Keempat modul tersebut memiliki fungsi yang saling berkaitan dan digunakan sebagai fondasi dalam proses pencarian solusi penempatan kendaraan pada geladak kapal kargo.

Nama File	Fungsi	Variabel	Deskripsi
model.py	Vehicle.size_taken(), Ship.in_boundary(), Ship.within_capacity(), Placement.rotated(), Placement.is_inside(), Placement.grids()	orientation: orientasi kendaraan; x, y: koordinat; vw, vl: dimensi kendaraan; total_weight: total berat; vehicle: objek kendaraan; max_capacity: kapasitas maksimum kapal; placements: kumpulan penempatan kendaraan.	Digunakan untuk merepresentasikan struktur data kendaraan dan kapal kargo. Berisi struktur data kendaraan, kapal, posisi kendaraan, keadaan solusi, dan orientasi kendaraan.
objective.py	objective(), total_fee(), total_weight(), count_overlap(), count_out_of_bounds(), excess_weigh	state: keadaan solusi; ship: data kapal; vehicles_by_id: data kendaraan; fee: total tarif; overlap:	Digunakan untuk menilai kualitas setiap keadaan agar dapat membandingkan kandidat solusi. Berisi perhitungan biaya pengiriman, berat total, pelanggaran kendala, nilai objektif, dan pemeriksaan kelayakan

	t(), total_eta(), is_feasible(), penalty_weights()	jumlah overlap; out_of_bounds: jumlah kendaraan di luar batas; overweight: kelebihan berat; weights: bobot penalti.	solusi.
loader.py	load_problem()	path: lokasi file JSON; data: data hasil pembacaan JSON; ship: objek kapal; vehicles_by_id: pemetaan ID kendaraan; raw_vehicle: data mentah satu kendaraan; vehicle: objek kendaraan.	Memuat data awal dari file JSON sebelum algoritma pencarian dimulai. Mengubah data masukan menjadi objek kapal dan kendaraan yang digunakan oleh modul lainnya.
neighbors.py	swap(), move(), move_out(), rotate(), all_neighbors(), random_neighbor(), valid_position	state: keadaan solusi awal; ship: data kapal; vehicles_by_id: data kendaraan; vehicle_id: ID	Dipanggil oleh algoritma untuk melakukan langkah-langkah pencarian solusi. Berisi operasi pertukaran posisi, pemindahan kendaraan, pengeluaran kendaraan dari kapal, rotasi orientasi, serta pembangkitan kandidat

	s(), _with_placements()	kendaraan; a_id dan b_id: pasangan kendaraan; x, y: koordinat; rng: generator acak; move_weights: bobot jenis operasi; neighbor: kandidat state.	tetangga.
initial.py	random_initial_state(), _overlaps(), default_p_inside()	ship: data kapal; vehicles_by_id: data kendaraan; rng: generator acak; p_inside: peluang kendaraan dicoba untuk dimuat; occupied: sel yang sudah terisi; load: total berat terangkut; chosen: penempatan terpilih; order: urutan kendaraan; orientation:	Digunakan untuk membentuk initial state atau keadaan awal secara acak. Mencoba menempatkan kendaraan dengan memperhatikan batas geladak, overlap, dan kapasitas berat kapal sehingga kendaraan yang berhasil dimuat membentuk keadaan awal yang feasible.

		orientasi yang dicoba; free: posisi yang tersedia.	

2.2.1 Kelas dan Fungsi pada Model.py

Modul Model.py digunakan untuk merepresentasikan model kendaraan dan kapal. Berikut atribut dan konstanta yang digunakan dalam modul.

Fungsi
Vehicle.size_taken(), Ship.in_boundary(), Ship.within_capacity(), Placement.rotated(), Placement.is_inside(), Placement.grids()
Source Code

```
from dataclasses import dataclass, replace
from enum import Enum


class Orientation(str, Enum):
    HORIZONTAL = "Horizontal"
    VERTICAL = "Vertical"

    def flipped(self):
        return (
            Orientation.VERTICAL
            if self is Orientation.HORIZONTAL
            else Orientation.HORIZONTAL
        )


@dataclass(frozen=True)
class Vehicle:
    id: str
    w: int  # patokan/orientasi lebar kendaraan: jarak dari ban
kiri ke ban kanan
    l: int  # patokan/orientasi panjang kendaraan: jarak dari
bagian depan ke bagian belakang mobil
    fee: int
    weight: int
```

```
    eta: int

    def size_taken(
        self, orientation: Orientation
    ) -> tuple[int, int]:  # expected output 2 buah int.
        if orientation is Orientation.VERTICAL:
            return (self.l, self.w)
        return (self.w, self.l)


@dataclass(frozen=True)
class Ship:
    w: int
    l: int
    max_capacity: int

    def in_boundary(self, x: int, y: int, vw: int, vl: int) ->
bool:
        # x & y = koordinat posisi kendaraan diletakkan
        # vw & vl = lebar dan panjang (respectively) kendaraan
        return x >= 0 and y >= 0 and x + vw <= self.w and y + vl
<= self.l

    def within_capacity(self, total_weight: int) -> bool:
        return self.max_capacity >= total_weight


@dataclass(frozen=True)
class Placement:
    vehicle_id: str
    x: int | None  # none = kendaraan ngga di kapal
    y: int | None
    orientation: Orientation  # vertical or horizontal

    def rotated(self):
        return replace(self,
orientation=self.orientation.flipped())

    # fungsi checker apakah placement kendaraan ada di dalam
kapal
    def is_inside(self):
        return self.x is not None and self.y is not None

    # fungsi untuk output grid yang di-occupy oleh kendaraan
tertentu dengan placement p
    def grids(self, vehicle: Vehicle):
        if not self.is_inside():
            return []
```

```
        vw, vl = vehicle.size_taken(self.orientation)
        return [
            (self.x + i, self.y + j) for i in range(vw) for j in
range(vl)
        ]  # misal: (1,2), (1,3), (2,2), (2,3)


@dataclass(frozen=True)
class State:
    placements: dict[str, Placement]  # lookup table key->value
```

2.2.3 Kelas dan Fungsi pada Modul neighbors.py

Modul neighbors.py menyediakan operasi untuk membentuk state baru dari state saat ini. Modul ini memungkinkan langkah seperti menukar dua kendaraan, memindahkan sebuah kendaraan ke suatu koordinat yang berbeda, mengubah orientasi kendaraan

Fungsi
swap(), move(), move_out(), rotate(), all_neighbors(), random_neighbor(), valid_positions(), _with_placements()

Source Code

```
import random
import dataclasses

from collections.abc import Iterator
from src.model import Orientation, Placement, Ship, State,
Vehicle

# KONSTANTA
ALLOW_MOVE_OUT = True
SWAP_ORIENTATION = False

def _with_placements(state: State, *new_placements: Placement)
-> State:
    d = dict(state.placements)
    for p in new_placements:
        d[p.vehicle_id] = p
    return State(d)

def valid_positions(
```

```
    ship: Ship, vehicle: Vehicle, orientation: Orientation
) -> Iterator[tuple[int, int]]:
    vw, vl = vehicle.size_taken(orientation)
    if vw > ship.w or vl > ship.l:
        return
    for x in range(0, ship.w - vw + 1):
        for y in range(0, ship.l - vl + 1):
            yield (x, y)

def swap(state: State, a_id: str, b_id: str) -> State | None:
    if a_id == b_id:
        return None
    pa = state.placements[a_id]
    pb = state.placements[b_id]

    if not (pa.is_inside() or pb.is_inside()):
        return None

    pa_baru = dataclasses.replace(pa, x=pb.x, y=pb.y)
    pb_baru = dataclasses.replace(pb, x=pa.x, y=pa.y)
    return _with_placements(state, pa_baru, pb_baru)

def move(state: State, vehicle_id: str, x: int, y: int) -> State
| None:
    p = state.placements[vehicle_id]
    if p.x == x and p.y == y:
        return None
    p_baru = dataclasses.replace(p, x=x, y=y)
    return _with_placements(state, p_baru)

def move_out(state: State, vehicle_id: str) -> State | None:
    p = state.placements[vehicle_id]
    if not p.is_inside():
        return None
    p_baru = dataclasses.replace(p, x=None, y=None)
    return _with_placements(state, p_baru)

def rotate(state: State, vehicle_id: str) -> State | None:
    p = state.placements[vehicle_id]
    if not p.is_inside():
        return None

    return _with_placements(state, p.rotated())

def all_neighbors(
    state: State, ship: Ship, vehicles_by_id: dict[str, Vehicle]
) -> Iterator[State]:
    ids = list(vehicles_by_id)
```

```
    for i, a_id in enumerate(ids):
        for b_id in ids[i + 1 :]:
            neighbor = swap(state, a_id, b_id)
            if neighbor is not None:
                yield neighbor

    for vehicle in vehicles_by_id.values():
        placement = state.placements[vehicle.id]
        for x, y in valid_positions(ship, vehicle,
placement.orientation):
            neighbor = move(state, vehicle.id, x, y)
            if neighbor is not None:
                yield neighbor

    if ALLOW_MOVE_OUT:
        for vehicle in vehicles_by_id.values():
            if state.placements[vehicle.id].is_inside():
                neighbor = move_out(state, vehicle.id)
                if neighbor is not None:
                    yield neighbor

    for vehicle in vehicles_by_id.values():
        placement = state.placements[vehicle.id]
        if placement.is_inside() and vehicle.w != vehicle.l:
            neighbor = rotate(state, vehicle.id)
            if neighbor is not None:
                yield neighbor

# define move_weights di masing-masing algorithm. move_weights =
{"swap": 1, ..., "move_out": 0.12} misalnya
def random_neighbor(
    state: State, ship: Ship, vehicles_by_id: dict[str,
Vehicle], rng: random.Random, move_weights: dict[str, float]) ->
State | None:
    ids = list(vehicles_by_id)
    move_types = [
        move_type
        for move_type in move_weights
        if move_type != "move_out" or ALLOW_MOVE_OUT
    ]
    weights = [move_weights[move_type] for move_type in
move_types]

    for _ in range(100):
        move_type = rng.choices(move_types, weights=weights,
k=1)[0]
```

```
        if move_type == "swap":
            if len(ids) < 2:
                continue
            a_id, b_id = rng.sample(ids, 2)
            neighbor = swap(state, a_id, b_id)
        elif move_type == "move":
            if not vehicles_by_id:
                continue
            vehicle = vehicles_by_id[rng.choice(ids)]
            placement = state.placements[vehicle.id]
            positions = [
                position
                for position in valid_positions(ship, vehicle,
placement.orientation)
                if position != (placement.x, placement.y)
            ]
            if not positions:
                continue
            x, y = rng.choice(positions)
            neighbor = move(state, vehicle.id, x, y)
        elif move_type == "move_out":
            inside_ids = [
                vehicle_id
                for vehicle_id in ids
                if state.placements[vehicle_id].is_inside()
            ]
            if not inside_ids:
                continue
            neighbor = move_out(state, rng.choice(inside_ids))
        elif move_type == "rotate":
            rotatable_ids = [
                vehicle.id
                for vehicle in vehicles_by_id.values()
                if state.placements[vehicle.id].is_inside()
                and vehicle.w != vehicle.l
            ]
            if not rotatable_ids:
                continue
            neighbor = rotate(state, rng.choice(rotatable_ids))
        else:
            continue

        if neighbor is not None:
            return neighbor

    return None
```

2.2.4 Kelas dan Fungsi pada Modul objective.py

Modul objective.py digunakan untuk menghitung nilai objektif dari setiap keadaan solusi berdasarkan total biaya pengiriman dan penalti pelanggaran kendala. Modul ini juga menyediakan fungsi pendukung untuk menghitung berat, mendeteksi tumpang tindih, memeriksa batas geladak, dan menentukan kelayakan solusi.

Fungsi
objective(), total_fee(), total_weight(), count_overlap(), count_out_of_bounds(), excess_weight(), total_eta(), is_feasible(), penalty_weights()
Source Code

```
from dataclasses import dataclass

from src.model import Ship, State, Vehicle
from collections import Counter


@dataclass(frozen=True)
class PenaltyWeights:
    overlap: float  # per sel kelebihan okupansi
    out_of_bounds: float  # per kendaraan yang keluar batas
    overweight_fixed: float  # sekali bila total berat >
kapasitas
    overweight_rate: float  # per unit berat di atas kapasitas


def penalty_weights(vehicles_by_id: dict[str, Vehicle]) ->
PenaltyWeights:
    max_fee = max((v.fee for v in vehicles_by_id.values()),
default=0)
    base = max(0, max_fee) + 1
    rate = max(
        (v.fee / v.weight for v in vehicles_by_id.values() if
v.weight > 0),
        default=0.0,
    )
    return PenaltyWeights(
        overlap=base,
        out_of_bounds=base,
        overweight_fixed=base,
```

```
        overweight_rate=max(0.0, rate),
    )

# Helper functions
# Gunakan vehicles_by_id dari loader untuk lookup kendaraan
berdasarkan ID.
def total_fee(state: State, vehicles_by_id: dict[str, Vehicle])
-> int:
    total = 0

    for p in state.placements.values():
        if p.is_inside():
            vehicle = vehicles_by_id[p.vehicle_id]
            total += vehicle.fee

    return total


def total_weight(state: State, vehicles_by_id: dict[str,
Vehicle]) -> int:
    total = 0

    for p in state.placements.values():
        if p.is_inside():
            vehicle = vehicles_by_id[p.vehicle_id]
            total += vehicle.weight

    return total


def count_overlap(state: State, vehicles_by_id: dict[str,
Vehicle]) -> int:
    occupancy: Counter[tuple[int, int]] = Counter()

    for p in state.placements.values():
        if p.is_inside():
            vehicle = vehicles_by_id[p.vehicle_id]
            occupancy.update(p.grids(vehicle))

    return sum(c - 1 for c in occupancy.values() if c > 1)


def count_out_of_bounds(
    state: State, ship: Ship, vehicles_by_id: dict[str, Vehicle]
) -> int:
    count = 0
    for p in state.placements.values():
        x, y = p.x, p.y
```

```
        if x is not None and y is not None:
            vehicle = vehicles_by_id[p.vehicle_id]
            vw, vl = vehicle.size_taken(p.orientation)
            if not ship.in_boundary(x, y, vw, vl):
                count += 1

    return count


def excess_weight(state: State, ship: Ship, vehicles_by_id:
dict[str, Vehicle]) -> int:
    w = total_weight(state, vehicles_by_id)
    return max(0, w - ship.max_capacity)

# Helper function untuk bonus objective function #2
def total_eta(state: State, vehicles_by_id: dict[str, Vehicle])
-> int:
    total = 0

    for p in state.placements.values():
        if p.is_inside():
            vehicle = vehicles_by_id[p.vehicle_id]
            total += vehicle.eta

    return total

# Objective function
def objective(state: State, ship: Ship, vehicles_by_id:
dict[str, Vehicle]) -> float:
    fee = total_fee(state, vehicles_by_id)
    overlap = count_overlap(state, vehicles_by_id)
    out_of_bounds = count_out_of_bounds(state, ship,
vehicles_by_id)
    overweight = excess_weight(state, ship, vehicles_by_id)

    weights = penalty_weights(vehicles_by_id)
    penalty = overlap * weights.overlap + out_of_bounds *
weights.out_of_bounds
    if overweight > 0:
        penalty += weights.overweight_fixed + overweight *
weights.overweight_rate

    return float(fee - penalty)

# Checker untuk state yang feasible; i.e. tidak melanggar
constraints
def is_feasible(state: State, ship: Ship, vehicles_by_id:
dict[str, Vehicle]) -> bool:
```

```
    return (
        excess_weight(state, ship, vehicles_by_id) == 0
        and count_out_of_bounds(state, ship, vehicles_by_id) ==
0
        and count_overlap(state, vehicles_by_id) == 0
    )
```

2.2.5 Kelas dan Fungsi pada Modul loader.py

Modul loader.py digunakan untuk load data dari file json sebagai problem awal.

Fungsi
load_problem()
Source Code

```
from src.model import Ship, Vehicle
import json


def load_problem(path: str) -> tuple[Ship, dict[str, Vehicle]]:
    with open(path) as f:
        data = json.load(f)

    ship = Ship(**data["ship"])

    # Dict mempertahankan urutan input; tolak ID duplikat
sebelum menimpa data.
    vehicles_by_id: dict[str, Vehicle] = {}
    for raw_vehicle in data["vehicles"]:
        vehicle = Vehicle(**raw_vehicle)
        if vehicle.id in vehicles_by_id:
            raise ValueError(f"Duplicate vehicle id:
{vehicle.id!r}")
        vehicles_by_id[vehicle.id] = vehicle

    return (ship, vehicles_by_id)


if __name__ == "__main__":
    ship, vehicles_by_id = load_problem("src/data/sample.json")
    print(ship)
    print(len(vehicles_by_id))
```

```
    for v in vehicles_by_id.values():
        print(v)
```

2.2.6 Kelas dan Fungsi pada Modul initial.py

Modul initial.py digunakan untuk menginisiasi random state awal.

Fungsi
random_initial_state(), _overlaps(), default_p_inside()
Source Code

```
import random
from src.model import Orientation, Placement, Ship, State,
Vehicle
from src.neighbors import valid_positions


def random_initial_state(ship:Ship, vehicles_by_id:dict[str,
Vehicle], rng:random.Random, p_inside:float | None = None) ->
State:
    # Initial state random yang harus feasible: kendaraan hanya
dinaikkan if and only if muat di dalam kapal, tidak overlap, dan
total berat tetap <= kapasitas.
    if p_inside is None:
        p_inside = default_p_inside(ship, vehicles_by_id)
    if not 0.0 <= p_inside <= 1.0:
        raise ValueError("p_inside harus di antara 0 dan 1")

    occupied: set[tuple[int, int]] = set()
    load = 0
    chosen: dict[str, Placement] = {}

    # Acak order biar rata distribusinya
    order = list(vehicles_by_id)
    rng.shuffle(order)
    for v_id in order:
        v = vehicles_by_id[v_id]
        orientations = rng.sample(list(Orientation), 2)
        placement = Placement(vehicle_id=v_id, x=None, y=None,
orientation=orientations[0])

        if rng.random() < p_inside and load + v.weight <=
ship.max_capacity:
```

```
            for orientation in orientations:
                free = [
                    (x, y)
                    for x, y in valid_positions(ship, v,
orientation)
                    if not _overlaps(occupied, x, y,
*v.size_taken(orientation))
                ]
                if free:
                    x, y = rng.choice(free)
                    placement = Placement(vehicle_id=v_id, x=x,
y=y, orientation=orientation)
                    occupied.update(placement.grids(v))
                    load += v.weight
                    break

        chosen[v_id] = placement

    # Simpan dengan urutan vehicles_by_id (urutan gen GA)
    return State({v_id: chosen[v_id] for v_id in
vehicles_by_id})


# Helper buat cek grid yang udah keisi
def _overlaps(occupied: set[tuple[int, int]], x: int, y: int,
vw: int, vl: int) -> bool:
    return any((x + i, y + j) in occupied for i in range(vw) for
j in range(vl))


# Pilih p supaya rata-rata (expected) total berat & luas di
state awal tidak melebihi kapasitas kapal: E[total] = p * total
->  p = batas / total
def default_p_inside(ship:Ship, vehicles_by_id:dict[str,
Vehicle]) -> float:
    total_weight = sum (v.weight for v in
vehicles_by_id.values())
    total_area = sum(v.w * v.l for v in vehicles_by_id.values())
    ratios = [1.0]
    if total_weight > 0:
        ratios.append(ship.max_capacity / total_weight)
    if total_area > 0:
        ratios.append((ship.w * ship.l) / total_area)
    return min(ratios)
```

2.3 Implementasi Algoritma *Local Search*

Implementasi algoritma *local search* yang digunakan bertujuan untuk membantu pencarian solusi yang paling optimal dalam penempatan kendaraan pada geladak kapal kargo. Algoritma disusun mulai dari *initial state* hingga tercapai local maksimum atau global optimum dengan melakukan serangkaian *moves* yang terdiri dari menukarkan dua kendaraan, memindahkan kendaraan dan mengubah orientasi kendaraan.

2.3.1 Algoritma *Steepest Ascent Hill-Climbing*

Steepest Ascent merupakan salah satu jenis algoritma *Hill-Climbing* dengan mencari susunan kendaraan di kapal dengan nilai fungsi objektif yang lebih tinggi. Algoritma ini memungkinkan *neighbor state* dengan nilai objektif terendah digantikan dengan *current state* yang memiliki nilai objektif lebih tinggi. Pencarian dimulai dari suatu *state* awal, kemudian seluruh *neighbor* melalui operasi pertukaran posisi, perpindahan posisi, pengeluaran kendaraan, dan rotasi. Setiap tetangga dievaluasi menggunakan fungsi objektif yang memperhitungkan total pendapatan serta penalti akibat tumpang tindih, pelanggaran batas kapal, dan kelebihan kapasitas berat. Lalu dipilih tetangga dengan nilai objektif tertinggi. Iterasi terus dilakukan hingga tidak ditemukan *neighbor* dengan nilai objektif yang lebih baik sehingga pencarian berhenti pada optimum lokal atau plateau.

Tabel 2.3.1 Algoritma *Steepest Ascent Hill-Climbing*

Nama File	
hillclimb_steepestascent.py	
Variabel	
start	State awal yang menjadi titik permulaan pencarian
ship	Data dimensi kapal dan kapasitas berat maksimum
vehicles_by_id	Dictionary data kendaraan berdasarkan ID untuk evaluasi dan pembangkitan tetangga
start_time	Waktu awal eksekusi untuk menghitung durasi pencarian

current	State yang sedang menjadi posisi pencarian saat ini
current_value	Nilai fungsi objektif dari state saat ini
history	Riwayat nilai objektif state awal dan setiap state yang dipilih setelah perpindahan
iterations	Jumlah perpindahan ke tetangga yang memberikan peningkatan nilai objektif, bukan jumlah tetangga yang diperiksa
best_state	State feasible dengan nilai objektif tertinggi di antara state yang telah dikunjungi
best_value	Nilai objektif dari best_state
best_neighbor	Tetangga terbaik pada iterasi berjalan yang memberikan peningkatan dibandingkan state saat ini
best_neighbor_value	Nilai objektif tertinggi selama iterasi pemeriksaan neighbor, dimana mula-mula diisi dengan current_value
neighbor	Satu state tetangga yang sedang diperiksa
neighbor_value	Nilai fungsi objektif dari tetangga yang sedang diperiksa
Fungsi	
Search ( )	Implementasi logika pencarian yaitu menjalankan pencarian dengan memeriksa seluruh tetangga, memilih *neighbor* terbaik, mencatat hasil, dan mengembalikan SearchResult.
all_neighbors( )	Menghasilkan seluruh kandidat *neighbors* melalui operasi swap, move, move_out (saat feasible), dan rotate untuk kendaraan yang memenuhi syarat (Kandidat belum tentu feasible).
Objective ( )	Menghitung nilai objektif berupa total pendapatan dikurangi penalti pelanggaran batasan.

is_feasible()	Memeriksa apakah state sudah sesuai dengan *constraint* yang ada (tidak tumpang tindih, tidak melanggar batas kapal, dan tidak kelebihan kapasitas berat)
perf_counter()	Mengukur waktu awal dan akhir untuk menghitung durasi pencarian.
SearchResult	Kelas data untuk menyimpan state awal, solusi feasible terbaik, nilai akhirnya, riwayat, jumlah perpindahan, durasi, dan *extra dictionary*.

Source Code

```
from time import perf_counter
from dataclasses import dataclass

from src.neighbors import all_neighbors
from src.objective import is_feasible, objective
from src.model import State, Vehicle, Ship

@dataclass
class SearchResult:
    initial_state: State
    final_state: State | None
    final_value: float | None
    history: list[float]
    iterations: int
    duration_s: float
    extra: dict

def search(
    start: State,
    ship: Ship,
    vehicles: dict[str, Vehicle],
    vehicles_by_id: dict[str, Vehicle],
    on_iteration=None,
) -> SearchResult:
    start_time = perf_counter()
    current = start
    current_value = objective(current, ship,
vehicles_by_id)
    history = [current_value]
    iterations = 0
    if on_iteration is not None:
```

```
        on_iteration(0, current, current_value)

    best_state: State | None = None
    best_value: float | None = None
    if is_feasible(current, ship, vehicles_by_id):
        best_state, best_value = current, current_value

    while True:
        best_neighbor = None
        best_neighbor_value = current_value

        for neighbor in all_neighbors(current, ship,
vehicles):
            neighbor_value = objective(neighbor, ship,
vehicles_by_id)

            if neighbor_value > best_neighbor_value:
                best_neighbor = neighbor
                best_neighbor_value = neighbor_value

        if best_neighbor is None:  # local optimum
            break

        current, current_value = best_neighbor,
best_neighbor_value
        iterations += 1
        history.append(current_value)
        if on_iteration is not None:
            on_iteration(iterations, current,
current_value)

        # catat best feasible dari state yang dikunjungi
        if (best_value is None or current_value >
best_value) and is_feasible(
            current, ship, vehicles_by_id
        ):
            best_state, best_value = current,
current_value

    return SearchResult(
        initial_state=start,
        final_state=best_state,
        final_value=best_value,
```

```
        history=history,
        iterations=iterations,
        duration_s=perf_counter() - start_time,
        extra={}
    )
```

Prinsip pencarian dari algoritma *Steepest Ascent Hill-Climbing* adalah mengevaluasi seluruh tetangga dari suatu susunan, kemudian memilih satu tetangga yang memberikan nilai objektif lebih tinggi dibandingkan dengan *current state*. Data kapal dan kendaraan dapat dimuat dari file JSON menggunakan fungsi load_problem( ). Fungsi ini menghasilkan objek Ship dan dictionary vehicles_by_id yang memetakan ID kendaraan ke objek Vehicle. State awal dibentuk menggunakan fungsi random_initial_state( ) yang dipanggil dari src/initial.py.

Pada awal fungsi search( ), state awal disimpan sebagai current, kemudian nilainya dihitung menggunakan objective(current, ship, vehicles_by_id) dan disimpan dalam current_value. Nilai awal tersebut juga dimasukkan ke dalam list history, sedangkan iterations diinisialisasi dengan nilai nol. Variabel best_state dan best_value digunakan untuk menyimpan solusi feasible terbaik yang dikunjungi. Keduanya mula-mula bernilai None, lalu diperbarui apabila state awal memenuhi pemeriksaan is_feasible() yang panggil dari src/obejctive.py. Selanjutnya pencarian dilakukan secara berulang dengan membangkitkan seluruh tetangga dari current melalui fungsi all_neighbors( ) pada src/neighbors.py. Setiap tetangga kemudian dievaluasi menggunakan fungsi objective( ) pada src/objective.py.

Dalam setiap iterasi dilakukan pemeriksaan seluruh neighbor dan memperbarui kandidat terbaik hanya jika neighbor_value > best_neighbor_value. Setelah seluruh tetangga diperiksa, algoritma berpindah ke kandidat terbaik dengan memperbarui current dan current_value, menambah iterations sebanyak satu, serta mencatat nilai objektif baru ke dalam history.

Proses berhenti ketika best_neighbor tetap bernilai None setelah seluruh tetangga diperiksa. Hasil pencarian dikembalikan dalam objek SearchResult. Jika tidak ada state feasible yang dikunjungi, final_state dan final_value bernilai None.

2.3.2 Algoritma *Sideways Move Hill-Climbing*

Sideways Move merupakan salah satu jenis algoritma *Hill-Climbing* yang mengatasi kelemahan dari algoritma *Steepest Ascent Hill-Climbing* sebelumnya yang terjebak pada plateau (area dengan nilai objektif yang sama). Algoritma ini memungkinkan neighbor state dengan nilai objektif yang sama dapat menjadi current state dengan sideways move, tapi pergerakan ini dibatasi langkah maksimum (*max_sideways_move*). Tujuannya agar algoritma dapat keluar dari kondisi plateau untuk menemukan solusi yang lebih baik.

Tabel 2.3.2 Algoritma *Sideways Move Hill-Climbing*

Nama File	
hillclimb_sideways.py	
Variabel	
initial_state	State awal pencarian
ship	Data dimensi dan kapasitas berat kapal
vehicles_by_id	Dictionary data kendaraan (*vehicles*) berdasarkan ID
max_sideways_moves	Batas perpindahan sideways berturut-turut, diinisialisasi dengan nilai 10 dan tidak boleh negatif
current, current_value	State saat ini beserta nilai objektifnya
neighbor, neighbor_value	Tetangga yang sedang diperiksa beserta nilai objektifnya
best_neighbor, best_neighbor_value	Kandidat tetangga terbaik untuk perpindahan berikutnya beserta nilainya
sideways_moves	Jumlah perpindahan

best_state, best_value	State feasible terbaik yang ditemukan beserta nilai objektifnya
history	Riwayat nilai objektif state awal dan setiap state yang dikunjungi setelah perpindahan
iterations	Jumlah perpindahan yang dilakukan, termasuk perpindahan sideways
stop_reason	Kondisi alasan berhenti yaitu mencapai local_optimum atau max_sideways_moves.
start_time, duration_s	Waktu awal dan durasi pencarian dalam detik.
Kelas dan Fungsi	
search()	Implementasi logika pencarian yaitu menjalankan iterasi memeriksa tetangga, pemilihan perpindahan, pembatasan sideways move, dan pencatatan hasil pencarian.
all_neighbors()	Menghasilkan kandidat tetangga melalui move yang dilakukan.
objective()	Menghitung total pendapatan dikurangi penalti tumpang tindih, pelanggaran batas kapal, dan kelebihan berat.
is_feasible()	Memeriksa apakah state memenuhi seluruh batasan penempatan dan kapasitas kapal.
time.perf_counter()	Mengukur durasi pencarian.
SearchResult	Kelas data untuk menyimpan state awal, solusi feasible terbaik, nilai akhirnya, riwayat, jumlah perpindahan, durasi, dan *extra dictionary*.
Source Code	

```
import time
from dataclasses import dataclass

from src.model import Ship, State, Vehicle
from src.neighbors import all_neighbors
from src.objective import is_feasible, objective

@dataclass
```

```
class SearchResult:
    initial_state: State
    final_state: State | None
    final_value: float | None
    history: list[float]
    iterations: int
    duration_s: float
    extra: dict

def search(
    initial_state: State,
    ship: Ship,
    vehicles_by_id: dict[str, Vehicle],
    max_sideways_moves: int = 10,
    *,
    on_iteration=None,
) -> SearchResult:
    """Batas sideways dihitung berturut-turut dan direset
setelah uphill."""
    if max_sideways_moves < 0:
        raise ValueError("max_sideways_moves tidak boleh
negatif")

    start_time = time.perf_counter()
    current = initial_state
    current_value = objective(current, ship,
vehicles_by_id)
    history = [current_value]
    iterations = 0
    sideways_moves = 0
    stop_reason = "local_optimum"
    best_state: State | None = None
    best_value: float | None = None
    if is_feasible(current, ship, vehicles_by_id):
        best_state, best_value = current, current_value
    if on_iteration is not None:
        on_iteration(0, current, current_value)

    while True:
        best_neighbor = None
        best_neighbor_value = current_value

        for neighbor in all_neighbors(current, ship,
```

```
vehicles_by_id):
            if neighbor == current:
                continue
            neighbor_value = objective(neighbor, ship,
vehicles_by_id)

            if neighbor_value > best_neighbor_value or (
                best_neighbor is None and neighbor_value
== current_value
            ):
                best_neighbor = neighbor
                best_neighbor_value = neighbor_value

            if (best_value is None or neighbor_value >
best_value) and is_feasible(
                neighbor, ship, vehicles_by_id
            ):
                best_state, best_value = neighbor,
neighbor_value

        if best_neighbor is None:  # local optimum
            break

        if best_neighbor_value == current_value:
            if sideways_moves >= max_sideways_moves:
                stop_reason = "max_sideways_moves"
                break
            sideways_moves += 1
        else:
            sideways_moves = 0

        current, current_value = best_neighbor,
best_neighbor_value
        iterations += 1
        history.append(current_value)
        if on_iteration is not None:
            on_iteration(iterations, current,
current_value)

        if sideways_moves > 0 and sideways_moves >=
max_sideways_moves:
            stop_reason = "max_sideways_moves"
            break
```

```
    duration_s = time.perf_counter() - start_time

    return SearchResult(
        initial_state=initial_state,
        final_state=best_state,
        final_value=best_value,
        history=history,
        iterations=iterations,
        duration_s=duration_s,
        extra={
            "max_sideways_moves": max_sideways_moves,
            "stop_reason": stop_reason,
        },
    )
```

Pencarian dimulai dengan menetapkan initial_state sebagai current, menghitung nilai awal menggunakan objective() yang dipanggil melalui src/objectivee.py, lalu menyimpannya dalam history. Jika state awal memenuhi is_feasible(), state tersebut dicatat sebagai solusi feasible terbaik. Kemudian dilakukan iterasi untuk setiap neighbors dalam all_neighbors( ) dibandingkan nilai objektif nya untuk disimpan state neighbornya yang memiliki nilai objektif yang lebih tinggi. Neighbor dengan nilai objektif tertinggi yang lebih besar daripada current_value akan diprioritaskan. Jika tidak ada peningkatan tetapi tersedia neighbor bernilai sama, neighbor tersebut dapat dipilih sebagai langkah sideways.

Sebelum melakukan langkah sideways, algoritma memeriksa batas max_sideways_moves. Setiap perpindahan yang bernilai sama menambah sideways_moves, sedangkan perpindahan yang meningkatkan nilai meresetnya menjadi nol. Setelah berpindah, algoritma memperbarui current, menambah iterations, dan mencatat nilai objektif ke history. Pencarian berhenti ketika tidak ada kandidat yang dapat diterima atau segera setelah batas sideways berturut-turut tercapai. Selama evaluasi tetangga, algoritma juga mencatat solusi feasible terbaik, termasuk kandidat yang diperiksa tetapi tidak dipilih sebagai langkah berikutnya. Solusi tersebut dikembalikan melalui SearchResult dan jika tidak ada state feasible yang ditemukan, final_state dan final_value bernilai None.

2.3.3 Algoritma *Random Restart Hill-Climbing*

Random Restart adalah salah satu jenis algoritma Hill-climbing yang melakukan pencarian ulang secara acak, algoritma Random Restart Hill-Climbing meningkatkan kemampuan Hill-Climbing dengan mengatasi jebakan local optima. Ketika Hill-Climbing yang sedang berjalan mencapai local optima, algoritma akan menginisiasi state baru secara acak dan memulai proses Hill-Climbing yang baru. Hasilnya adalah solusi terbaik dengan skor terendah yang ditemukan dari semua restart yang telah dilakukan, dibatasi oleh max_restart.

Tabel 2.3.3 Algoritma *Random Restart Hill-Climbing*

Nama File	
hillclimb_randomrestart.py	
Variabel	
initial_state	State awal sebelum melakukan pencarian untuk pencarian pertama
ship	Data dimensi dan kapasitas berat dari kapal
vehicles_by_id	Dictionary kendaraan berdasarkan ID yang dibentuk dari vehicles
rng	Generator bilangan acak untuk membentuk state awal baru
run	Hasil pemanggilan Steepest Ascent (objek hasil datu kali pencarian)
max_restart	Nilai maksimal untuk melakukan restart pencarian → jumlah total pencarian
restart	Indeks pencarian yang sedang dijalankan
best_state, best_value	State feasible terbaik yang ditemukan beserta nilai objektifnya
history	Riwayat nilai objektif state awal dan setiap state yang dikunjungi setelah perpindahan
restart_starts	Indeks awal setiap pencarian dalam gabungan history
iterations_per_rest	Jumlah perpindahan pada masing-masing

art	iterasi
restart_final_values	Nilai solusi feasible terbaik dari setiap pencarian (bisa bernilai None)
total_iterations	Akumulasi jumlah perpindahan seluruh pencarian
start_time	Waktu awal untuk menghitung total durasi eksekusi
Kelas dan Fungsi	
search()	Mengatur pengulangan pencarian, menentukan state awal, menyimpan riwayat, dan memilih hasil terbaik.
steepest_ascent()	Alias fungsi search() yang menjalankan satu pencarian dengan memilih tetangga terbaik sampai tidak ada peningkatan.
random_initial_state()	Membentuk state awal baru dengan pemilihan orientasi, posisi, dan keikutsertaan kendaraan di kapal secara acak.
objective() dan is_feasible()	Digunakan melalui Steepest Ascent untuk menghitung nilai objektif dan memeriksa kelayakan solusi.
perf_counter()	Mengukur total durasi seluruh pencarian.
SearchResult	Menyimpan solusi terbaik, riwayat gabungan, total perpindahan, durasi, dan informasi setiap restart.
Source Code	

```
import random
from time import perf_counter
from dataclasses import dataclass

from src.neighbors import all_neighbors
from src.initial import random_initial_state
from src.objective import is_feasible, objective
from src.model import State, Vehicle, Ship
from src.algo.hillclimb_steepestascent import search as
steepest_ascent
```

```
@dataclass
class SearchResult:
    initial_state: State
    final_state: State | None
    final_value: float | None
    history: list[float]
    iterations: int
    duration_s: float
    extra: dict

def search(
    initial_state: State,
    ship: Ship,
    vehicles: dict[str, Vehicle],
    rng: random.Random,
    max_restart: int = 10,
    *,
    on_iteration=None,
) -> SearchResult:
    """Callback memakai jumlah move kumulatif, termasuk
state awal tiap restart."""

    if max_restart < 1:
        raise ValueError("max_restart minimal harus 1
yak")

    start_time = perf_counter()
    vehicles_by_id = vehicles
    best_state: State | None = None
    best_value: float | None = None
    history: list[float] = []
    restart_starts: list[int] = [] # mencatat posisi awal
tiap restart (penanda mulai restart baru)
    iterations_per_restart: list[int] = []
    restart_final_values: list[float | None] = []  # value
akhir tiap restart
    total_iterations = 0

    def report_iteration(iteration, state, value):
        if on_iteration is not None:
            on_iteration(total_iterations + iteration,
state, value)
```

```
    for restart in range(max_restart):
        start = (
            initial_state
            if restart == 0
            else random_initial_state(ship, vehicles, rng)
        )

        run = steepest_ascent(
            start, ship, vehicles, vehicles_by_id,
            on_iteration=report_iteration if on_iteration
is not None else None,
        )

        # mencatat seluruh history dari setiap restart
        restart_starts.append(len(history)) # mencatat
panjang history sebelum restart baru dimulai
        history.extend(run.history)
        iterations_per_restart.append(run.iterations)
        restart_final_values.append(run.final_value)
        total_iterations += run.iterations


        if (
            run.final_state is not None
            and run.final_value is not None
            and (best_value is None or run.final_value >
best_value)
        ):
            best_state, best_value = run.final_state,
run.final_value

    return SearchResult(
        initial_state=initial_state,
        final_state=best_state,
        final_value=best_value,
        history=history,
        iterations=total_iterations,
        duration_s=perf_counter() - start_time,
        extra={
            "variant": "random_restart",
            "max_restart": max_restart,
            "restart_count": max_restart,
            "iterations_per_restart":
```

```
iterations_per_restart,
            "restart_starts": restart_starts,
            "restart_final_values": restart_final_values,
        },
    )
```

2.3.4 Algoritma *Stochastic Hill-Climbing*

Stochastic merupakan salah satu jenis algoritma Hill-Climbing yang algoritma mengevaluasi seluruh tetangga dan mengumpulkan kandidat yang memiliki nilai objektif lebih tinggi daripada state saat ini. Satu kandidat kemudian dipilih secara acak, bukan berdasarkan peningkatan terbesar seperti pada Steepest Ascent. Pencarian berhenti ketika tidak ada tetangga yang memberikan peningkatan. Selama proses berlangsung, algoritma mencatat solusi feasible terbaik yang ditemukan, riwayat nilai objektif, jumlah perpindahan, dan durasi pencarian.

Tabel 2.3.4 Algoritma *Stochastic Hill-Climbing*

Nama File	
hillclimb_stochastic.py	
Variabel	
initial_state	State awal pencarian.
ship	Data dimensi dan kapasitas berat kapal.
vehicles_by_id	Dictionary data kendaraan berdasarkan ID.
rng	Generator bilangan acak untuk memilih satu kandidat perpindahan.
t	Waktu awal untuk menghitung durasi pencarian.
current, current_value	State saat ini beserta nilai objektifnya.
n, val	Tetangga yang sedang dievaluasi beserta nilai objektifnya.
uphill	Daftar pasangan state dan nilai objektif

	tetangga yang lebih baik daripada state saat ini.
best_state, best_value	State feasible terbaik yang ditemukan beserta nilai objektifnya.
history	Riwayat nilai objektif state awal dan setiap state yang dipilih.
iterations	Jumlah perpindahan yang dilakukan selama pencarian.
Kelas dan Fungsi	
search()	Menjalankan evaluasi tetangga, pemilihan kandidat acak, pencatatan solusi, dan penghentian pencarian.
all_neighbors()	Menghasilkan kandidat melalui pertukaran posisi, perpindahan, pengeluaran kendaraan jika diizinkan, dan rotasi.
objective()	Menghitung total pendapatan dikurangi penalti tumpang tindih, pelanggaran batas kapal, dan kelebihan berat.
is_feasible()	Memeriksa apakah state memenuhi seluruh batasan penempatan dan kapasitas kapal.
rng.choice(uphill)	Memilih satu kandidat secara acak dengan peluang sama untuk setiap elemen dalam uphill.
time.perf_counter()	Mengukur durasi pencarian.
SearchResult	Kelas seragam dengan algoritma lainnya. Sebagai template hasil pencarian (Search())
Source Code	

```
import random
import time
from dataclasses import dataclass

from src.model import Ship, State, Vehicle
from src.neighbors import all_neighbors
from src.objective import is_feasible, objective
```

```
@dataclass
class SearchResult:
    initial_state: State
    final_state: State | None
    final_value: float | None
    history: list[float]
    iterations: int
    duration_s: float
    extra: dict

def search(
    initial_state: State,
    ship: Ship,
    vehicles_by_id: dict[str, Vehicle],
    rng: random.Random, #dipakai untuk memilih 1 tetangga
uphill
    *,
    on_iteration=None,
) -> SearchResult:
    t = time.perf_counter() #untuk mengukur selang waktu
>> dipake hitung durasi
    current = initial_state
    current_value = objective(current, ship,
vehicles_by_id)
    best_state: State | None = None
    best_value: float | None = None

    if is_feasible(current, ship, vehicles_by_id):
        best_state = current
        best_value = current_value

    history = [current_value]
    iterations = 0
    if on_iteration is not None:
        on_iteration(0, current, current_value)
    while True:
        uphill = []
        for n in all_neighbors(current, ship,
vehicles_by_id):
            val = objective(n, ship, vehicles_by_id)
            if val > current_value:
                uphill.append((n, val))
```

```
            # Simpan neighbor feasible
            if (best_value is None or val > best_value)
and is_feasible(
                n, ship, vehicles_by_id
            ):
                best_state, best_value = n, val

        if not uphill:
            break

        current, current_value = rng.choice(uphill)
        iterations += 1
        history.append(current_value)
        if on_iteration is not None:
            on_iteration(iterations, current,
current_value)


    return SearchResult(
        initial_state=initial_state,
        final_state=best_state,
        final_value=best_value,
        history=history,
        iterations=iterations,
        duration_s=time.perf_counter() - t,
        extra={"stop_reason": "local_optimum"},
    )
```

Pencarian dimulai dengan menetapkan initial_state sebagai current dan menghitung nilai awal menggunakan objective(). Nilai tersebut dimasukkan ke history, sedangkan state awal dicatat sebagai solusi terbaik jika memenuhi is_feasible(). Pada setiap iterasi, algoritma membangkitkan seluruh tetangga menggunakan all_neighbors(), lalu mengevaluasi masing-masing kandidat. Tetangga dengan nilai objektif lebih besar secara ketat daripada current_value dimasukkan ke daftar uphill. Dengan demikian, perpindahan bernilai sama maupun lebih rendah tidak diterima.

Jika uphill tidak kosong, algoritma memilih satu kandidat menggunakan rng.choice(uphill), memperbarui state saat ini, menambah iterations, dan mencatat nilainya ke history. Pemilihan ini

tidak memberi bobot lebih besar kepada kandidat dengan peningkatan tertinggi. Jika uphill kosong, pencarian berhenti karena tidak ada peningkatan yang tersedia. Selama evaluasi, algoritma juga menyimpan solusi feasible terbaik dari seluruh tetangga yang diperiksa, termasuk yang tidak dipilih sebagai langkah berikutnya. Hasil dikembalikan melalui SearchResult; apabila tidak ada state feasible yang ditemukan, final_state dan final_value bernilai None.

2.3.5 Algoritma *Simulated Annealing*

Simulated Annealing Algorithm adalah algoritma *local search* probabilistik yang menggabungkan metode *hill-climbing* dengan *random walk* untuk menemukan solusi optimal global dan menghindari jebakan *local optima*. Dengan probabilitas tertentu, algoritma ini dapat menerima dan berpindah ke *neighbor state* yang lebih buruk (nilai objektif lebih tinggi). Suhu dan perubahan energi mengatur probabilitas penerimaan ini:

$$P = e^{\frac{-\Delta E}{T}}$$

Suhu awal yang tinggi memungkinkan algoritma menjelajahi lebih banyak kemungkinan solusi, termasuk menerima solusi yang lebih buruk dari solusi saat ini. Seiring berjalannya waktu, suhu diturunkan secara bertahap melalui skema pendinginan (*cooling rate*). Semakin rendah suhu, semakin kecil kemungkinan algoritma menerima solusi yang lebih buruk, sehingga pencarian menjadi lebih fokus pada solusi terbaik. Proses ini berlanjut hingga suhu mencapai batas minimum suhu.

Dibandingkan dengan Hill-Climbing biasa, Simulated Annealing memiliki peluang lebih besar untuk menemukan solusi optimal global karena dapat keluar dari jebakan optimum lokal.

Tabel 2.3.5.1 Algoritma *Simulated Annealing*

Nama File
simulated_annealing.py

Variabel	
MOVE_WEIGHTS	Bobot peluang dipilihnya tiap jenis langkah tetangga: swap 1, move 1, move_out 0,1, rotate 1
initial_state	State awal sebelum pencarian dimulai (parameter search), juga disimpan di hasil
ship	Data dimensi geladak (lebar, panjang) dan kapasitas berat kapal
vehicles_by_id	Dictionary data kendaraan berdasarkan ID kendaraan
rng	Generator bilangan acak untuk memilih tetangga dan memutuskan penerimaan langkah
on_iteration	Fungsi opsional yang dipanggil setiap iterasi untuk menampilkan progres pencarian
t0	Suhu awal pencarian (default 1000)
alpha	Cooling rate. suhu dikalikan alpha setiap iterasi (default 0,999)
t_min	Suhu minimum; pencarian berhenti jika suhu ≤ t_min (default 0,01)
max_iter	Batas maksimum jumlah iterasi sebagai pengaman (default 20.000)
stuck_iterations	Daftar nomor iterasi tempat setiap kejadian stuck tercatat (untuk menandai stuck pada plot objective)
stuck_k	Jumlah iterasi berturut-turut tanpa perbaikan nilai terbaik yang dihitung sebagai satu kali stuck (default 500)
seed	Nilai awal generator acak, dipakai jika rng tidak diberikan
move_weights	Bobot jenis langkah yang dipakai pada pencarian (default MOVE_WEIGHTS)
start_time	Waktu awal untuk menghitung durasi pencarian
current_state	State yang sedang dikunjungi pada iterasi saat ini
current_value	Nilai objektif dari current_state

best_value	Nilai objektif tertinggi yang pernah dicapai (termasuk state infeasible), dipakai untuk menghitung stuck
best_feasible_state	State feasible terbaik yang pernah ditemukan dan menjadi jawaban akhir
best_feasible_value	Nilai objektif dari best_feasible_state
temperature	Suhu pada iterasi saat ini
value_history	Riwayat nilai objektif current_state di setiap iterasi (untuk plot objective)
probability_history	Riwayat nilai min(1, e^(ΔE/T)) di setiap iterasi (untuk plot e^(ΔE/T))
temperature_history	Riwayat suhu di setiap iterasi
stuck_count	Jumlah kejadian stuck selama pencarian
iterations_without_improvement	Penghitung iterasi berturut-turut tanpa perbaikan nilai terbaik
iteration	Nomor iterasi yang sedang berjalan; di akhir menjadi total iterasi
neighbor	State tetangga acak yang dibangkitkan dari current_state
neighbor_value	Nilai objektif dari neighbor
delta	Selisih nilai (ΔE = neighbor_value – current_value)
probability	Peluang neighbor diterima: 1 jika ΔE ≥ 0, e^(ΔE/T) jika ΔE < 0
random_number	Bilangan acak 0–1 yang dibandingkan dengan probability untuk memutuskan neighbor diterima atau tidak
duration	Total durasi pencarian dalam detik
stop_reason	Alasan pencarian berhenti: "t_min", "max_iter", atau "no_neighbor"
params	Dictionary parameter yang dipakai pada pencarian (t0, alpha, t_min, max_iter, stuck_k, seed, move_weights)
extra	Dictionary ringkasan hasil (stop_reason, stuck_count, dan parameter SA) untuk visualisasi

result	Objek SearchResult berisi seluruh hasil pencarian
placements	Dictionary posisi setiap kendaraan pada empty_state (semua di luar kapal)
vehicle_id	ID kendaraan yang sedang diproses saat membentuk empty_state
final_state	State feasible terbaik hasil pencarian (jawaban akhir)
final_value	Nilai objektif dari final_state
last_state	State terakhir yang dikunjungi saat pencarian berhenti
history	Riwayat nilai objektif per iterasi (dari value_history)
probabilities	Riwayat nilai e^(ΔE/T) per iterasi (dari probability_history)
temperatures	Riwayat suhu per iterasi (dari temperature_history)
iterations	Total iterasi yang dijalankan
duration_s	Durasi pencarian dalam detik
Kelas dan Fungsi	
SearchResult	Kelas seragam dengan algoritma lainnya. Sebagai template hasil pencarian (Search())
empty_state()	Membentuk state dengan semua kendaraan di luar kapal; dipakai sebagai jawaban cadangan jika tidak pernah ditemukan state feasible
search()	Implementasi logika Simulated Annealing dengan membangkitkan tetangga acak, menerima atau menolaknya dengan peluang e^(ΔE/T), menurunkan suhu, mencatat riwayat dan stuck, lalu mengembalikan SearchResult
random_neighbor()	(dari neighbors.py) Membangkitkan satu tetangga acak dengan langkah swap, move, move_out, atau rotate sesuai bobot

objective()	(dari objective.py) Menghitung nilai objektif state: total fee dikurangi penalti pelanggaran batasan
is_feasible()	(dari objective.py) Mengecek apakah state memenuhi semua batasan (tidak bertumpuk, tidak keluar geladak, tidak melebihi kapasitas berat)
math.exp()	Menghitung e^(ΔE/T) untuk peluang penerimaan langkah yang memburuk
time.perf_counter()	Mengambil waktu untuk menghitung durasi pencarian

Source Code

```
import math
import random
import time
from dataclasses import dataclass
from src.model import Orientation, Placement, Ship, State,
Vehicle
from src.neighbors import random_neighbor
from src.objective import is_feasible, objective

MOVE_WEIGHTS = {"swap": 1, "move": 1, "move_out": 0.1,
"rotate": 1}

@dataclass
class SearchResult:
    initial_state: State
    final_state: State
    final_value: float
    last_state: State
    history: list[float]
    probabilities: list[float]
    temperatures: list[float]
    iterations: int
    stuck_count: int
    stuck_iterations: list[int]
    duration_s: float
    params: dict
    extra: dict
```

```
def empty_state(vehicles_by_id: dict[str, Vehicle]) -> State:
    placements = {}
    for vehicle_id in vehicles_by_id:
        placements[vehicle_id] = Placement(
            vehicle_id=vehicle_id,
            x=None,
            y=None,
            orientation=Orientation.HORIZONTAL,
        )
    return State(placements)

def search(
    initial_state: State,
    ship: Ship,
    vehicles_by_id: dict[str, Vehicle],
    rng: random.Random | None = None,
    *,
    on_iteration=None,
    t0: float = 1000.0,
    alpha: float = 0.999,
    t_min: float = 0.01,
    max_iter: int = 20000,
    stuck_k: int = 500,
    seed: int | None = None,
    move_weights: dict[str, float] | None = None,
) -> SearchResult:
    if move_weights is None:
        move_weights = MOVE_WEIGHTS

    #rng dari main.py dipake langsung, kalo nggak ada, dibuat
dari seed
    if rng is None:
        rng = random.Random(seed)
    start_time = time.perf_counter()

    current_state = initial_state
    current_value = objective(current_state, ship,
vehicles_by_id)
    best_value = current_value

    if is_feasible(current_state, ship, vehicles_by_id):
        best_feasible_state = current_state
        best_feasible_value = current_value
```

```
    else:
        best_feasible_state = None
        best_feasible_value = -math.inf

    temperature = t0
    value_history = [current_value]
    probability_history = []
    temperature_history = []
    stuck_count = 0
    stuck_iterations = []
    iterations_without_improvement = 0
    iteration = 0

    if on_iteration is not None:
        on_iteration(0, current_state, current_value)

    while iteration < max_iter and temperature > t_min:
        neighbor = random_neighbor(current_state, ship,
vehicles_by_id, rng, move_weights)
        if neighbor is None:
            break
        iteration = iteration + 1

        neighbor_value = objective(neighbor, ship,
vehicles_by_id)
        delta = neighbor_value - current_value

        if delta >= 0:
            probability = 1.0
        else:
            probability = math.exp(delta / temperature)

        random_number = rng.random()
        if random_number <= probability:
            current_state = neighbor
            current_value = neighbor_value
        if current_value > best_value:
            best_value = current_value
            iterations_without_improvement = 0
        else:
            iterations_without_improvement =
iterations_without_improvement + 1
            if iterations_without_improvement == stuck_k:
```

```
                stuck_count = stuck_count + 1
                stuck_iterations.append(iteration)
                iterations_without_improvement = 0

        if current_value > best_feasible_value:
            if is_feasible(current_state, ship,
vehicles_by_id):
                best_feasible_state = current_state
                best_feasible_value = current_value

        value_history.append(current_value)
        probability_history.append(probability)
        temperature_history.append(temperature)

        if on_iteration is not None:
            on_iteration(iteration, current_state,
current_value)

        temperature = temperature * alpha

    if best_feasible_state is None:
        best_feasible_state = empty_state(vehicles_by_id)
        best_feasible_value = objective(best_feasible_state,
ship, vehicles_by_id)

    duration = time.perf_counter() - start_time

    if temperature <= t_min:
        stop_reason = "t_min"
    elif iteration >= max_iter:
        stop_reason = "max_iter"
    else:
        stop_reason = "no_neighbor"

    # ringkasan untuk visualize.py, bentuknya sama dengan
extra di HC/GA
    extra = {
        "stop_reason": stop_reason,
        "stuck_count": stuck_count,
        "stuck_k": stuck_k,
        "t0": t0,
        "alpha": alpha,
        "t_min": t_min,
```

```
        "max_iter": max_iter,
    }

    params = {
        "t0": t0,
        "alpha": alpha,
        "t_min": t_min,
        "max_iter": max_iter,
        "stuck_k": stuck_k,
        "seed": seed,
        "move_weights": dict(move_weights),
    }

    result = SearchResult(
        initial_state=initial_state,
        final_state=best_feasible_state,
        final_value=best_feasible_value,
        last_state=current_state,
        history=value_history,
        probabilities=probability_history,
        temperatures=temperature_history,
        iterations=iteration,
        stuck_count=stuck_count,
        stuck_iterations=stuck_iterations,
        duration_s=duration,
        params=params,
        extra=extra,
    )
    return result
```

2.3.6 Algoritma *Genetic*

Genetic Algorithm adalah algoritma yang memanfaatkan skema pewarisan sifat layaknya rekayasa genetika. *Core operations* pada algoritma ini adalah pencarian dua induk untuk setiap generasi, lalu *crossover*, yaitu penyilangan sifat antara dua induk, lalu *mutation*, yaitu perubahan *unexpected* dengan probabilitas kecil tertentu.

Tabel 2.3.6.1 *Genetic Algorithm*

Nama File

genetic_algorithm.py	
Variabel	
MUTATION_PROBABILITY	Peluang seorang anak dimutasi (0,1)
MOVE_WEIGHTS	Bobot jenis langkah saat mutasi: swap 1, move 0,2, move_out 0,4, rotate 0,2
TOURNAMENT_SIZE	Jumlah peserta tournament selection (3)
ELITE_COUNT	Jumlah individu terbaik yang langsung dibawa ke generasi berikutnya (2)
initial_state	State awal opsional yang dimasukkan sebagai individu pertama
ship	Data dimensi geladak dan kapasitas berat kapal
vehicles_by_id	Dictionary data kendaraan berdasarkan ID
rng	Generator bilangan acak untuk seleksi, crossover, dan mutasi
population_size	Jumlah individu dalam satu populasi (default 50)
max_iter	Jumlah generasi yang dijalankan (default 300)
seed	Seed yang dicatat di hasil
on_iteration	Fungsi opsional yang dipanggil setiap generasi untuk menampilkan progres
population, fitness	Daftar individu dan nilai objektifnya; indeks keduanya sejajar
initial_result, initial_value	Individu terbaik populasi awal beserta nilainya

Kelas dan Fungsi	
SearchResult	Dataclass untuk menyimpan state awal, state feasible terbaik, nilai akhir, riwayat, jumlah iterasi, durasi, dan extra (history_avg dan parameter)
initialize_population()	Membentuk populasi awal dari initial_state (jika ada) dan state acak yang feasible
evaluate_population()	Menghitung fitness setiap individu dengan objective()
select_parent()	Memilih induk dengan tournament selection berukuran TOURNAMENT_SIZE
crossover()	Menyilangkan dua induk dengan garis potong acak pada geladak dan menghasilkan anak yang feasible
mutate()	Menerapkan satu langkah tetangga acak dari random_neighbor() pada anak

Source Code

```
import dataclasses
import random
from time import perf_counter

from src.model import Orientation, Ship, State, Vehicle,
Placement
from src.initial import random_initial_state
from src.neighbors import random_neighbor
from src.objective import is_feasible, objective

MUTATION_PROBABILITY = 0.1

MOVE_WEIGHTS = {"swap": 1, "move": 0.2, "move_out": 0.4,
"rotate": 0.2}

# k numbers of contestants in select_parent()
TOURNAMENT_SIZE = 3
```

```
# Elitism (preserve child superior)
ELITE_COUNT = 2


@dataclasses.dataclass
class SearchResult:
    initial_state: State
    final_state: State
    final_value: float
    history: list[float]
    iterations: int
    duration_s: float
    extra: dict[str, object]


def initialize_population(
    initial_state: State | None,
    ship: Ship,
    vehicles_by_id: dict[str, Vehicle],
    rng: random.Random,
    population_size: int,
) -> list[State]:

    population = []
    if initial_state is None:
        for i in range (population_size):
            population.append(random_initial_state(ship,
vehicles_by_id, rng))
    else:

population.append(State(dict(initial_state.placements))) #
new State dari initial_state
        for i in range (population_size - 1):
            population.append(random_initial_state(ship,
vehicles_by_id, rng))
    return population

def evaluate_population(
    population: list[State],
    ship: Ship,
    vehicles_by_id: dict[str, Vehicle],
) -> list[float]:
```

```
    fitness = [ objective(s, ship, vehicles_by_id) for s
in population]
    return fitness


def select_parent(
    population: list[State], fitness: list[float], rng:
random.Random
) -> State:

    # tournament selection with k contestants
    n = len(population)
    if n == 0 or n != len(fitness):
        raise ValueError("populasi kosong atau panjang
tidak sama dengan fitness")
    if n == 1:
        return population[0]

    k = min(n, TOURNAMENT_SIZE) # handling if population <
tournament_size
    tourney_idx = []

    # population & fitness paralel, ambil index fitness.
repeating index allowed
    for _ in range (k):
        tourney_idx.append(rng.randrange(n))
    parent_idx: int = max(tourney_idx, key = lambda i:
fitness[i]) # index max dari fitness

    return population[parent_idx]

def crossover(
    parent_a: State,
    parent_b: State,
    ship: Ship,
    vehicles_by_id: dict[str, Vehicle],
    rng: random.Random,
) -> State:
    # Acak induk mana yang dapet prioritas area
    if rng.random() < 0.5:
        parent_a, parent_b = parent_b, parent_a
```

```
    axis = rng.choice(["x", "y"])
    extent = ship.w if axis == "x" else ship.l
    cut = rng.randint(1, extent - 1) if extent > 1 else
extent
    ids = list(vehicles_by_id)
    rng.shuffle(ids)

    # Ambil vehicle A yang keseluruhannya di kiri/atas
garis potong
    candidates = []
    for vehicle_id in ids:
        p = parent_a.placements[vehicle_id]
        if not p.is_inside():
            continue
        vw, vl =
vehicles_by_id[vehicle_id].size_taken(p.orientation)
        edge = p.x + vw if axis == "x" else p.y + vl
        if edge <= cut:
            candidates.append(p)

    # then try B, terus sisanya A yang masih bisa masuk
    for parent in (parent_b, parent_a):
        for vehicle_id in ids:

candidates.append(parent.placements[vehicle_id])

    placements = {}
    for vehicle_id in vehicles_by_id:
        p = parent_a.placements[vehicle_id]
        placements[vehicle_id] = Placement(vehicle_id,
None, None, p.orientation)

    occupied = set()
    total_weight = 0
    for p in candidates:
        if not p.is_inside() or
placements[p.vehicle_id].is_inside():
            continue
        vehicle = vehicles_by_id[p.vehicle_id]
        vw, vl = vehicle.size_taken(p.orientation)
        if not ship.in_boundary(p.x, p.y, vw, vl):
            continue
        if not ship.within_capacity(total_weight +
```

```
vehicle.weight):
            continue
        cells = p.grids(vehicle)
        if any(cell in occupied for cell in cells):
            continue

        placements[p.vehicle_id] = p
        occupied.update(cells)
        total_weight += vehicle.weight

    return State(placements)


def mutate(
    child: State, ship: Ship, vehicles_by_id: dict[str,
Vehicle], rng: random.Random
) -> State:

    potential_mutation = random_neighbor(child, ship,
vehicles_by_id, rng, MOVE_WEIGHTS)
    return potential_mutation if potential_mutation is not
None else child


def update_best_feasible(
    population: list[State],
    fitness: list[float],
    ship: Ship,
    vehicles_by_id: dict[str, Vehicle],
    best_state: State,
    best_value: float,
) -> tuple[State, float]:

    current_best_state = best_state
    current_best_value = best_value

    for i in range (len(population)):
        if fitness[i] > current_best_value and
is_feasible(population[i], ship, vehicles_by_id):
            current_best_state =
State(dict(population[i].placements))
            current_best_value = fitness[i]
```

```
    return current_best_state, current_best_value


def search(
    initial_state: State | None,
    ship: Ship,
    vehicles_by_id: dict[str, Vehicle],
    rng: random.Random,
    *, # pisahin so that 3 args di bawah perlu explicit
name di parameter, misal: search(..., seed = 67)
    population_size: int = 50,
    max_iter: int = 300,
    seed: int | None = None,
    on_iteration=None,  # callback opsional (iterasi,
state terbaik generasi, value)
) -> SearchResult:
    if isinstance(population_size, bool) or not
isinstance(population_size, int) or population_size < 1:
        raise ValueError("population_size harus integer >=
1")

    if isinstance(max_iter, bool) or not
isinstance(max_iter, int) or max_iter < 0:
        raise ValueError("max_iter harus integer >= 0")

    for key, vehicle in vehicles_by_id.items():
        if key != vehicle.id:
            raise ValueError(f"Key {key!r} tidak sama
dengan vehicle.id {vehicle.id!r}")

    start_time = perf_counter()

    # start new population states
    population = initialize_population(initial_state,
ship, vehicles_by_id, rng, population_size)

    # list of weights for each state di population
    fitness = evaluate_population(population, ship,
vehicles_by_id)

    current_idx = fitness.index(max(fitness)) # since
fitness paralel dgn population, idxnya sama
    initial_result =
```

```
State(dict(population[current_idx].placements)) # best
current population
    initial_value = fitness[current_idx]

    best_state = State({vehicle_id: Placement(vehicle_id,
None, None, Orientation.HORIZONTAL) for vehicle_id in
vehicles_by_id})
    best_value = objective(best_state, ship,
vehicles_by_id)
    best_state, best_value =
update_best_feasible(population, fitness, ship,
vehicles_by_id, best_state, best_value)

    history = [max(fitness)]
    history_avg = [sum(fitness) / len(fitness)]
    if on_iteration is not None:
        on_iteration(0, population[current_idx],
initial_value)

    for i in range(max_iter):
        elite_n = min(ELITE_COUNT, population_size - 1) #
count agar populasi 1 masih bisa punya child
        top_child = sorted(range(population_size),
key=lambda i: fitness[i], reverse=True)[:elite_n]
        new_population =
[State(dict(population[i].placements)) for i in top_child]
        while len(new_population) < population_size:
            a = select_parent(population, fitness, rng)
            b = select_parent(population, fitness, rng)
            child = crossover(a, b, ship, vehicles_by_id,
rng)
            if rng.random() < MUTATION_PROBABILITY:
                child = mutate(child, ship,
vehicles_by_id, rng)
            new_population.append(child)

        population = new_population
        fitness = evaluate_population(population, ship,
vehicles_by_id)
        best_state, best_value =
update_best_feasible(population, fitness, ship,
vehicles_by_id, best_state, best_value)
        history.append(max(fitness))
```

```
        history_avg.append(sum(fitness) / len(fitness))
        if on_iteration is not None:
            current_idx = fitness.index(max(fitness))
            on_iteration(i + 1, population[current_idx],
fitness[current_idx])


    if not is_feasible(best_state, ship, vehicles_by_id):
        raise RuntimeError("best_state not feasible
(bug)")
    final_value = objective(best_state, ship,
vehicles_by_id)
    duration = perf_counter() - start_time

    return SearchResult(
        initial_state=initial_result,
        final_state=best_state,
        final_value=final_value,
        history=history,
        iterations=max_iter,
        duration_s=duration,
        extra={
            "history_avg": history_avg,
            "population_size": population_size,
            "max_iter": max_iter,
            "mutation_probability": MUTATION_PROBABILITY,
            "tournament_size": TOURNAMENT_SIZE,
            "elite_count": ELITE_COUNT,
            "move_weights": MOVE_WEIGHTS,
            "seed": seed,
            "initial_value": initial_value
        },
    )
```

III. Analisis dan Hasil Eksperimen

3.1 Algoritma *Steepeset Ascent Hill Climbing*

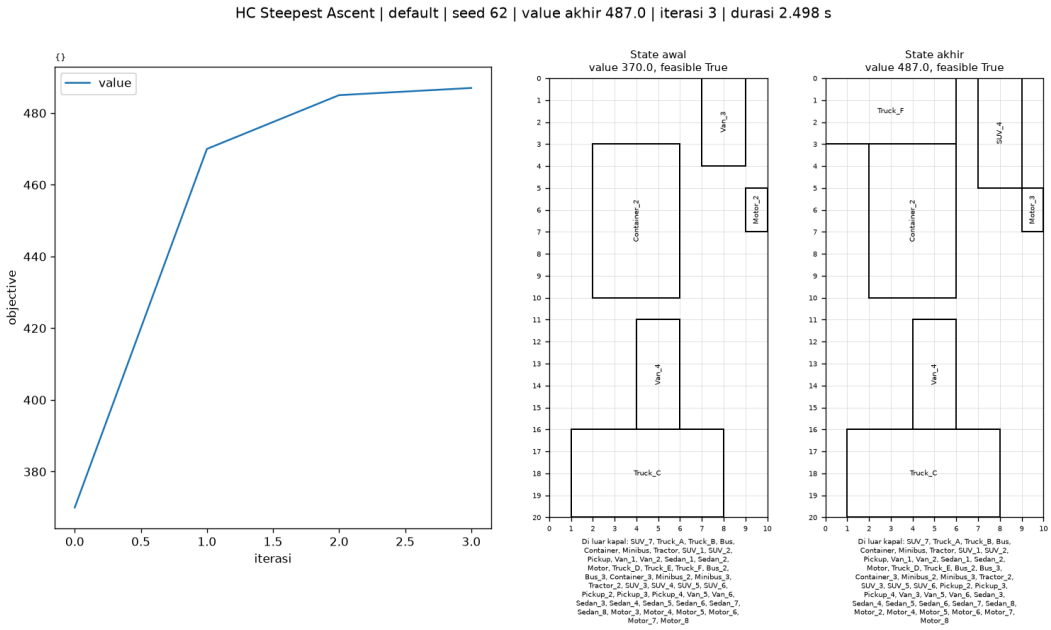
3.1.1 Eksperimen Pertama

input	Seed: 62

Hasil Eksperimen	`--- HC Steepest Ascent ---`
	`Simpan visualisasi? [y/N]: y`
	`Iterasi 0 | value 370.0 | feasible | inside: Truck_C, Container_2, Van_3, Van_4, Motor_2`
	`Iterasi 1 | value 470.0 | feasible | inside: Truck_C, Truck_F, Container_2, Van_3, Van_4, Motor_2`
	`Iterasi 2 | value 485.0 | feasible | inside: Truck_C, Truck_F, Container_2, SUV_4, Van_4, Motor_2`
	`Iterasi 3 | value 487.0 | feasible | inside: Truck_C, Truck_F, Container_2, SUV_4, Van_4, Motor_3`
	`Value awal : 370.0`
	`Value akhir : 487.0`
	`Feasible : True`
	`Iterasi : 3`
	`Durasi : 2.498 s`
	`Visualisasi : output\hc_steepest_ascent\seed62.png`

Durasi	2.498 s
Banyak Iterasi	3

Berikut merupakan hasil visualisasi dari eksperimen yang merepresentasikan *initial state, final state,* plot nilai objective function terhadap banyak iterasi yang telah dilewati.

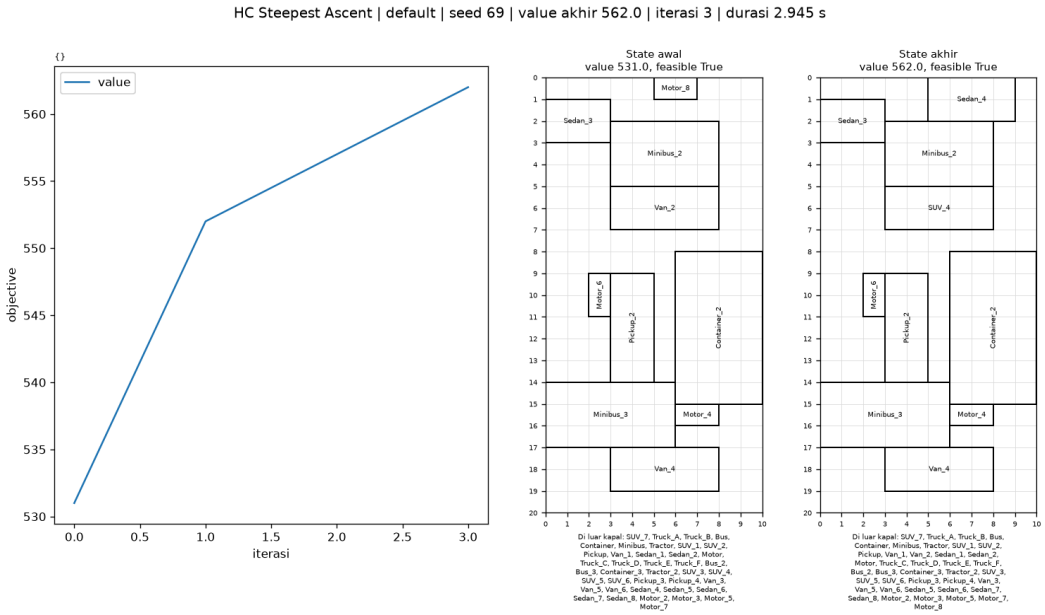


3.1.2 Eksperimen Kedua

input	Seed: 69

Hasil Eksperimen	Iterasi 0 | value 531.0 | feasible | inside: Van_2, Container_2, Minibus_2, Minibus_3, Pickup_2, Van_4, Sedan_3, Motor_4, Motor_6, Motor_8 Iterasi 1 | value 552.0 | feasible | inside: Van_2, Sedan_2, Container_2, Minibus_2, Minibus_3, Pickup_2, Van_4, Sedan_3, Motor_4, Motor_6 Iterasi 2 | value 557.0 | feasible | inside: Sedan_2, Container_2, Minibus_2, Minibus_3, SUV_4, Pickup_2, Van_4, Sedan_3, Motor_4, Motor_6 Iterasi 3 | value 562.0 | feasible | inside: Container_2, Minibus_2, Minibus_3, SUV_4, Pickup_2, Van_4, Sedan_3, Sedan_4, Motor_4, Motor_6 Value awal : 531.0 Value akhir : 562.0 Feasible : True Iterasi : 3 Durasi : 2.945 s Visualisasi : output\hc_steepest_ascent\seed69.png
Durasi	2.945 s
Banyak iterasi	3

Berikut merupakan hasil visualisasi dari eksperimen yang merepresentasikan *initial state*, *final state*, plot nilai objective function terhadap banyak iterasi yang telah dilewati.

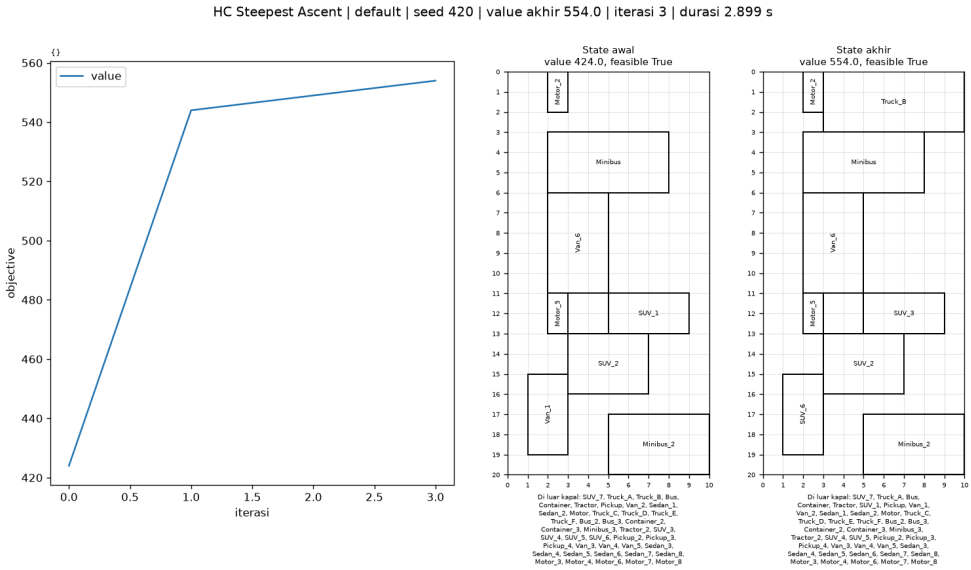


3.1.3 Eksperimen Ketiga

input	Seed: 420
Hasil Eksperimen	--- HC Steepest Ascent --- Simpan visualisasi? [y/N]: Y Iterasi 0 | value 424.0 | feasible | inside: Minibus, SUV_1, SUV_2, Van_1, Minibus_2, Van_6, Motor_2, Motor_5 Iterasi 1 | value 544.0 | feasible | inside: Truck_B, Minibus, SUV_1, SUV_2, Van_1, Minibus_2, Van_6, Motor_2, Motor_5 Iterasi 2 | value 549.0 | feasible | inside: Truck_B, Minibus, SUV_1, SUV_2, Minibus_2, SUV_6, Van_6, Motor_2, Motor_5 Iterasi 3 | value 554.0 | feasible | inside: Truck_B, Minibus, SUV_2, Minibus_2, SUV_3, SUV_6, Van_6, Motor_2, Motor_5 Value awal : 424.0 Value akhir : 554.0 Feasible : True Iterasi : 3 Durasi : 2.899 s Visualisasi : output\hc_steepest_ascent\seed420.png
Durasi	2.899 s

Banyak Iterasi	3

Berikut merupakan hasil visualisasi dari eksperimen yang merepresentasikan *initial state, final state,* plot nilai objective function terhadap banyak iterasi yang telah dilewati.



3.1.4 Analisis Singkat

Eksperimen Hill-Climbing Steepest Ascent pada tiga seed yaitu 62, 69 dan 420 menghasilkan peningkatan objective dari rata-rata 441,67 menjadi 534,33, dengan seluruh solusi akhir memenuhi *constraint* (*feasible*). Hasil terbaik diperoleh pada seed 69 dengan objective 562, sedangkan peningkatan terbesar sebesar 130 diperoleh pada seed 420. Seluruh eksperimen berhenti setelah tiga iterasi, dengan rata-rata durasi 2,781 detik. Perbedaan objective akhir menunjukkan bahwa hasil algoritma dipengaruhi oleh kondisi awal (initial state).

Berdasarkan hasil ketiga eksperimen tersebut, terlihat bahwa peningkatan terbesar umumnya terjadi pada langkah awal, terutama pada seed 62 dan 420, kemudian diikuti peningkatan yang lebih kecil. Pola ini sesuai dengan cara kerja steepest ascent, yaitu selalu memilih tetangga yang memberikan kenaikan nilai paling besar pada langkah saat itu.

Algoritma menunjukkan konsistensi tinggi pada jumlah langkah pencarian, tetapi kualitas solusi akhirnya masih dipengaruhi oleh

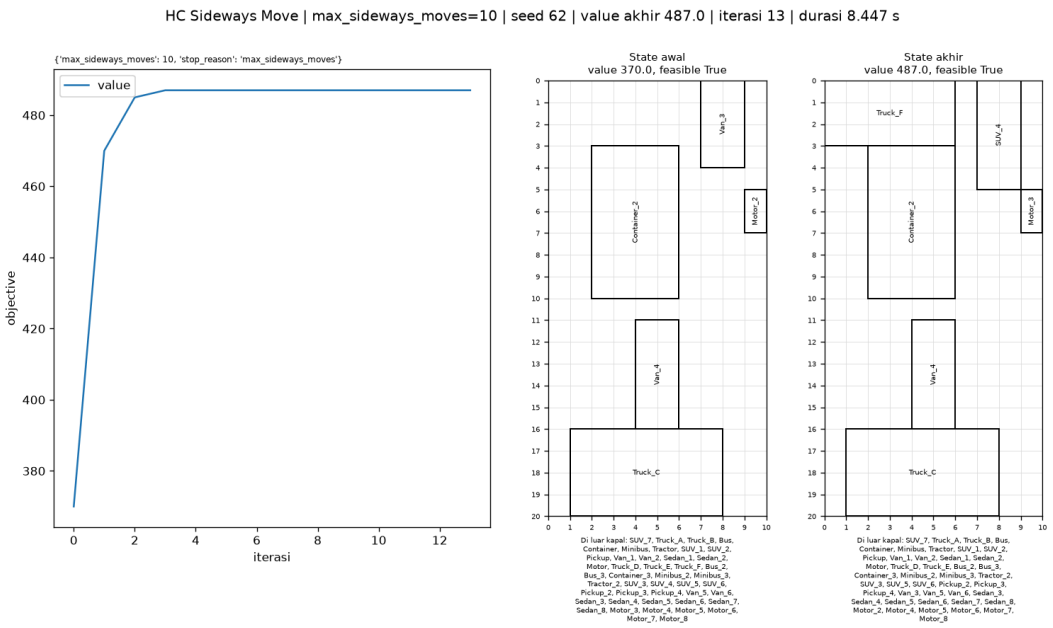
kondisi awal. Nilai objective function yang dihasilkan berada pada rentang 487–562, dengan seluruh pengujian berhenti setelah 3 iterasi. Kesamaan jumlah iterasi menunjukkan bahwa algoritma cepat mencapai kondisi ketika tidak ada lagi tetangga langsung yang memberikan peningkatan nilai. Pola grafik memperlihatkan peningkatan yang relatif besar pada awal pencarian, kemudian diikuti perbaikan yang lebih kecil hingga berhenti pada local optimum. Perilaku ini sesuai dengan sifat deterministik steepest ascent yang selalu memilih tetangga dengan nilai terbaik pada setiap langkah. Namun, pemilihan keuntungan terbesar dalam satu langkah tidak menjamin hasil akhir terbaik, sebagaimana terlihat dari algoritma lain yang mampu menghasilkan nilai lebih tinggi pada seed yang sama. Dengan rata-rata nilai akhir 534,33 dan waktu 2,781 detik, steepest ascent menjadi metode tercepat dalam pengujian ini, tetapi memiliki keterbatasan dalam menjelajahi alternatif solusi di luar jalur perbaikan langsung.

3.2 Algoritma Sideways Move Hill Climbing

3.2.1 Eksperimen Pertama

input	Seed: 62 Max_sideways_moves: 10
Hasil Eksperimen	Iterasi 11 | value 487.0 | feasible | inside: Van_2, Truck_C, Truck_F, Container_2, SUV_4, Motor_3 Iterasi 12 | value 487.0 | feasible | inside: Van_2, Truck_C, Truck_F, Container_2, SUV_4, Motor_3 Iterasi 13 | value 487.0 | feasible | inside: Van_2, Truck_C, Truck_F, Container_2, SUV_4, Motor_3 Value awal : 370.0 Value akhir : 487.0 Feasible : True Iterasi : 13 Durasi : 8.447 s Visualisasi : output\hc_sideways_move\seed62.png
Durasi	8.447 s
Banyak Iterasi	13

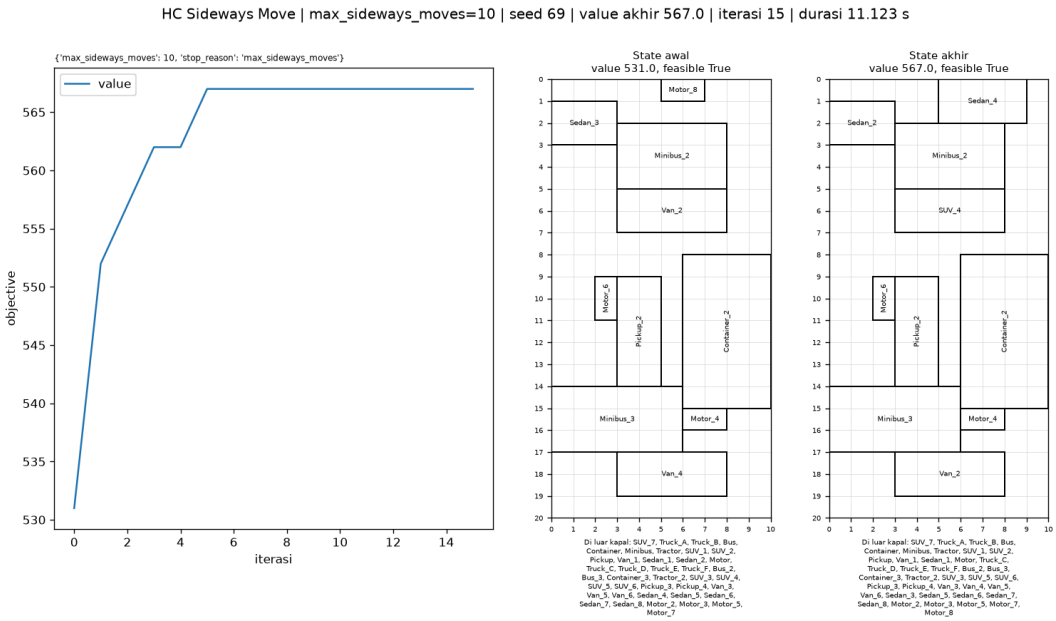
Berikut merupakan hasil visualisasi dari eksperimen yang merepresentasikan *initial state, final state,* plot nilai objective function terhadap banyak iterasi yang telah dilewati.



3.2.2 Eksperimen Kedua

input	Seed: 69 Max_sideways_moves: 10
Hasil Eksperimen	Iterasi 13 | value 567.0 | feasible | inside: Van_2, Sedan_2, Container_2, Minibus_2, Minibus_3, S Iterasi 14 | value 567.0 | feasible | inside: Van_2, Sedan_2, Container_2, Minibus_2, Minibus_3, S Iterasi 15 | value 567.0 | feasible | inside: Van_2, Sedan_2, Container_2, Minibus_2, Minibus_3, S Value awal : 531.0 Value akhir : 567.0 Feasible : True Iterasi : 15 Durasi : 11.123 s Visualisasi : output\hc_sideways_move\seed69.png
Durasi	11.123 s
Banyak Iterasi	15

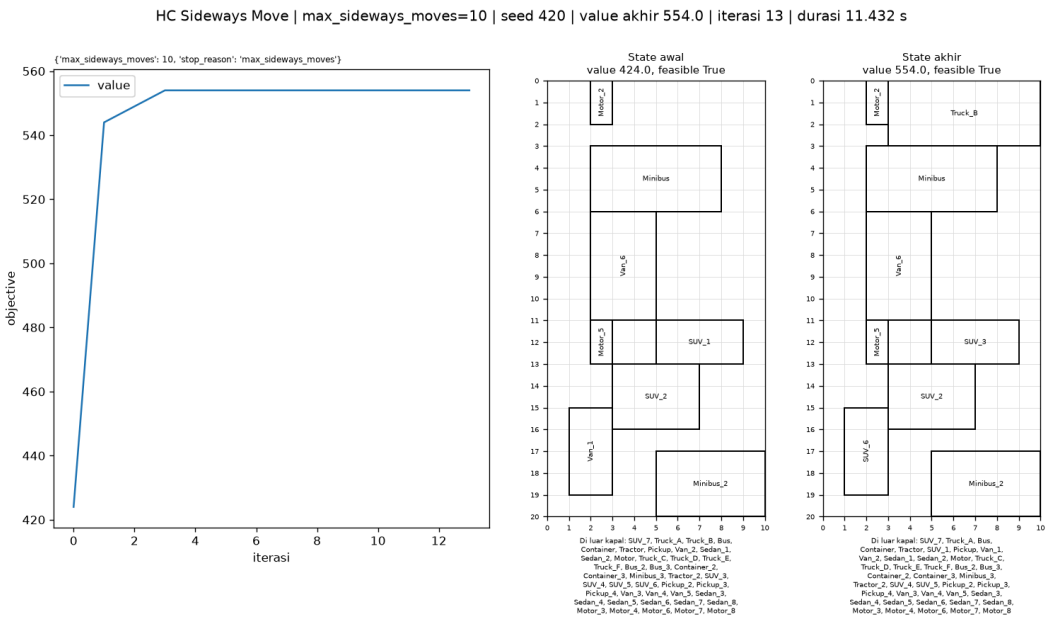
Berikut merupakan hasil visualisasi dari eksperimen yang merepresentasikan *initial state*, *final state*, plot nilai objective function terhadap banyak iterasi yang telah dilewati.



3.2.3 Eksperimen Ketiga

input	Seed: 420 Max_sideways_moves: 10
Hasil Eksperimen	Iterasi 9 | value 554.0 | feasible | inside: Truck_B, Minibus, SUV_2, Minibus_2, SUV Iterasi 10 | value 554.0 | feasible | inside: Truck_B, Minibus, SUV_2, Minibus_2, SUV Iterasi 11 | value 554.0 | feasible | inside: Truck_B, Minibus, SUV_2, Minibus_2, SUV Iterasi 12 | value 554.0 | feasible | inside: Truck_B, Minibus, SUV_2, Minibus_2, SUV Iterasi 13 | value 554.0 | feasible | inside: Truck_B, Minibus, SUV_2, Minibus_2, SUV Value awal : 424.0 Value akhir : 554.0 Feasible : True Iterasi : 13 Durasi : 11.432 s Visualisasi : output\hc_sideways_move\seed420.png
Durasi	11.432 s
Banyak Iterasi	13

Berikut merupakan hasil visualisasi dari eksperimen yang merepresentasikan *initial state*, *final state*, plot nilai objective function terhadap banyak iterasi yang telah dilewati.



3.2.4 Analisis Singkat

Eksperimen Sideways Move menghasilkan rata-rata objective akhir sebesar 536,00, dengan rata-rata durasi 10,334 detik. Dibandingkan Steepest Ascent, peningkatan hanya terjadi pada seed 69, dari 562 menjadi 567, sedangkan dua seed lainnya menghasilkan nilai yang sama. Hal ini menunjukkan bahwa langkah sideways dapat membantu pencarian melewati plateau, tetapi tidak selalu menghasilkan solusi yang lebih baik. Pada eksperimen ini, tambahan langkah pencarian belum memberikan peningkatan kualitas yang signifikan.

Bedasarkan hasil analisis pada eksperimen yang telah dilakukan, eksperimen menunjukkan peningkatan kualitas solusi yang terbatas dibandingkan steepest ascent, meskipun menggunakan jumlah iterasi dan waktu pencarian yang lebih besar. Dengan batas 10 sideways move berturut-turut, algoritma menghasilkan nilai akhir pada rentang 487–567. Pengujian seed 62 menghasilkan nilai 487 dalam 13 iterasi dengan durasi 8,447 detik, seed 69 menghasilkan 567 dalam 15 iterasi dengan durasi 11,123 detik, dan seed 420 menghasilkan 554 dalam 13 iterasi dengan durasi 11,432 detik. Pada seed 62 dan 420, hasil akhirnya sama dengan steepest ascent, sehingga tambahan perpindahan bernilai sama belum memberikan keuntungan terhadap kualitas solusi. Sebaliknya, pada seed 69, grafik menunjukkan perpindahan mendatar pada nilai 562 yang kemudian diikuti peningkatan menjadi 567.

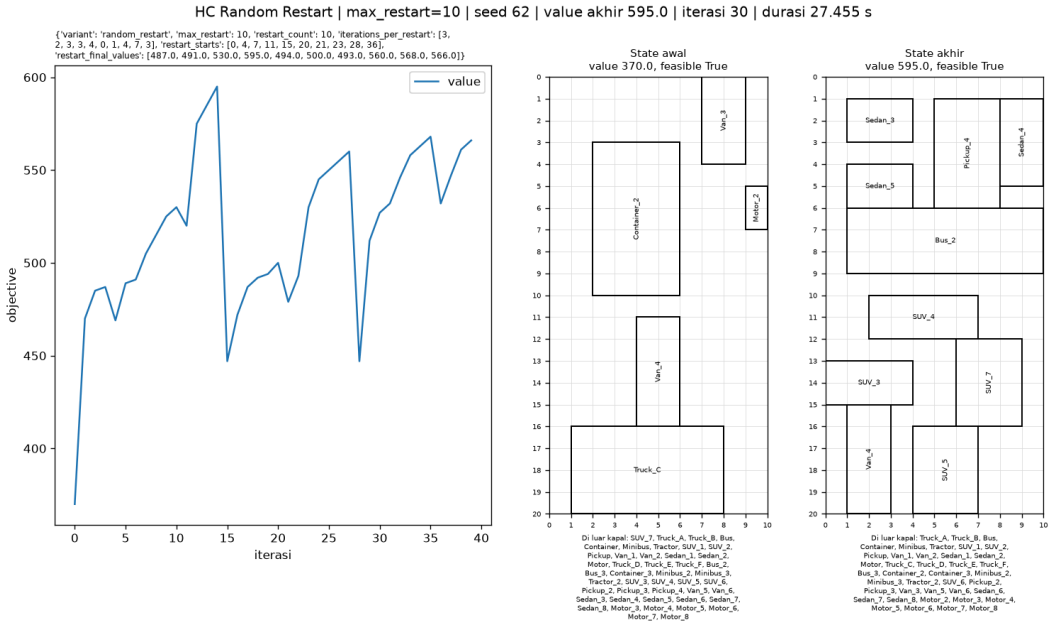
Hal ini menunjukkan bahwa sideways move dapat membuka jalur perbaikan melalui susunan berbeda dengan nilai objektif yang sama. Namun, seluruh pengujian akhirnya berhenti karena mencapai batas sideways, bukan karena berhasil menemukan solusi terbaik secara keseluruhan. Dengan rata-rata nilai akhir 536 dan waktu 10,334 detik, tambahan waktu pencarian belum diimbangi peningkatan hasil yang sebanding. Efektivitas sideways move pada pengujian ini bergantung pada apakah perpindahan di area bernilai sama benar-benar mengarah ke peluang perbaikan berikutnya.

3.3 Algoritma Random Restart Hill Climbing

3.3.1 Eksperimen Pertama

input	Seed: 62 max_restart: 10
Hasil Eksperimen	Value awal : 370.0 Value akhir : 595.0 Feasible : True Iterasi : 30 Durasi : 27.455 s Visualisasi : output\hc_random_restart\seed62.png
Durasi	27.455 s
Banyak restart	10
Iterasi per restart	3, 2, 3, 3, 4, 0, 1, 4, 7, 3

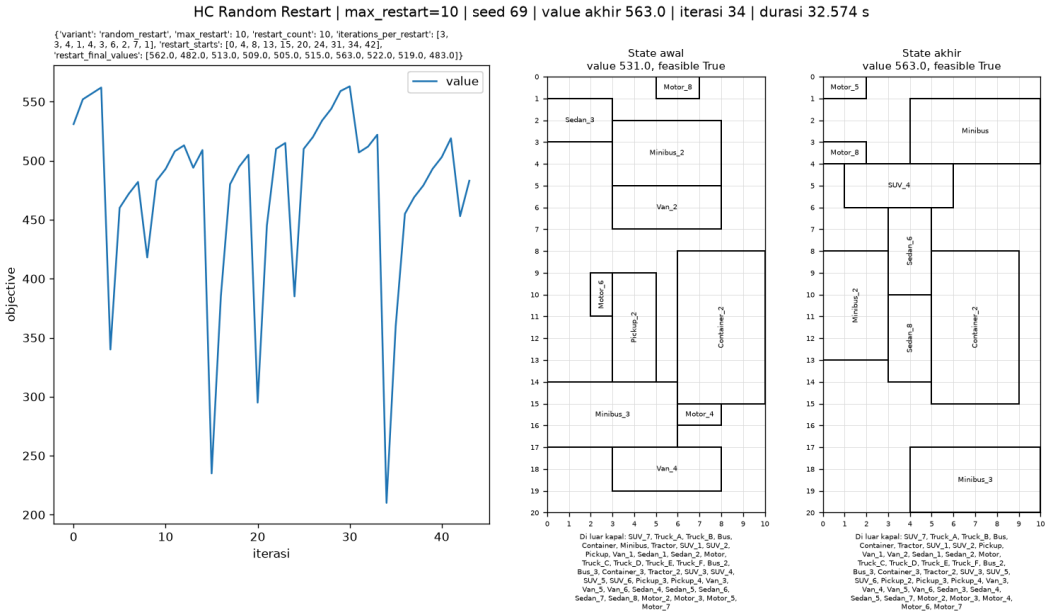
Berikut merupakan hasil visualisasi dari eksperimen yang merepresentasikan *initial state, final state,* plot nilai objective function terhadap banyak iterasi yang telah dilewati.



3.3.2 Eksperimen Kedua

input	Seed: 69 max_restart: 10
Hasil Eksperimen	Value awal : 531.0 Value akhir : 563.0 Feasible : True Iterasi : 34 Durasi : 32.574 s Visualisasi : output\hc_random_restart\seed69.png
Durasi	32.574 s
Banyak restart	10
Iterasi per restart	3, 3, 4, 1, 4, 3, 6, 2, 7, 1

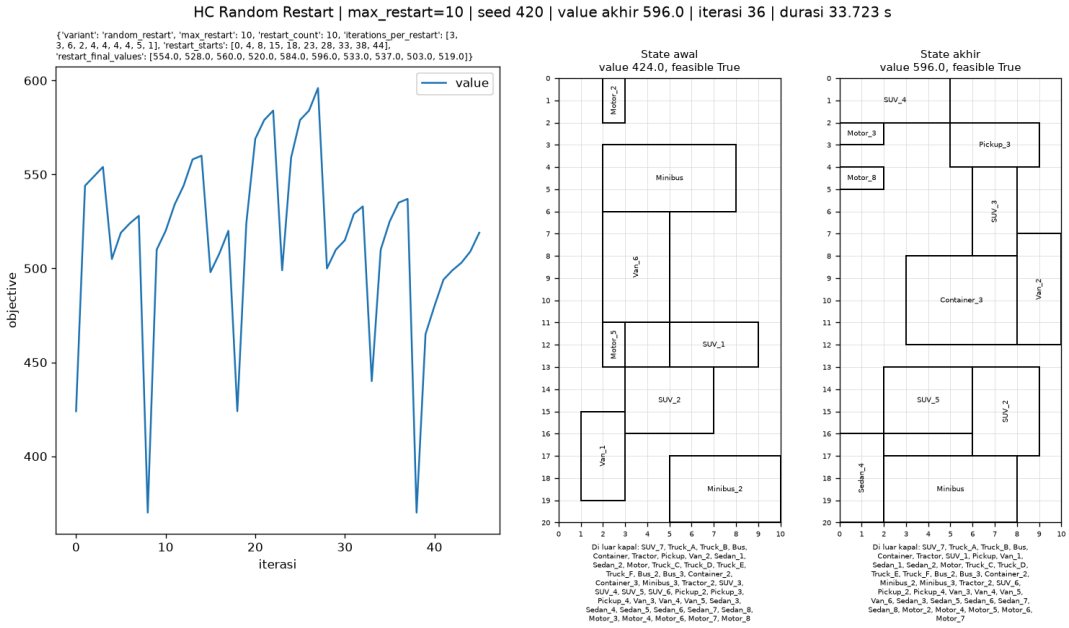
Berikut merupakan hasil visualisasi dari eksperimen yang merepresentasikan *initial state, final state,* plot nilai objective function terhadap banyak iterasi yang telah dilewati.



3.3.3 Eksperimen Ketiga

input	Seed: 420 max_restart: 10
Hasil Eksperimen	Value awal : 424.0 Value akhir : 596.0 Feasible : True Iterasi : 36 Durasi : 33.723 s Visualisasi : output\hc_random_restart\seed420.png
Durasi	33.723 s
Banyak restart	10
Iterasi per restart	3, 3, 6, 2, 4, 4, 4, 4, 5, 1

Berikut merupakan hasil visualisasi dari eksperimen yang merepresentasikan *initial state, final state,* plot nilai objective function terhadap banyak iterasi yang telah dilewati.



3.3.4 Analisis Singkat

Eksperimen Random Restart menghasilkan rata-rata objective akhir sebesar 584,67, dengan nilai terbaik 596 pada seed 420. Algoritma menjalankan sepuluh pencarian dari titik awal yang berbeda dan memilih solusi terbaik dari seluruh pencarian. Strategi ini membantu mengurangi ketergantungan pada satu initial state dan menghasilkan rentang objective akhir terkecil dibandingkan varian lainnya. Namun, keunggulan tersebut membutuhkan biaya komputasi lebih besar, dengan rata-rata durasi 31,251 detik, tertinggi di antara seluruh varian.

Hasil eksperimen menunjukkan kemampuan memperoleh solusi yang lebih baik daripada steepest ascent melalui pencarian dari beberapa kondisi awal, dengan waktu paling tinggi. Dari konfigurasi 10 percobaan pencarian, termasuk pencarian awal, algoritma menghasilkan nilai akhir pada rentang 563–596. Pengujian seed 62 menghasilkan nilai 595 dalam total 30 iterasi dengan durasi 27,455 detik, seed 69 menghasilkan 563 dalam 34 iterasi dengan durasi 32,574 detik, dan seed 420 menghasilkan 596 dalam 36 iterasi dengan durasi 33,723 detik. Peningkatan paling besar dibandingkan steepest ascent terjadi pada seed 62, yaitu dari 487 menjadi 595, sedangkan pada seed 69 peningkatannya hanya dari 562 menjadi 563. Perbedaan ini menunjukkan bahwa manfaat restart dipengaruhi oleh kualitas kondisi awal baru dan wilayah solusi yang berhasil dijangkau.

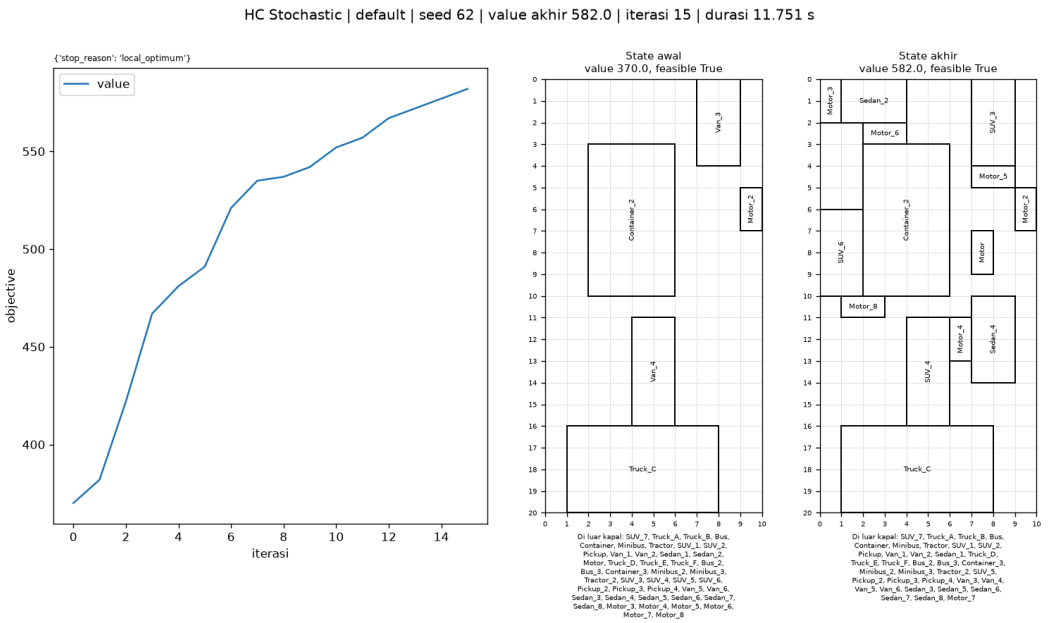
Grafik yang naik-turun menunjukkan dimulainya pencarian baru. Algoritma tetap menyimpan hasil terbaik dari seluruh percobaan, sehingga nilai akhir tidak harus sama dengan titik terakhir grafik. Dengan rata-rata nilai akhir 584,67 dan waktu 31,251 detik, random restart berhasil meningkatkan kualitas solusi secara rata-rata, tetapi tambahan percobaan tidak selalu menghasilkan solusi yang sebanding dengan biaya waktunya.

3.4 Algoritma Stochastic Hill Cimbing

3.4.1 Eksperimen Pertama

input	Seed: 62
Hasil Eksperimen	Value awal : 370.0 Value akhir : 582.0 Feasible : True Iterasi : 15 Durasi : 11.751 s Visualisasi : output\hc_stochastic\seed62.png
Durasi	11.751 s
Banyak iterasi hingga proses pencarian berhenti	15

Berikut merupakan hasil visualisasi dari eksperimen yang merepresentasikan *initial state, final state,* plot nilai objective function terhadap banyak iterasi yang telah dilewati.



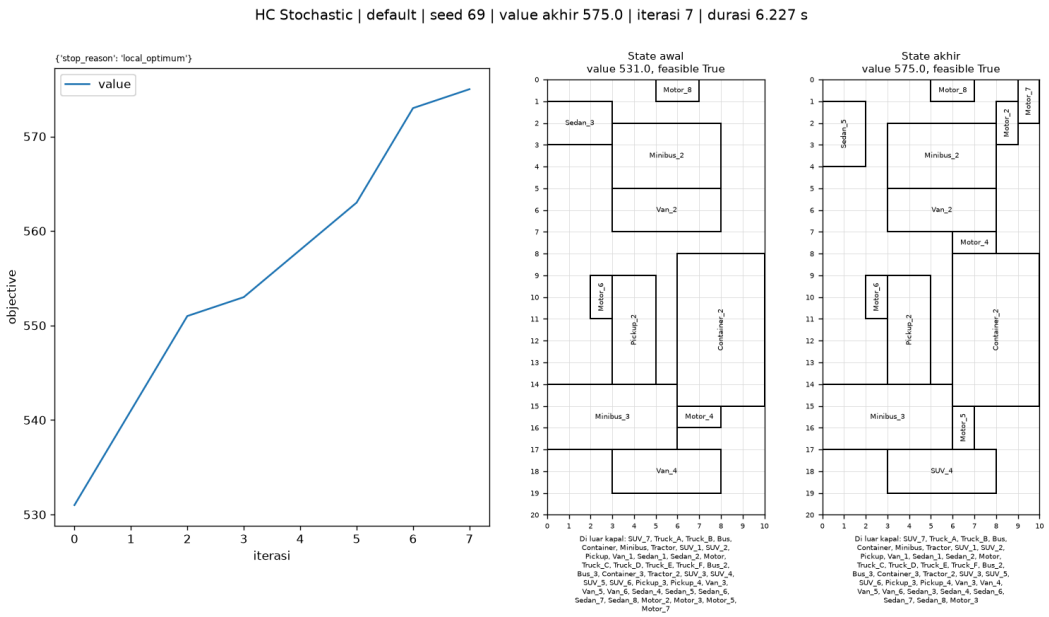
3.4.2 Eksperimen Kedua

input	Seed: 69

Hasil Eksperimen	`Value awal  : 531.0`
	`Value akhir : 575.0`
	`Feasible    : True`
	`Iterasi     : 7`
	`Durasi      : 6.227 s`
	`Visualisasi : output\hc_stochastic\seed69.png`

Durasi	6.227 s
Banyak iterasi hingga proses pencarian berhenti	7

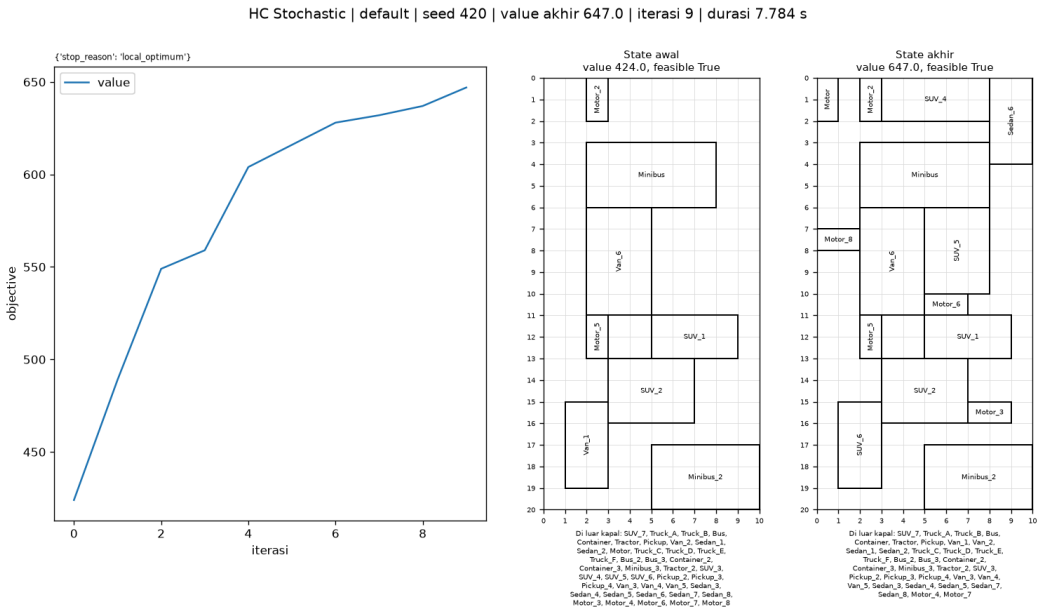
Berikut merupakan hasil visualisasi dari eksperimen yang merepresentasikan *initial state, final state,* plot nilai objective function terhadap banyak iterasi yang telah dilewati.



3.4.3 Eksperimen Ketiga

input	Seed: 420
Hasil Eksperimen	Value awal : 424.0 Value akhir : 647.0 Feasible : True Iterasi : 9 Durasi : 7.784 s Visualisasi : output\hc_stochastic\seed420.png
Durasi	7.784 s
Banyak iterasi hingga proses pencarian berhenti	9

Berikut merupakan hasil visualisasi dari eksperimen yang merepresentasikan *initial state, final state,* plot nilai objective function terhadap banyak iterasi yang telah dilewati.



3.4.4 Analisis Singkat

Eksperimen Stochastic menghasilkan rata-rata objective akhir tertinggi, yaitu 601,33, dengan nilai terbaik 647 pada seed 420. Berbeda dengan Steepest Ascent yang memilih peningkatan objective terbesar, Stochastic memilih secara acak salah satu neighbor yang memiliki objective lebih tinggi. Algoritma ini menghasilkan jalur pencarian yang berbeda dan berpotensi menemukan solusi yang lebih baik. Rata-rata durasinya sebesar 8,587 detik, lebih singkat daripada Sideways Move dan Random Restart. Meskipun memberikan hasil terbaik dalam eksperimen ini, performanya tetap dipengaruhi oleh pengacakan dan belum menjamin tercapainya global optimum.

Analisis stochastic menunjukkan kualitas solusi yang lebih baik secara rata-rata dengan waktu pencarian yang relatif lebih efisien dibandingkan sideways move dan random restart. Nilai akhir yang dihasilkan berada pada rentang 575–647. Pengujian seed 62 menghasilkan nilai 582 dalam 15 iterasi dengan durasi 11,751 detik, seed 69 menghasilkan 575 dalam 7 iterasi dengan durasi 6,227 detik, dan seed 420 menghasilkan 647 dalam 9 iterasi dengan durasi 7,784 detik.

Berbeda dari steepest ascent, stochastic memilih secara acak salah satu tetangga yang meningkatkan nilai, sehingga jalur pencariannya tidak selalu mengikuti kenaikan terbesar pada setiap langkah. Hasil eksperimen terlihat bahwa algoritma tersebut dapat

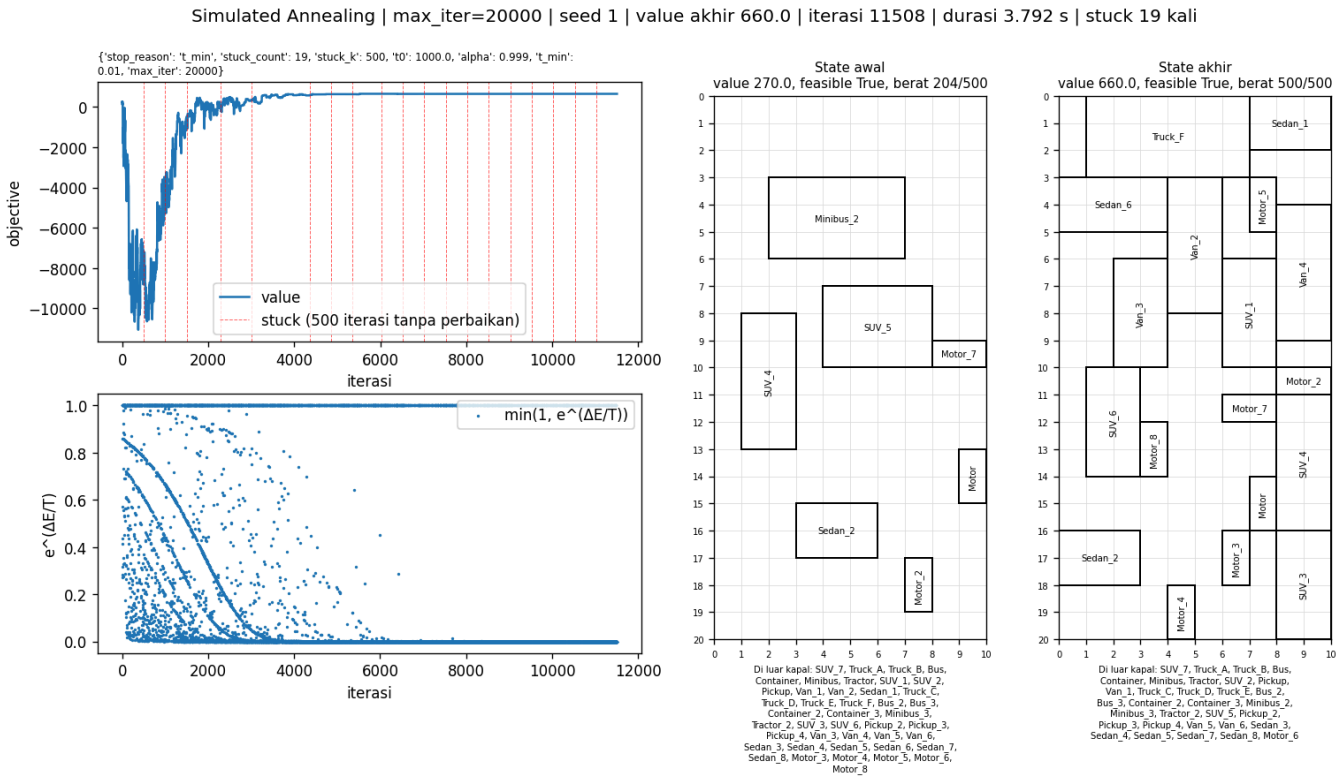
mengarah pada solusi akhir yang lebih baik, meskipun peningkatan pada masing-masing langkah tidak selalu maksimal. Stochastic melampaui hasil steepest ascent pada seluruh seed serta memperoleh nilai tertinggi di antara keempat algoritma pada seed 69 dan 420. Namun, algoritma tetap berhenti pada local optimum, sehingga unsur acak tidak menghilangkan keterbatasan pencarian lokal. Dengan rata-rata nilai akhir 601,33 dan waktu 8,587 detik, stochastic memberikan keseimbangan terbaik antara kualitas solusi dan waktu pada pengujian yang tersedia, meskipun keunggulan tersebut masih perlu diuji menggunakan lebih banyak seed.

3.5 Algoritma Simulated Annealing

3.5.1 Eksperimen Pertama

input	Suhu awal T0: 1000 Cooling rate (alpha): 0.999 Suhu minimum t_min: 0.01 Iterasi maksimum: 20000 Bobot langkah: swap 1, move 1, move_out 0.1, rotate 1 K stuck: 500 iterasi Seed: 1
Hasil Eksperimen	Stuck : 19 kali Value awal : 270.0 Value akhir : 660.0 Feasible : True Iterasi : 11508 Durasi : 3.792 s Visualisasi : output\simulated_annealing\seed1max_iter20000.png

Berikut merupakan visualisasi hasil penerapan algoritma Simulated Annealing, yang meliputi initial state, final state, grafik nilai objective function terhadap iterasi, serta grafik plot e^(ΔE/T) terhadap iterasi



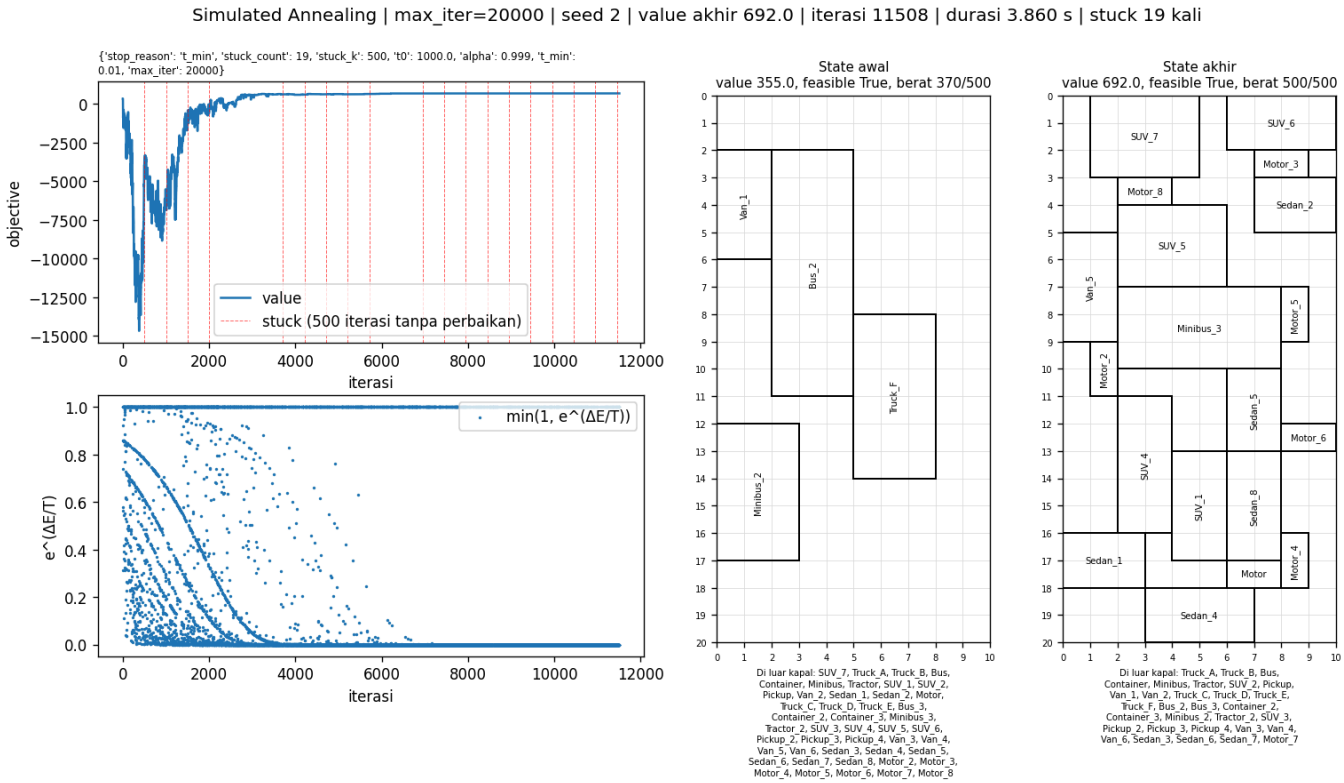
.

3.5.2 Eksperimen Kedua

input	Suhu awal T0: 1000 Cooling rate (alpha): 0.999 Suhu minimum t_min: 0.01 Iterasi maksimum: 20000 Bobot langkah: swap 1, move 1, move_out 0.1, rotate 1 K stuck: 500 iterasi Seed: 2
Hasil Eksperimen	Stuck : 19 kali Value awal : 355.0 Value akhir : 692.0 Feasible : True Iterasi : 11508 Durasi : 3.860 s Visualisasi : output\simulated_annealing\seed2max_iter20000.png

Berikut merupakan visualisasi hasil penerapan algoritma Simulated Annealing, yang meliputi initial state, final state, grafik nilai

objective function terhadap iterasi, serta grafik plot e^(ΔE/T) terhadap iterasi

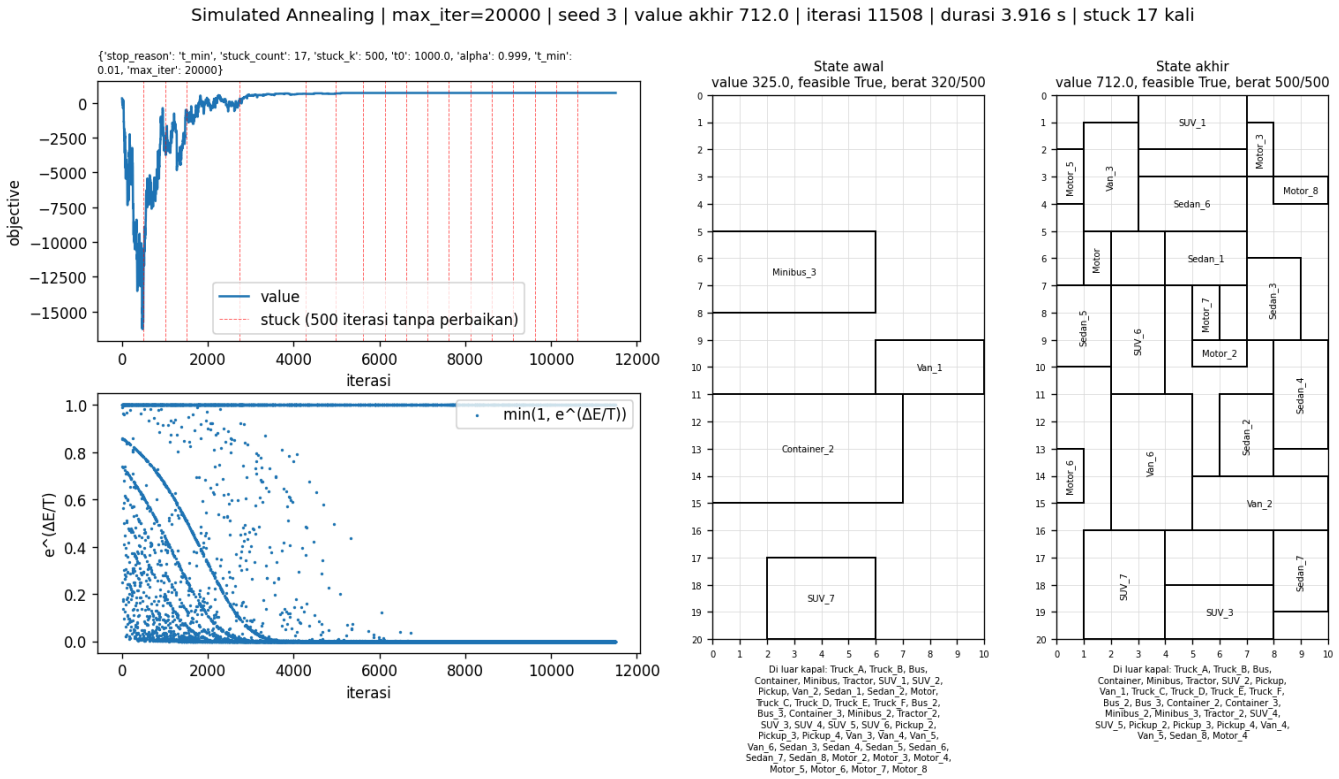


3.5.3 Eksperimen Ketiga

input	Suhu awal T0: 1000 Cooling rate (alpha): 0.999 Suhu minimum t_min: 0.01 Iterasi maksimum: 20000 Bobot langkah: swap 1, move 1, move_out 0.1, rotate 1 K stuck: 500 iterasi Seed: 3
Hasil Eksperimen	Stuck : 17 kali Value awal : 325.0 Value akhir : 712.0 Feasible : True Iterasi : 11508 Durasi : 3.916 s Visualisasi : output\simulated_annealing\seed3max_iter20000.png

Berikut merupakan visualisasi hasil penerapan algoritma Simulated Annealing, yang meliputi initial state, final state, grafik nilai

objective function terhadap iterasi, serta grafik plot e^(ΔE/T) terhadap iterasi



Berikut parameter keberhasilan pengujian dan dibandingkan dari tiap eksperimen.

Parameter	Eksperimen 1	Eksperimen 2	Eksperimen 3
Durasi (s)	3.792	3.860	3.916
Frekuensi Stuck	19	19	17
Nilai Objective Function Awal	270	355	325
Nilai Objective Function Akhir	660	692	712

Ketiga run berhasil meningkatkan nilai objective function dari kondisi awal yang acak menjadi hasil yang lebih baik. Seluruh solusi akhir juga memenuhi batasan yang ditetapkan, yaitu tidak terdapat kendaraan yang bertumpuk atau keluar dari geladak, serta total berat

tidak melebihi kapasitas kapal. Ketiganya memanfaatkan kapasitas berat secara penuh, yaitu 500 dari 500, meskipun luas geladak yang digunakan hanya 114, 124, dan 129 dari total 200 sel. Hal ini menunjukkan bahwa pada dataset yang digunakan, kapasitas berat menjadi batasan yang lebih menentukan dibandingkan luas geladak. Meskipun demikian, perbedaan nilai akhir menunjukkan bahwa hasil pencarian dapat dipengaruhi oleh kondisi awal dan variasi langkah yang terjadi selama proses pencarian solusi.

Proses pencarian SA dapat dibagi menjadi tiga fase berdasarkan perubahan suhu. Pada fase panas, algoritma masih leluasa menerima solusi yang lebih buruk untuk menjelajahi berbagai kemungkinan susunan kendaraan. Akibatnya, nilai solusi saat ini dapat turun drastis, bahkan menjadi negatif akibat penalti kelebihan berat atau kendaraan bertumpuk.

Memasuki fase pendinginan, suhu yang mulai menurun membuat algoritma lebih jarang menerima solusi yang lebih buruk, sehingga pencarian lebih berfokus pada menemukan susunan kendaraan dengan nilai yang lebih tinggi. Pada fase ini, sebagian besar peningkatan nilai terbaik terjadi karena algoritma masih memiliki cukup peluang untuk menjelajahi susunan baru, tetapi sudah lebih selektif dalam menerima solusi yang kurang menguntungkan. Sementara itu, pada fase dingin, solusi yang lebih buruk semakin jarang diterima sehingga pencarian cenderung mempertahankan solusi terbaik yang telah ditemukan.

Grafik objective function memperlihatkan bahwa nilai kondisi saat ini dan nilai terbaik bergerak dengan cara yang berbeda. Pada awal pencarian, nilai kondisi saat ini naik-turun cukup tajam dan sempat berada di bawah nol. Hal ini terjadi karena suhu masih tinggi, sehingga SA masih sering menerima langkah yang justru memperburuk solusi. Nilai terbaik tidak pernah turun dan grafiknya berbentuk seperti anak tangga, sebab nilai ini hanya berubah ketika algoritma menemukan solusi yang lebih baik dari semua solusi sebelumnya. Kenaikannya pun tidak terjadi terus-menerus, tetapi muncul setiap kali ditemukan susunan kendaraan yang lebih menguntungkan.

Kenaikan nilai terbaik paling banyak terjadi sebelum kurang lebih iterasi ke-5.000. Pada fase ini suhu belum terlalu rendah, sehingga algoritma masih leluasa mengeksplorasi dan memperbaiki solusi.

Setelah itu, grafik mulai mendatar. Suhu yang makin rendah membuat langkah yang memperburuk nilai hampir selalu ditolak, sehingga algoritma lebih banyak bertahan di sekitar solusi yang sudah ada. Secara keseluruhan, perbaikan terbesar terjadi pada tahap awal hingga pertengahan pencarian, sedangkan sisa iterasi lebih banyak berperan mempertahankan solusi terbaik yang sudah ditemukan.

Grafik e^(ΔE/T) menunjukkan bagaimana peluang diterimanya langkah yang memperburuk nilai berubah seiring turunnya suhu. Di awal pencarian, banyak titik berada di antara 0 dan 1, artinya langkah yang memperburuk solusi masih punya kemungkinan untuk diterima. Pada 500 iterasi pertama, rata-rata peluang penerimaan untuk langkah semacam ini berada di kisaran 0,4. Hal ini sesuai dengan peran suhu tinggi, yaitu membantu SA menjelajahi berbagai susunan kendaraan, termasuk susunan yang untuk sementara nilainya lebih rendah.

Semakin banyak iterasi berjalan, titik-titik untuk langkah yang memperburuk nilai makin mendekati nol karena suhu terus turun. Seberapa cepat peluang ini berkurang juga bergantung pada besar perubahan nilai yang diusulkan. Sekitar iterasi ke-4.000 hingga ke-5.000, hampir semua peluang penerimaan untuk langkah yang memperburuk nilai sudah mendekati nol. Meski begitu, garis rata-rata keseluruhan tidak ikut turun ke nol, karena langkah yang tidak memperburuk nilai selalu diterima dengan peluang 1. Pola ini sejalan dengan grafik objective function, menjelang akhir pencarian, SA makin jarang menerima solusi yang lebih buruk dan cenderung bertahan pada solusi yang sudah ditemukan.

Selama pencarian, jumlah kejadian stuck dengan parameter K=500 tercatat sebanyak 19, 19, dan 17 kali pada ketiga run. Namun, kejadian stuck tidak selalu berarti algoritma telah mencapai local optimum, karena pencarian masih berpeluang menemukan solusi yang lebih baik. Pada seed 1 dan 2, nilai akhir yang diperoleh tidak lagi meningkat setelah mencapai nilai terbaiknya, yaitu 660 dan 692. Sementara itu, seed 3 masih menemukan perbaikan hingga mendekati akhir pencarian dan menghasilkan nilai tertinggi, yaitu 712. Hal ini menunjukkan bahwa jumlah kejadian stuck tidak secara langsung menentukan kualitas solusi yang dihasilkan.

Untuk memastikan apakah solusi akhir telah mencapai local optimum, seluruh tetangga yang dapat dibangkitkan dari setiap solusi akhir diperiksa. Hasilnya, tidak ditemukan tetangga dengan nilai yang lebih tinggi pada ketiga run. Dengan demikian, seluruh solusi akhir dapat dikategorikan sebagai local optimum terhadap jenis langkah yang digunakan. Meskipun demikian, masih terdapat tetangga dengan nilai yang sama, yang menunjukkan adanya plateau, yaitu kondisi ketika perubahan susunan kendaraan tidak mengubah nilai objective function. SA dapat berpindah di antara susunan tersebut, tetapi perpindahan ini tidak meningkatkan nilai solusi. Oleh karena itu, ketiga run berakhir pada nilai yang berbeda meskipun semuanya telah mencapai local optimum.

Selain pengujian wajib di atas, dilakukan juga uji coba tambahan untuk melihat pengaruh bobot move_out terhadap hasil pencarian. Bobot 0,1 yang digunakan pada pengujian wajib merupakan hasil dari uji coba ini. Pengujian ini membandingkan tiga bobot move_out, yaitu 1, 0,1, dan 0, sementara parameter lainnya dibuat tetap. Pada variasi pertama, semua jenis langkah diberi bobot yang sama sehingga peluang terpilihnya setara. Pada variasi kedua, bobot move_out diturunkan menjadi 0,1 agar langkah ini lebih jarang dipilih dibandingkan langkah lain. Variasi ketiga memberi bobot 0 untuk melihat apa yang terjadi jika move_out tidak pernah dipakai sama sekali. Ketiga variasi dijalankan dengan seed 1, 2, dan 3, sama seperti pada pengujian wajib.

Move_out dipilih sebagai fokus pengujian karena hanya langkah inilah yang bisa mengurangi jumlah kendaraan di kapal. Swap hanya menukar posisi kendaraan, move bisa memasukkan kendaraan dari luar kapal tetapi tidak bisa mengeluarkannya, dan rotate hanya mengubah orientasi kendaraan. Padahal, selama suhu masih tinggi, kapal bisa kelebihan muatan atau berisi kendaraan yang bertumpuk, sehingga sebagian kendaraan perlu dikeluarkan agar solusi akhirnya kembali feasible. Masalahnya, mengeluarkan kendaraan biasanya juga mengurangi total fee, apalagi jika kapal sebenarnya tidak kelebihan muatan. Kendaraan dengan fee kecil juga lebih mudah terbuang karena penurunan nilai yang ditimbulkannya kecil, padahal gabungan banyak kendaraan kecil justru bisa menghasilkan nilai total yang tinggi. Karena itu, frekuensi move_out perlu diatur: cukup sering untuk

memulihkan kelayakan solusi, tetapi tidak terlalu sering sampai membuang kendaraan yang menyumbang nilai.

Hasil uji coba:

Bobot move_out	Seed	Objective awal	Objective akhir	Kendaraan di kapal (akhir)	Berat akhir	Stuck
1	1	270	545	9	497/500	20
1	2	355	543	9	500/500	19
1	3	325	618	11	500/500	18
0.1	1	270	660	18	500/500	19
0.1	2	355	692	19	500/500	19
0.1	3	325	712	21	500/500	17
0	1	270	270	7	204/500	23
0	2	355	355	4	370/500	23
0	3	325	325	4	320/500	23

Di ketiga seed, bobot 0,1 selalu memberikan nilai akhir tertinggi dengan rata-rata 688,0, disusul bobot 1 (568,7) dan bobot 0 (316,7). Karena state awal dan parameter lain sama, perbedaan ini hanya berasal dari bobot move_out.

Dengan bobot 1, move_out terlalu sering dipilih sehingga kendaraan sering dikeluarkan dari kapal. Kendaraan kecil paling mudah terbuang karena penurunan nilainya kecil, sehingga peluangnya untuk diterima oleh SA lebih besar. Akibatnya, kapal hanya berisi 9–11 kendaraan meskipun beratnya sudah hampir penuh. Kapasitas kapal habis oleh sedikit kendaraan besar.

Dengan bobot 0, tidak ada langkah yang bisa mengurangi jumlah kendaraan. Kapal terus terisi hingga jauh melebihi kapasitas dan tidak pernah kembali feasible, sehingga hasil akhirnya sama dengan

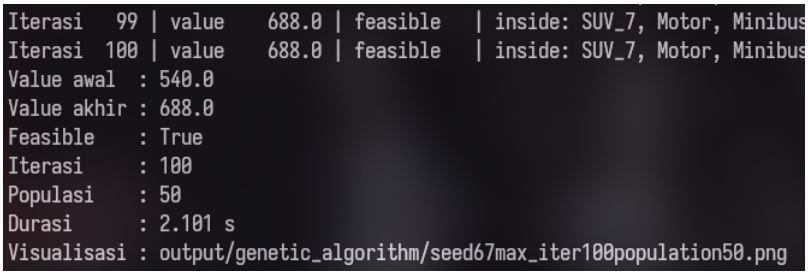
state awal. Ini menunjukkan bahwa move_out tetap dibutuhkan meskipun biasanya menurunkan nilai.

Dengan bobot 0,1, move_out cukup jarang dipilih sehingga kendaraan kecil tidak terus terbuang, tetapi masih tersedia untuk memperbaiki kapal saat kelebihan berat. Kapal akhirnya berisi 18–21 kendaraan dengan berat tepat 500, sekitar dua kali lebih banyak dibandingkan bobot 1.

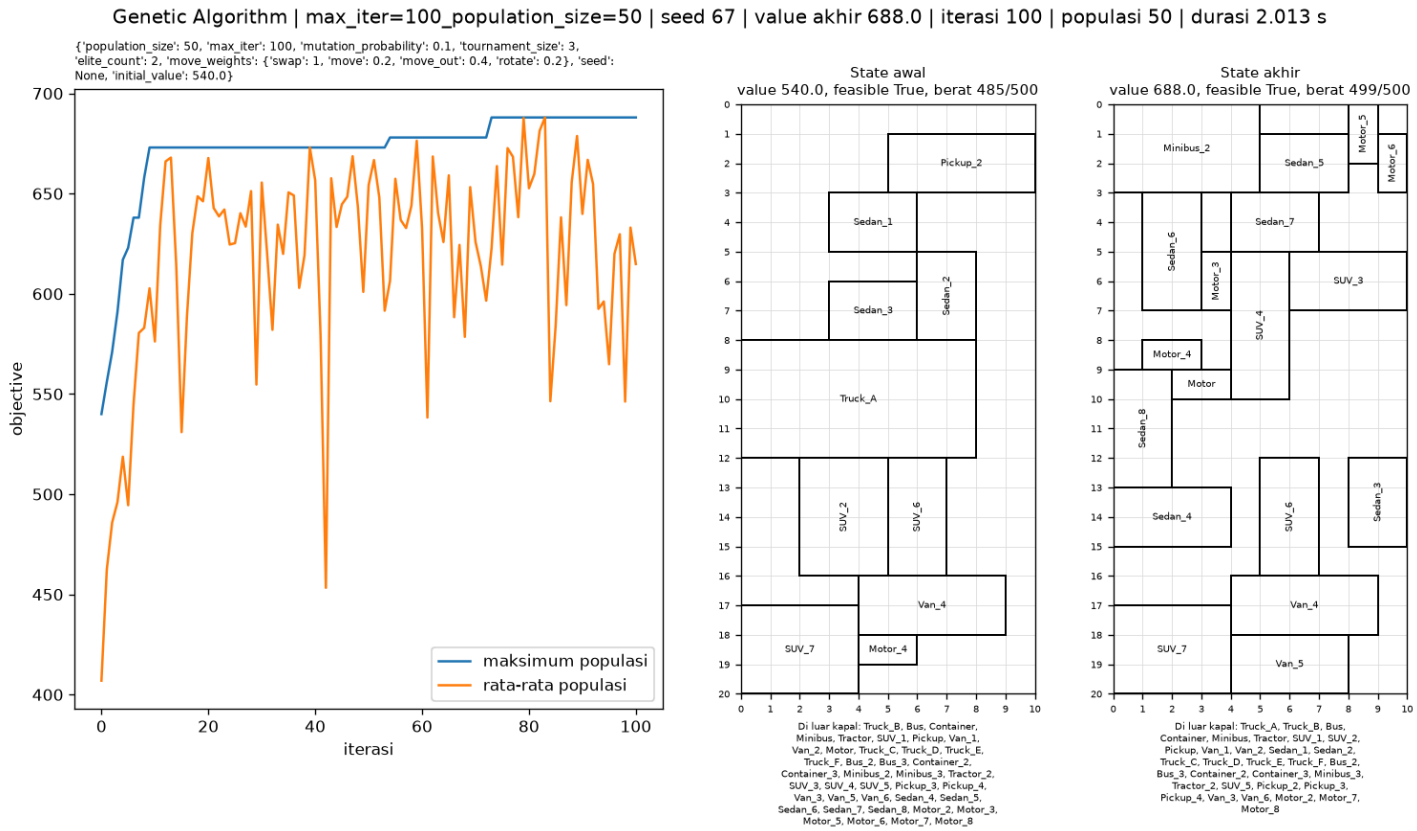
Dari hasil ini, move_out perlu tetap digunakan tetapi dengan frekuensi rendah. Bobot 0,1 memberikan hasil terbaik sehingga digunakan pada pengujian wajib. Namun, karena hanya tiga nilai bobot dan tiga seed yang diuji, 0,1 adalah nilai terbaik di antara variasi tersebut, belum tentu nilai yang paling optimal.

3.6 Algoritma Genetic

3.6.1 Eksperimen Pertama

input	seed = 67 population_size = 50 max_iter = 100
Hasil Eksperimen	

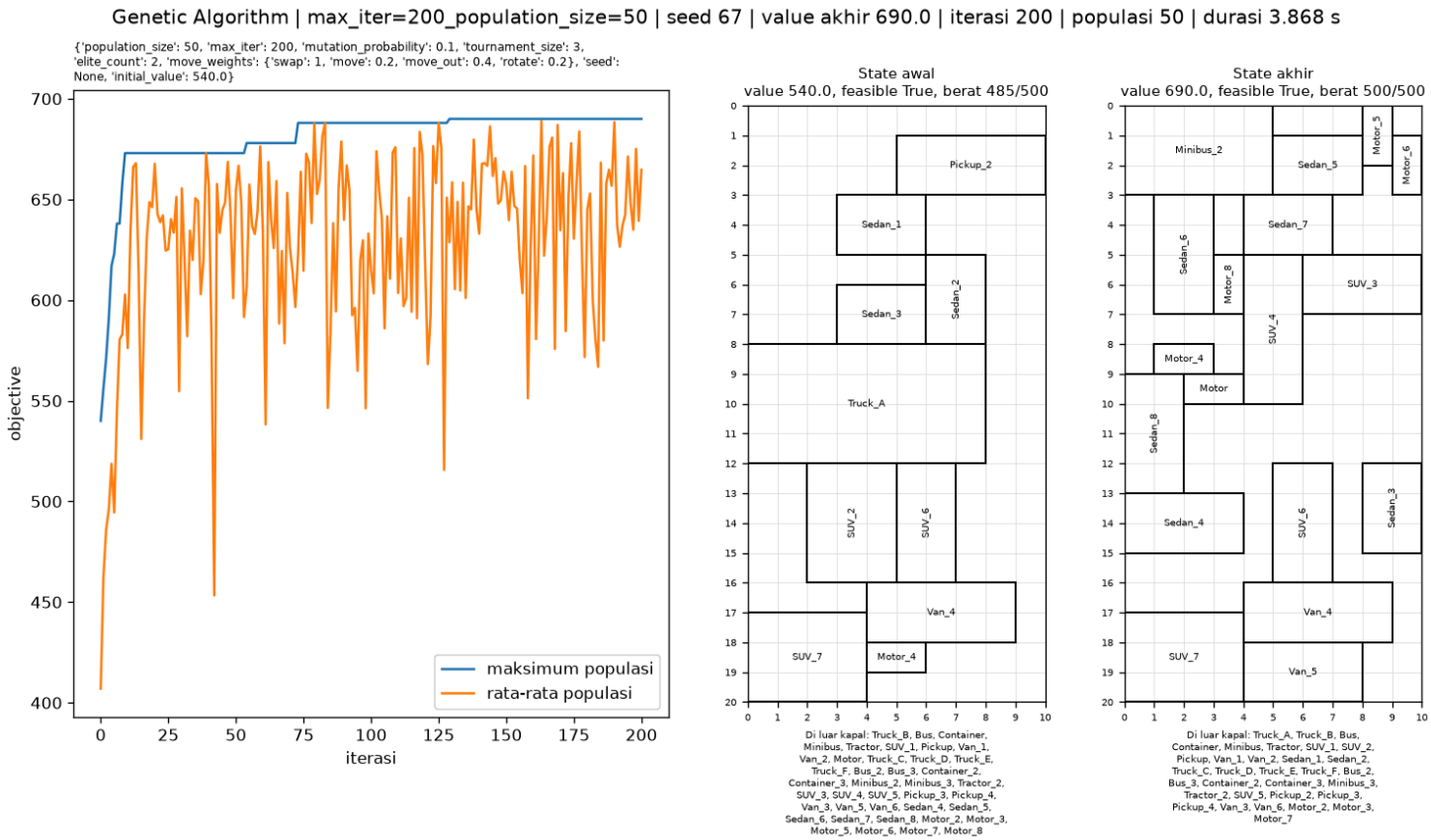
Di bawah merupakan visualisasi hasil eksperimen:



3.6.2 Eksperimen Kedua

input	seed = 67 population_size = 50 max_iter = 200
Hasil Eksperimen	Iterasi 199 | value 690.0 | feasible | inside: SUV_7, Motor, Minibus Iterasi 200 | value 690.0 | feasible | inside: SUV_7, Motor, Minibus Value awal : 540.0 Value akhir : 690.0 Feasible : True Iterasi : 200 Populasi : 50 Durasi : 3.868 s Visualisasi : output/genetic_algorithm/seed67max_iter200population50.png

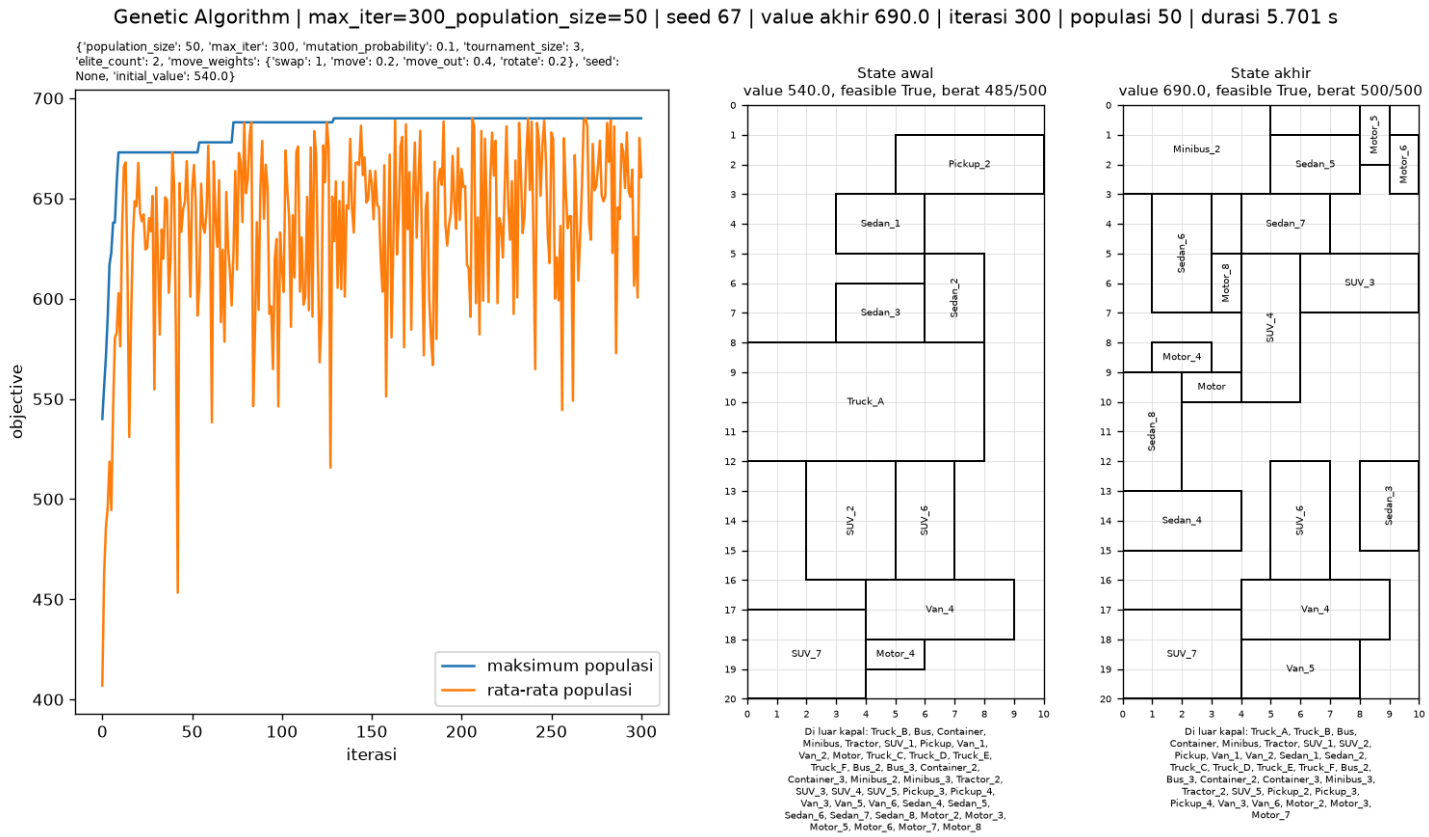
Di bawah merupakan visualisasi hasil eksperimen:



3.6.3 Eksperimen Ketiga

input	seed = 67 population_size = 50 max_iter = 300
Hasil Eksperimen	Iterasi 299 | value 690.0 | feasible | inside: SUV_7, Motor, Minibus Iterasi 300 | value 690.0 | feasible | inside: SUV_7, Motor, Minibus Value awal : 540.0 Value akhir : 690.0 Feasible : True Iterasi : 300 Populasi : 50 Durasi : 5.701 s Visualisasi : output/genetic_algorithm/seed67max_iter300population50.png

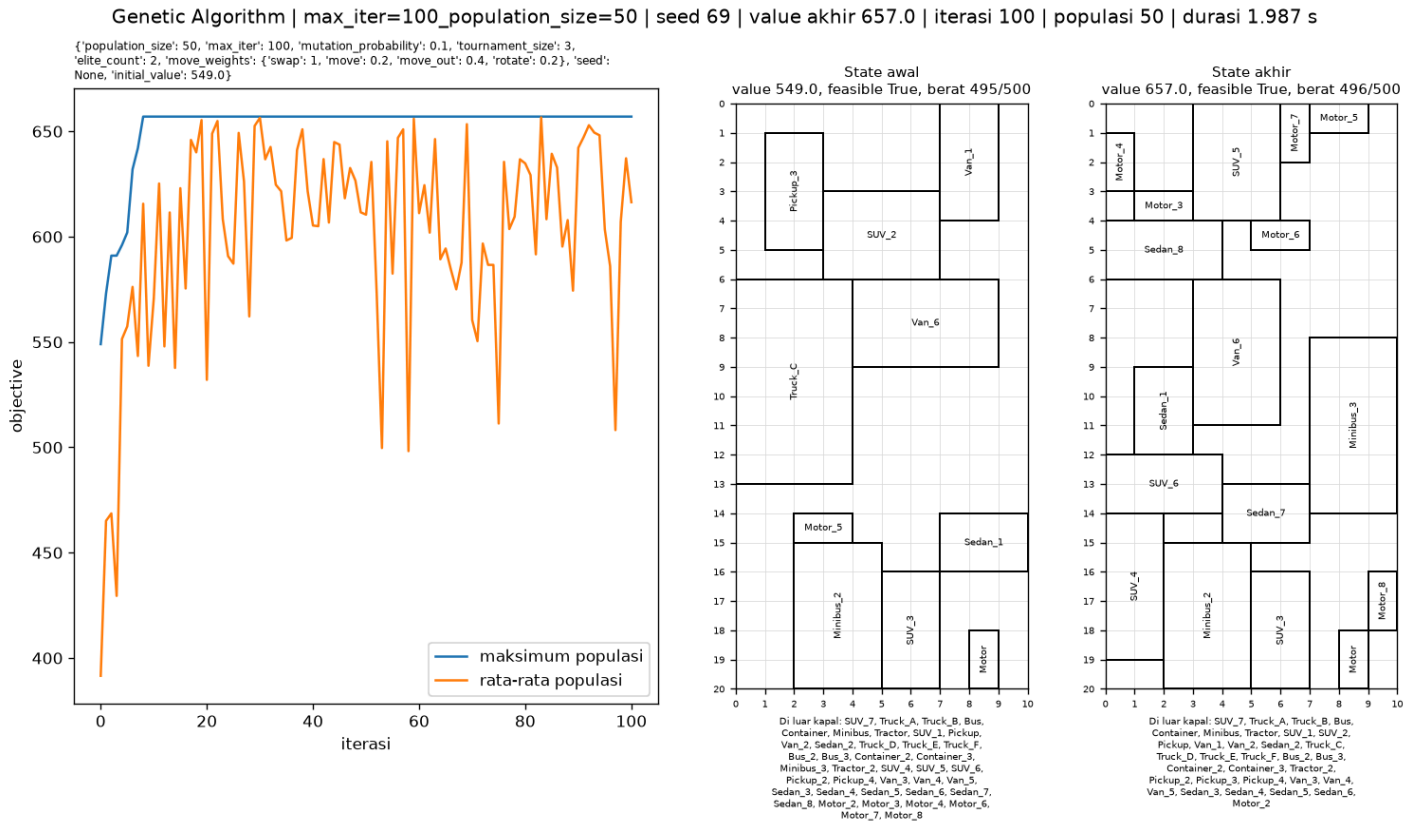
Di bawah merupakan visualisasi hasil eksperimen:



3.6.4 Eksperimen Keempat

input	seed = 69 population_size = 50 max_iter = 100
Hasil Eksperimen	Iterasi 99 | value 657.0 | feasible | inside: Sedan_1, Motor, Minib Iterasi 100 | value 657.0 | feasible | inside: Sedan_1, Motor, Minib Value awal : 549.0 Value akhir : 657.0 Feasible : True Iterasi : 100 Populasi : 50 Durasi : 1.987 s Visualisasi : output/genetic_algorithm/seed69max_iter100population50.png

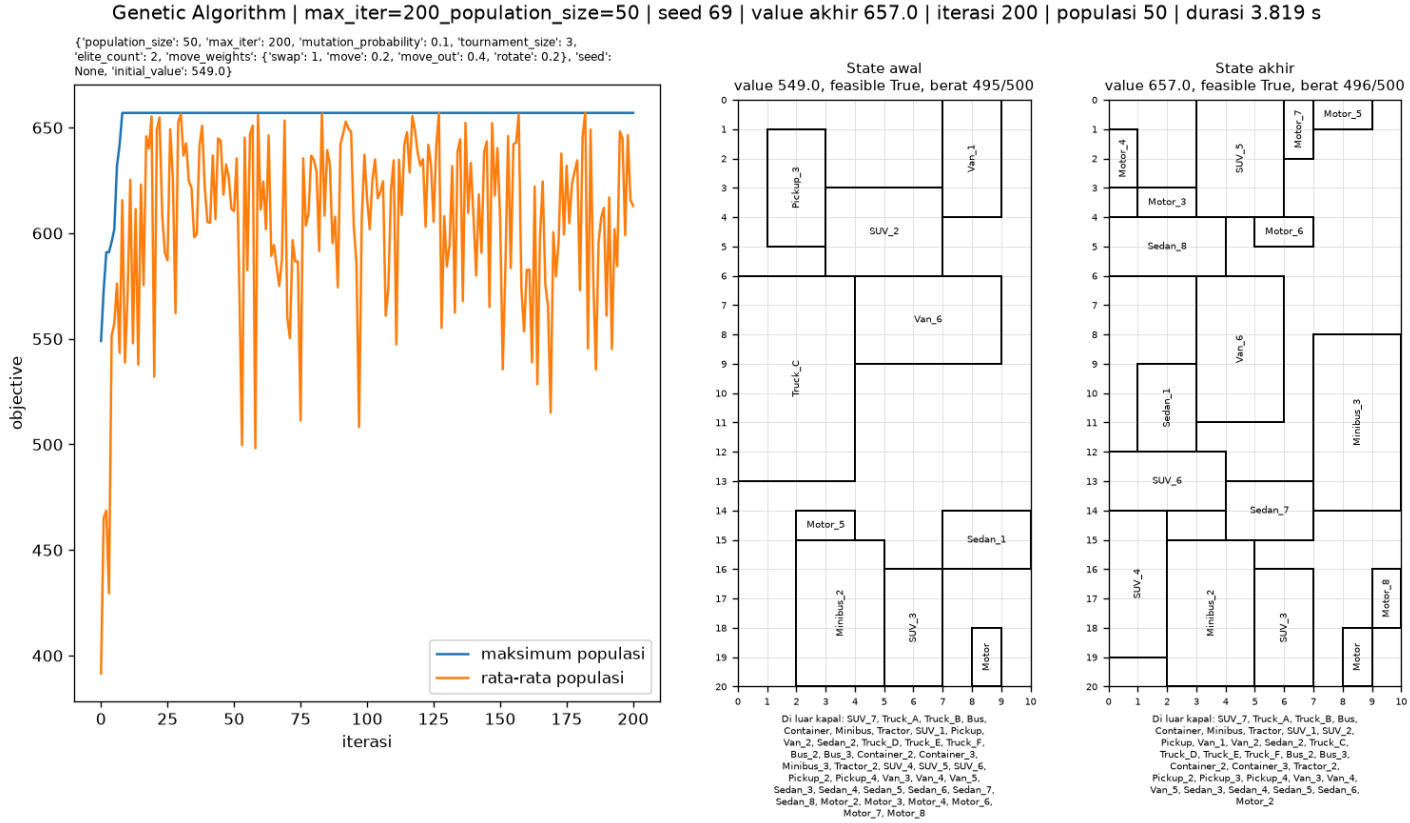
Di bawah merupakan visualisasi hasil eksperimen:



3.6.5 Eksperimen Kelima

input	seed = 69 population_size = 50 max_iter = 200
Hasil Eksperimen	Iterasi 199 | value 657.0 | feasible | inside: Sedan_1, Motor, Minit Iterasi 200 | value 657.0 | feasible | inside: Sedan_1, Motor, Minit Value awal : 549.0 Value akhir : 657.0 Feasible : True Iterasi : 200 Populasi : 50 Durasi : 3.819 s Visualisasi : output/genetic_algorithm/seed69max_iter200population50.png

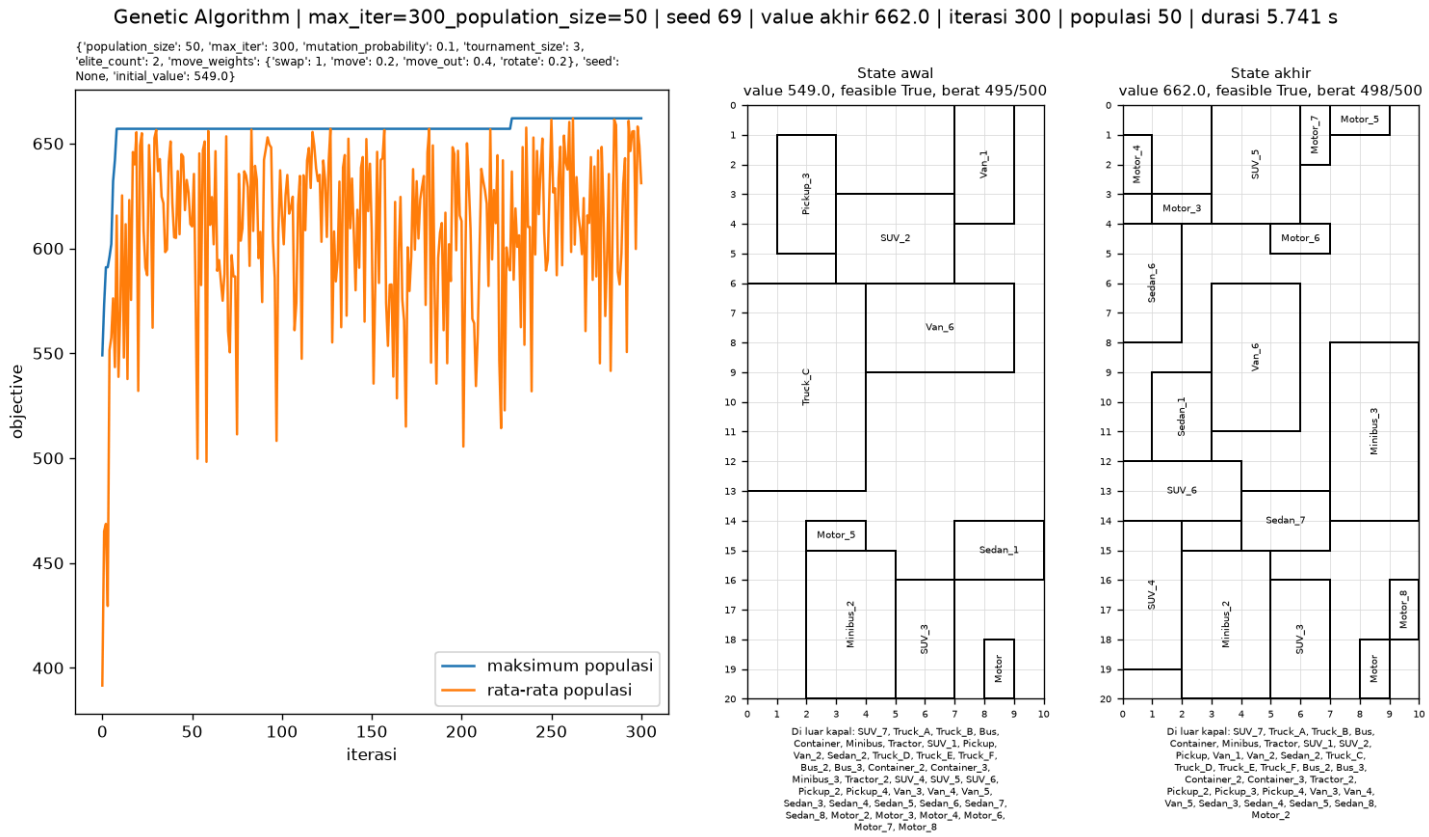
Di bawah merupakan visualisasi hasil eksperimen:



3.6.6 Eksperimen Keenam

input	seed = 69 population_size = 50 max_iter = 300
Hasil Eksperimen	Iterasi 299 | value 662.0 | feasible | inside: Sedan_1, Motor, Minib Iterasi 300 | value 662.0 | feasible | inside: Sedan_1, Motor, Minib Value awal : 549.0 Value akhir : 662.0 Feasible : True Iterasi : 300 Populasi : 50 Durasi : 5.741 s Visualisasi : output/genetic_algorithm/seed69max_iter300population50.png

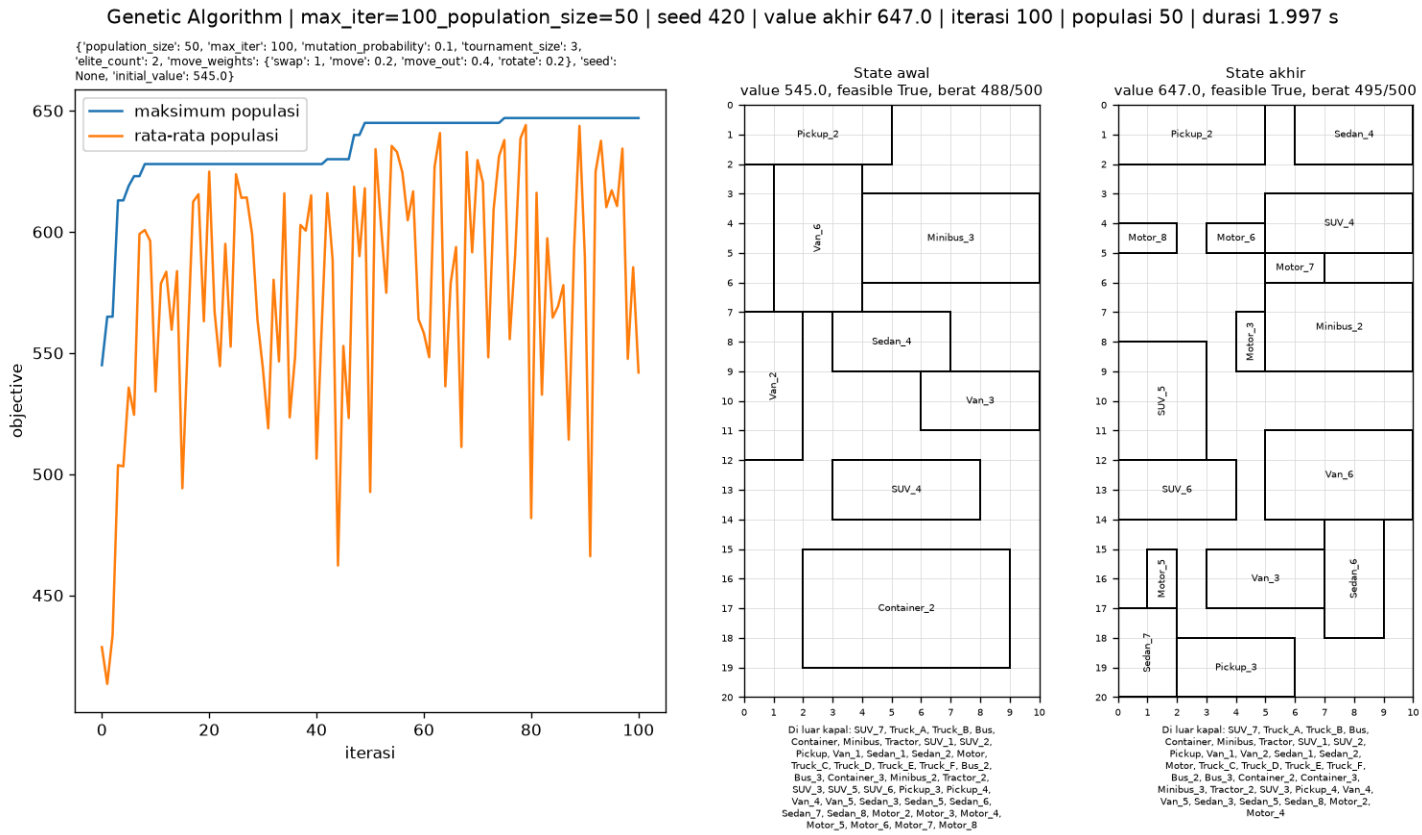
Di bawah merupakan visualisasi hasil eksperimen:



3.6.7 Eksperimen Ketujuh

input	seed = 420 population_size = 50 max_iter = 100
Hasil Eksperimen	Iterasi 99 | value 647.0 | feasible | inside: Minibus_2, SUV_4, SUV Iterasi 100 | value 647.0 | feasible | inside: Minibus_2, SUV_4, SUV Value awal : 545.0 Value akhir : 647.0 Feasible : True Iterasi : 100 Populasi : 50 Durasi : 1.997 s Visualisasi : output/genetic_algorithm/seed420max_iter100population50.png

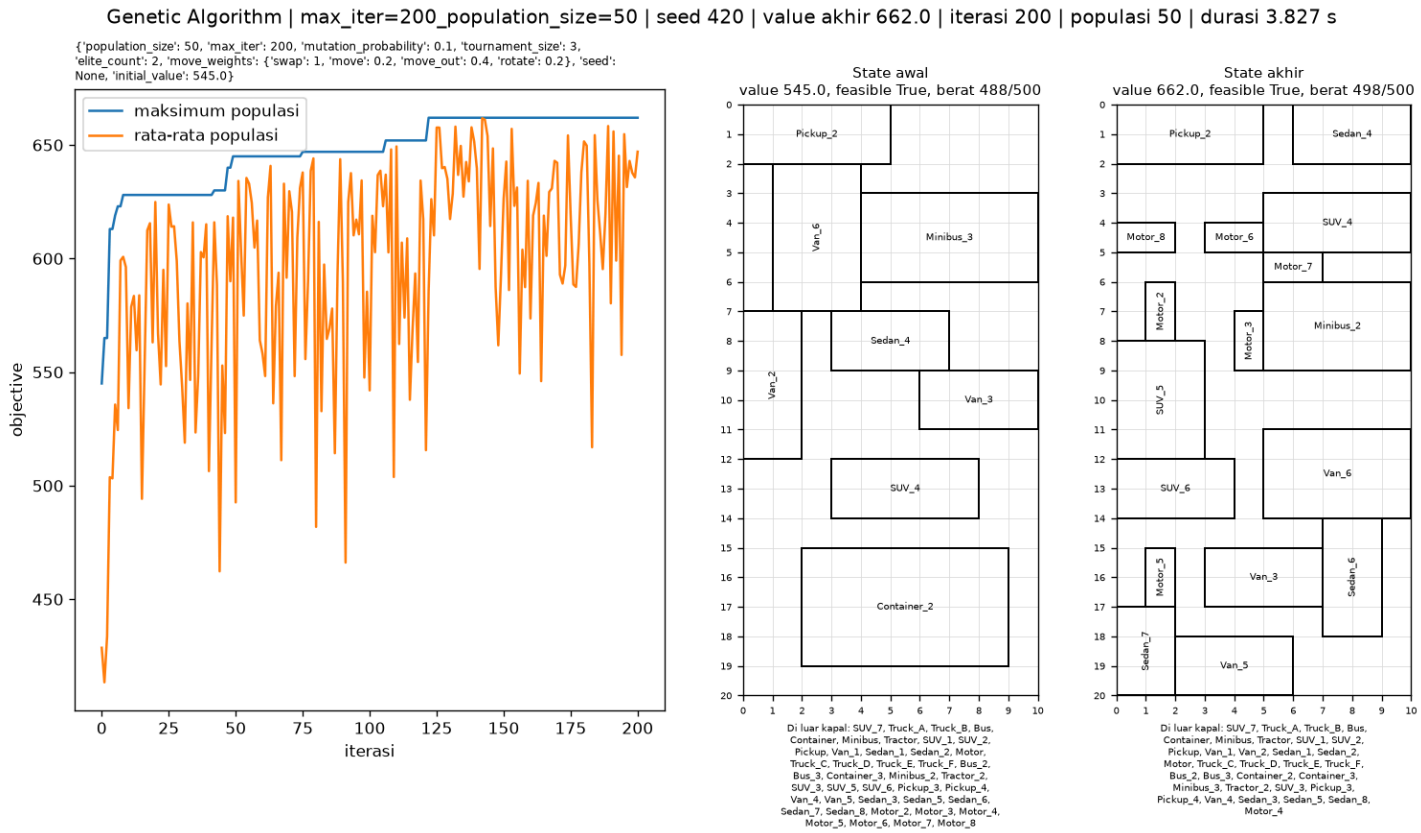
Di bawah merupakan visualisasi hasil eksperimen:



3.6.8 Eksperimen Kedelapan

input	seed = 420 population_size = 50 max_iter = 200
Hasil Eksperimen	Iterasi 199 | value 662.0 | feasible | inside: Minibus_2, SUV_4, SUV Iterasi 200 | value 662.0 | feasible | inside: Minibus_2, SUV_4, SUV Value awal : 545.0 Value akhir : 662.0 Feasible : True Iterasi : 200 Populasi : 50 Durasi : 3.827 s Visualisasi : output/genetic_algorithm/seed420max_iter200population50.png

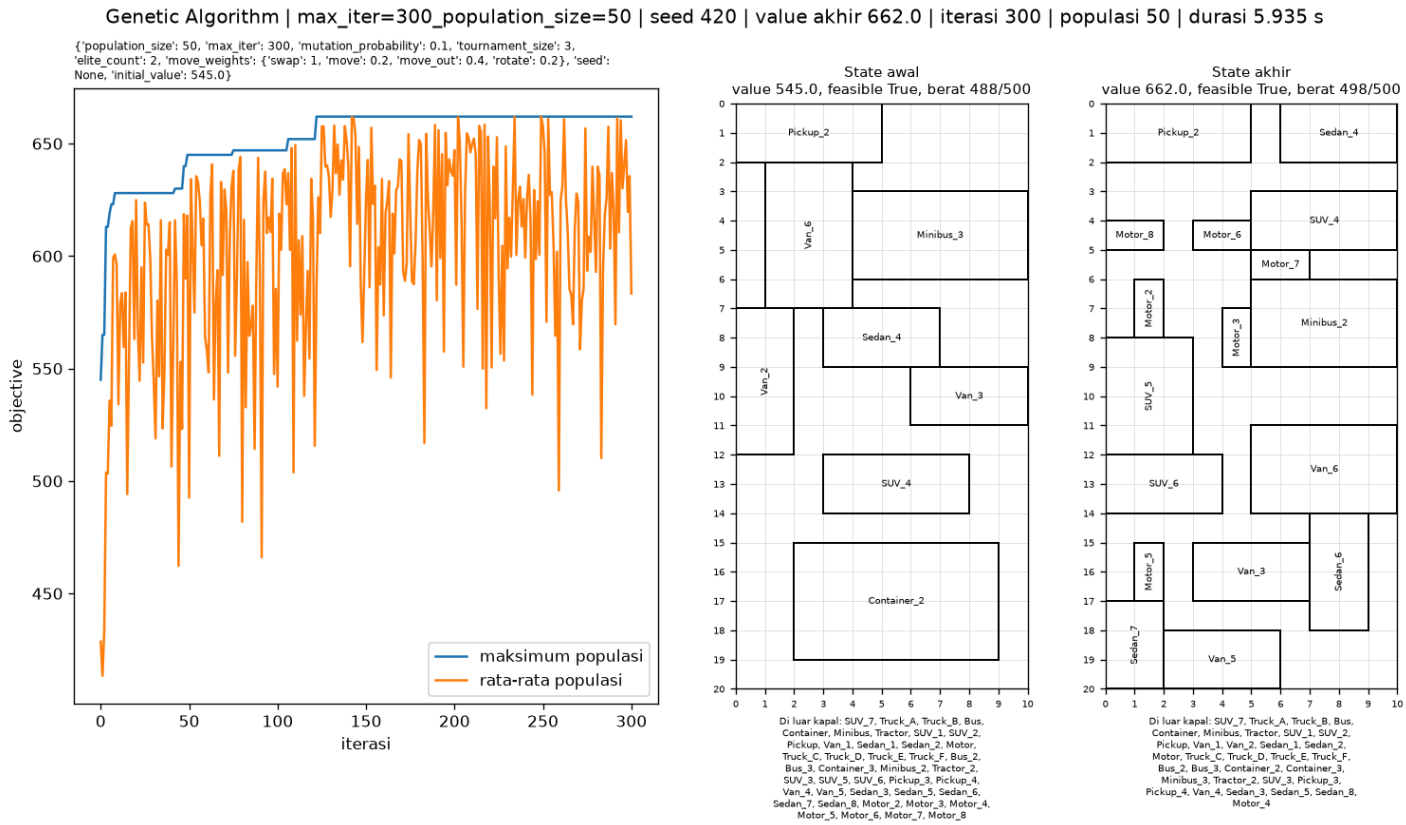
Di bawah merupakan visualisasi hasil eksperimen:



3.6.9 Eksperimen Kesembilan

input	seed = 420 population_size = 50 max_iter = 300
Hasil Eksperimen	Iterasi 299 | value 662.0 | feasible | inside: Minibus_2, SUV_4, SUV Iterasi 300 | value 662.0 | feasible | inside: Minibus_2, SUV_4, SUV Value awal : 545.0 Value akhir : 662.0 Feasible : True Iterasi : 300 Populasi : 50 Durasi : 5.935 s Visualisasi : output/genetic_algorithm/seed420max_iter300population50.png

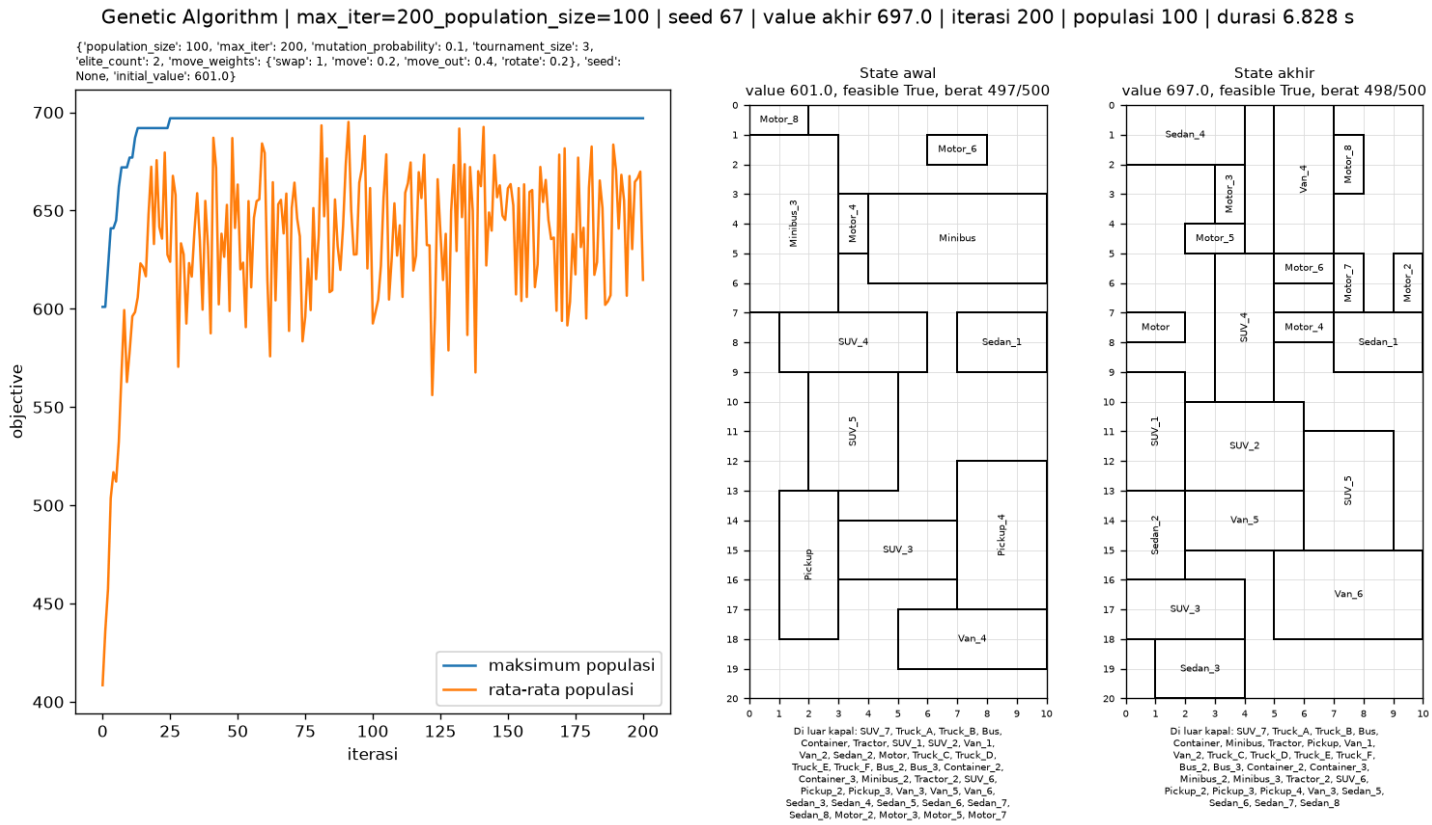
Di bawah merupakan visualisasi hasil eksperimen:



3.6.10 Eksperimen Kesepuluh

input	seed = 67 population_size = 100 max_iter = 200
Hasil Eksperimen	Iterasi 200 | value 697.0 | feasible | inside: SUV_1, SUV_2, Sedan_1, S _3, Motor_4, Motor_5, Motor_6, Motor_7, Motor_8 Value awal : 601.0 Value akhir : 697.0 Feasible : True Iterasi : 200 Populasi : 100 Durasi : 6.828 s Visualisasi : output/genetic_algorithm/seed67max_iter200population100.png

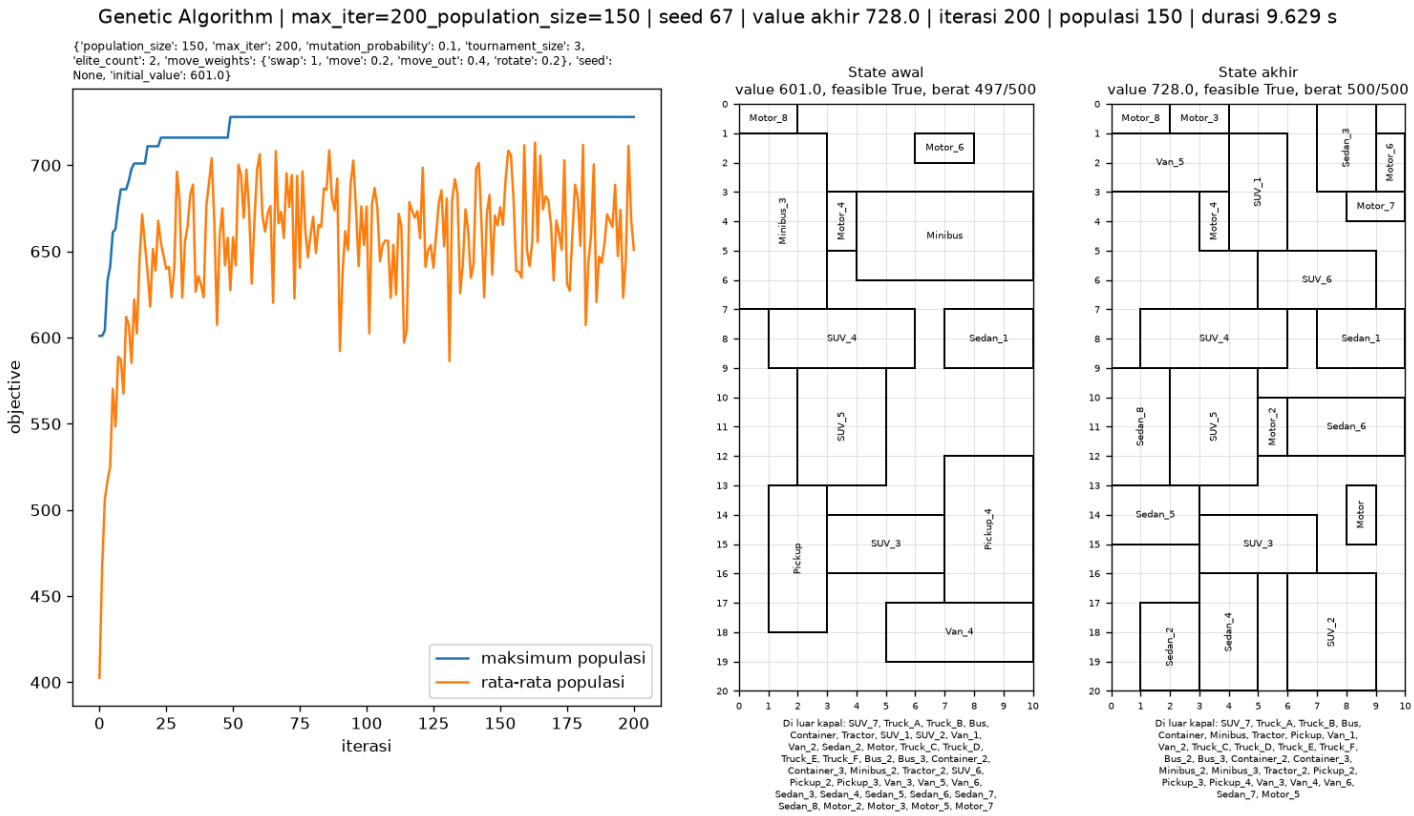
Di bawah merupakan visualisasi hasil eksperimen:



3.6.11 Eksperimen Kesebelas

input	seed = 67 population_size = 150 max_iter = 200
Hasil Eksperimen	Iterasi 200 | value 728.0 | feasible | inside: SUV_1, SUV_2, Sedan_1, Sedan_8, Motor_2, Motor_3, Motor_4, Motor_6, Motor_7, Motor_8 Value awal : 601.0 Value akhir : 728.0 Feasible : True Iterasi : 200 Populasi : 150 Durasi : 9.629 s Visualisasi : output/genetic_algorithm/seed67max_iter200population150.png

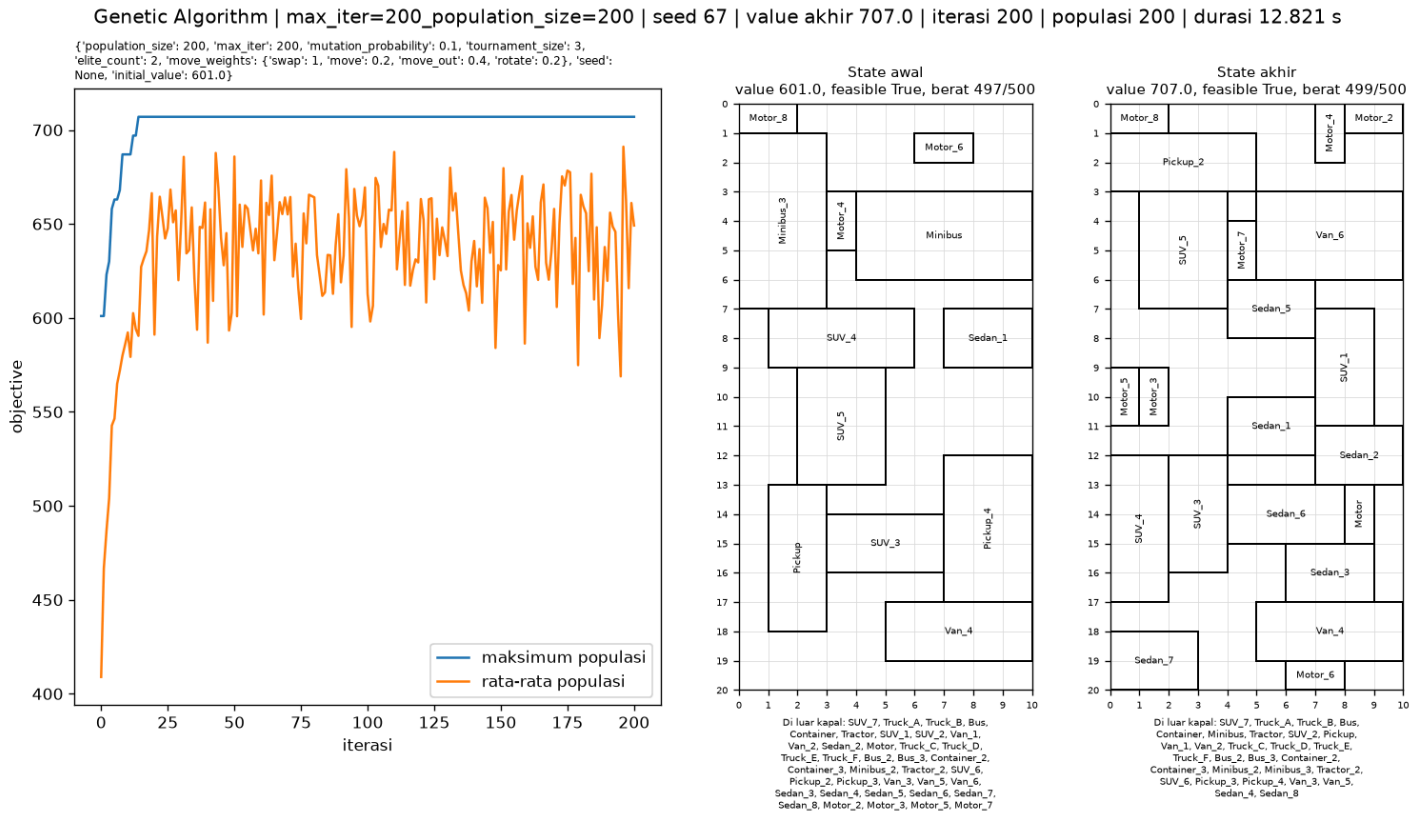
Di bawah merupakan visualisasi hasil eksperimen:



3.6.12 Eksperimen Kedua belas

input	seed = 67 population_size = 200 max_iter = 200
Hasil Eksperimen	Iterasi 200 | value 707.0 | feasible | inside: SUV_1, Sedan_1, Sedan_2, Motor_2, Motor_3, Motor_4, Motor_5, Motor_6, Motor_7, Motor_8 Value awal : 601.0 Value akhir : 707.0 Feasible : True Iterasi : 200 Populasi : 200 Durasi : 12.821 s Visualisasi : output/genetic_algorithm/seed67max_iter200population200.png

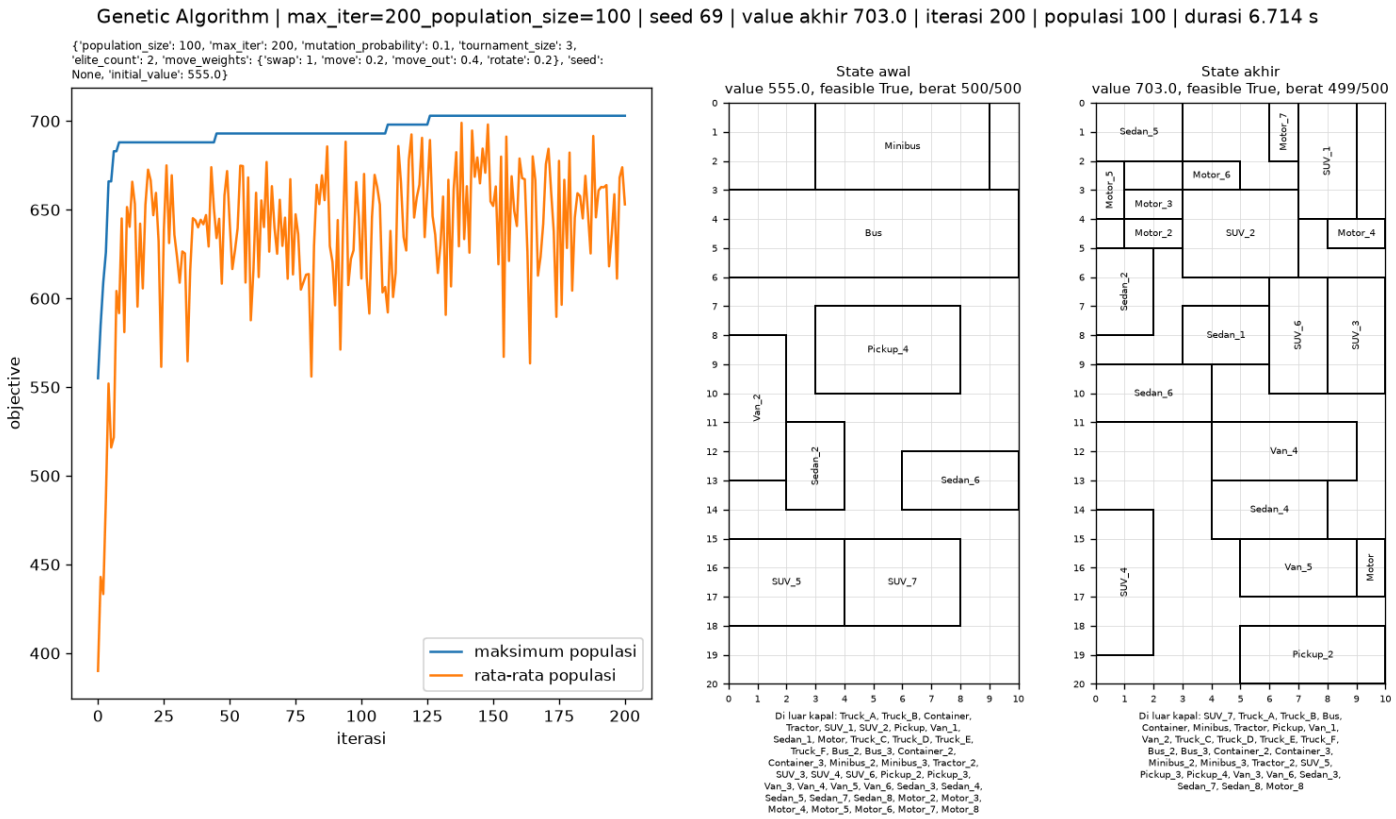
Di bawah merupakan visualisasi hasil eksperimen:



3.6.13 Eksperimen Ketiga belas

input	seed = 69 population_size = 100 max_iter = 200
Hasil Eksperimen	Iterasi 200 | value 703.0 | feasible | inside: SUV_1, SUV_2, Sedan_1, Sedan_2, Motor_2, Motor_3, Motor_4, Motor_5, Motor_6, Motor_7 Value awal : 555.0 Value akhir : 703.0 Feasible : True Iterasi : 200 Populasi : 100 Durasi : 6.714 s Visualisasi : output/genetic_algorithm/seed69max_iter200population100.png

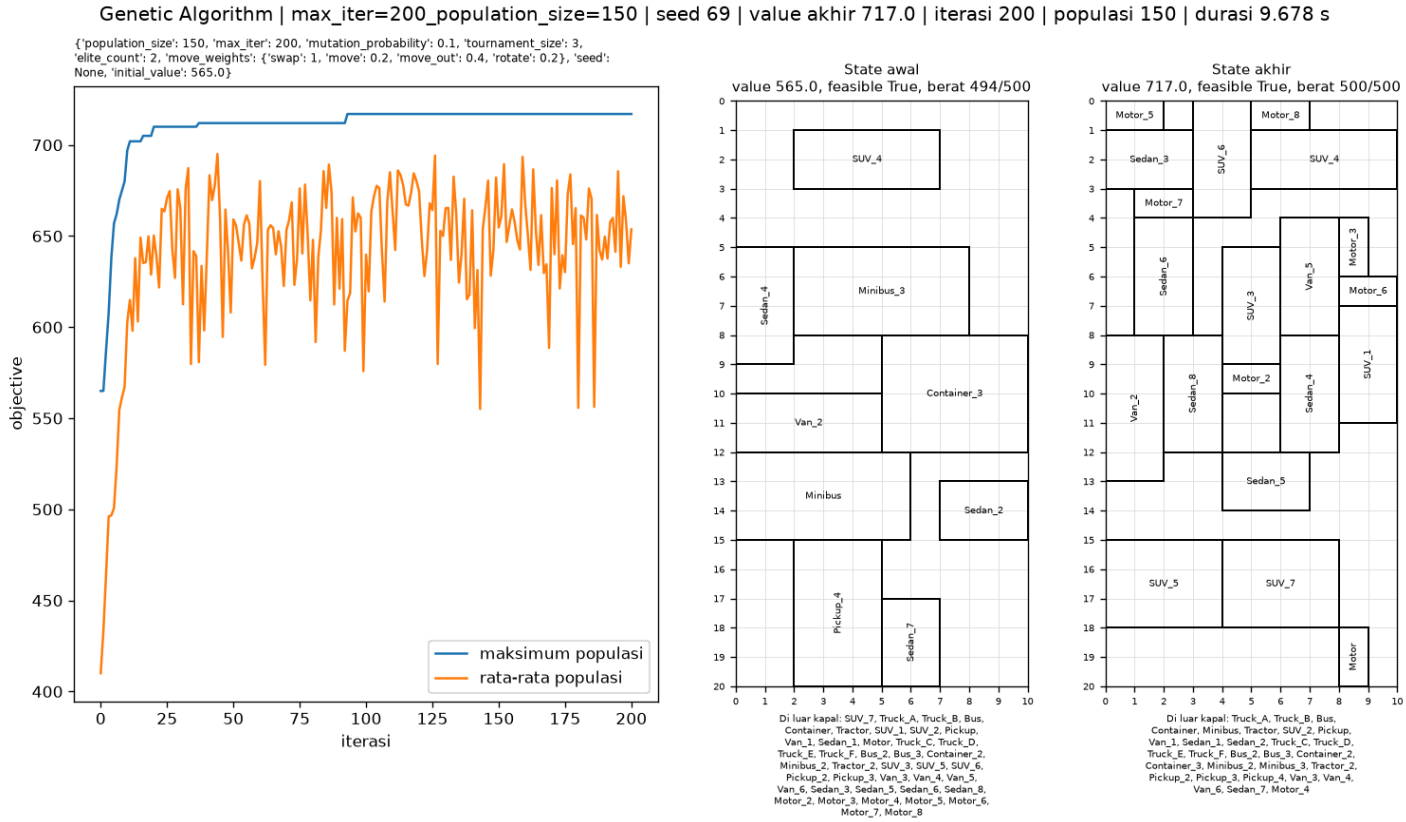
Di bawah merupakan visualisasi hasil eksperimen:



3.6.14 Eksperimen Keempat belas

input	seed = 69 population_size = 150 max_iter = 200
Hasil Eksperimen	Iterasi 200 | value 717.0 | feasible | inside: SUV_7, SUV_1, Van_2, Mot _2, Motor_3, Motor_5, Motor_6, Motor_7, Motor_8 Value awal : 565.0 Value akhir : 717.0 Feasible : True Iterasi : 200 Populasi : 150 Durasi : 9.678 s Visualisasi : output/genetic_algorithm/seed69max_iter200population150.png

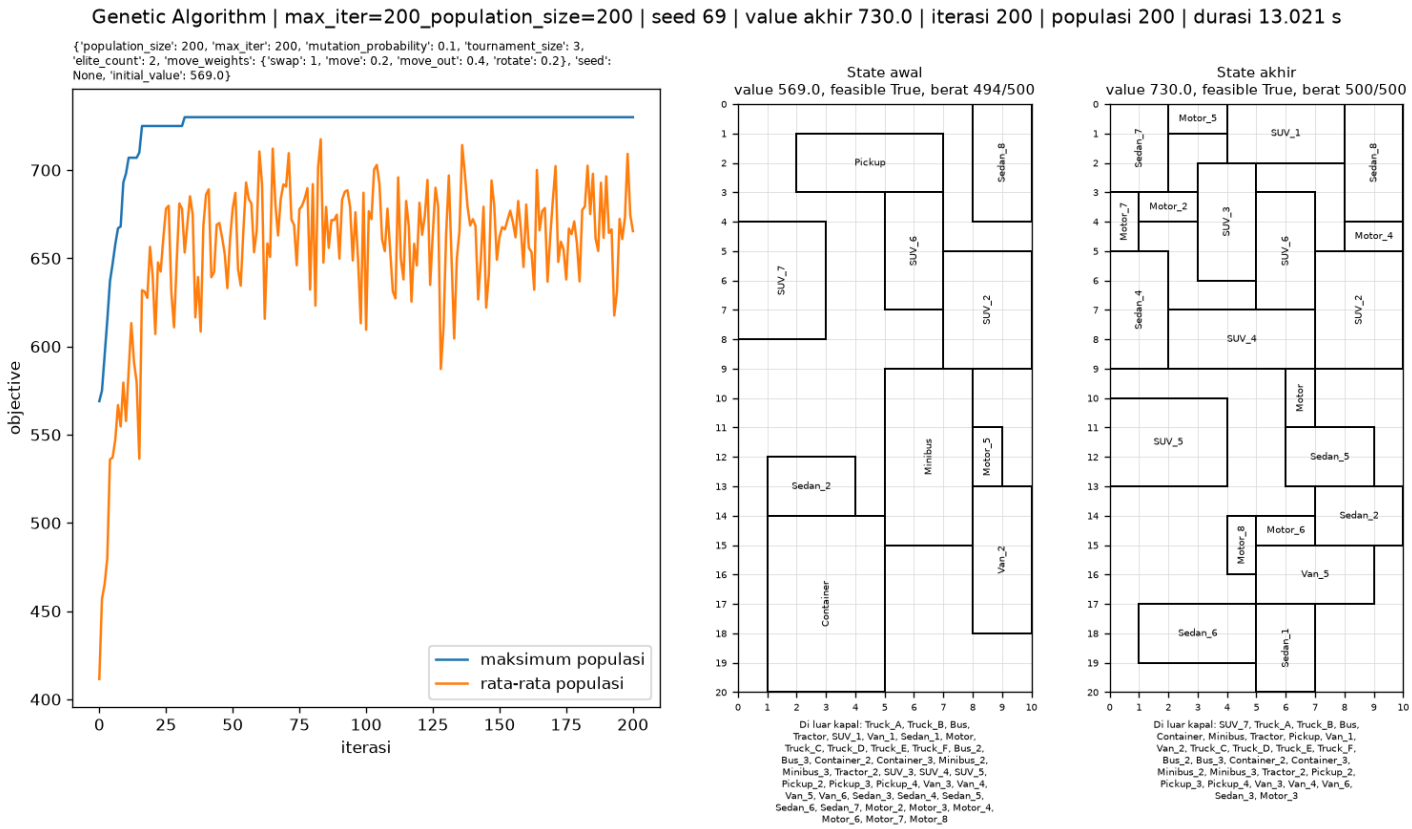
Di bawah merupakan visualisasi hasil eksperimen:



3.6.15 Eksperimen Kelima belas

input	seed = 69 population_size = 200 max_iter = 200
Hasil Eksperimen	Iterasi 200 | value 730.0 | feasible | inside: SUV_1, SUV_2, Sedan_1, Sedan_8, Motor_2, Motor_4, Motor_5, Motor_6, Motor_7, Motor_8 Value awal : 569.0 Value akhir : 730.0 Feasible : True Iterasi : 200 Populasi : 200 Durasi : 13.021 s Visualisasi : output/genetic_algorithm/seed69max_iter200population200.png

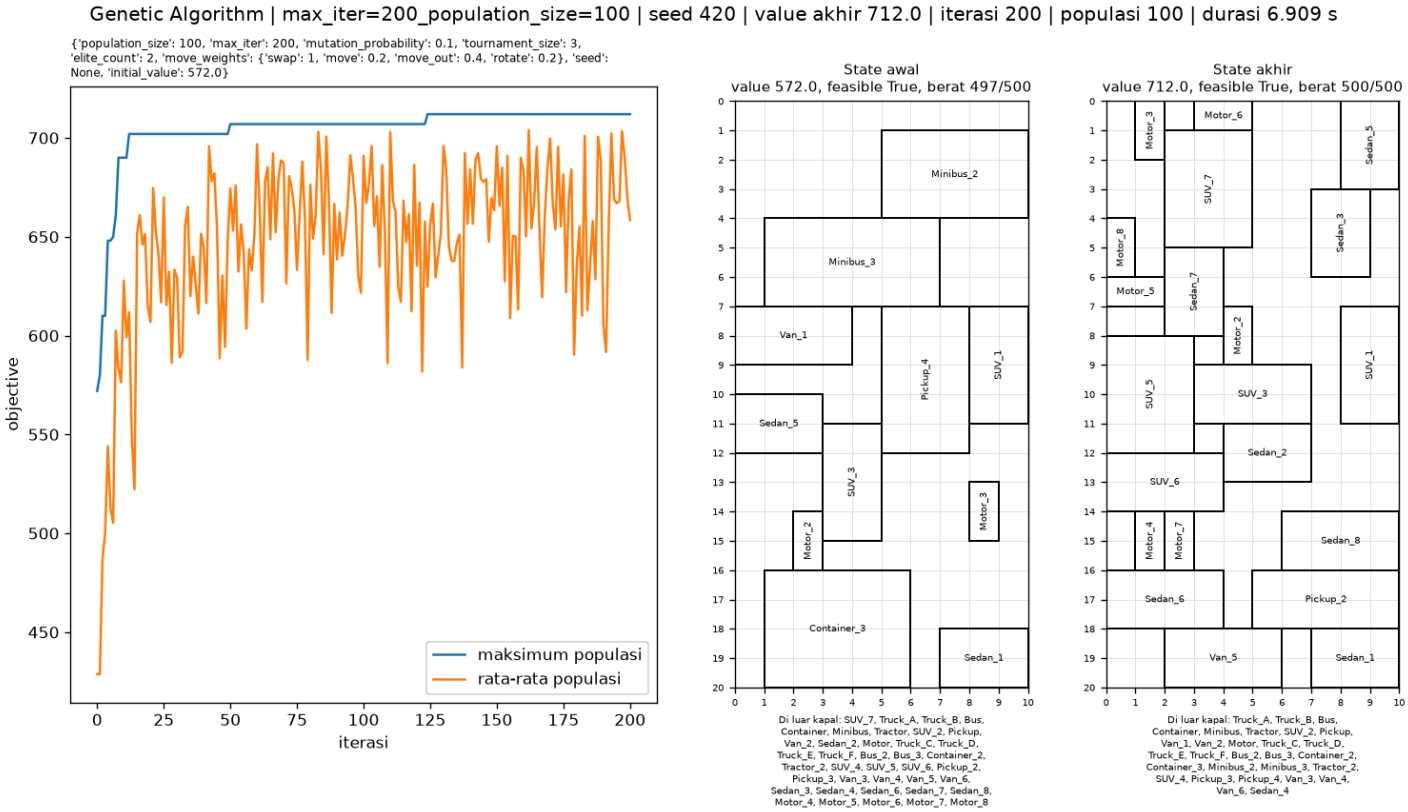
Di bawah merupakan visualisasi hasil eksperimen:



3.6.16 Eksperimen Keenam belas

input	seed = 420 population_size = 100 max_iter = 200
Hasil Eksperimen	Iterasi 200 | value 712.0 | feasible | inside: SUV_7, SUV_1, Sedan_1, S , Motor_2, Motor_3, Motor_4, Motor_5, Motor_6, Motor_7, Motor_8 Value awal : 572.0 Value akhir : 712.0 Feasible : True Iterasi : 200 Populasi : 100 Durasi : 6.909 s Visualisasi : output/genetic_algorithm/seed420max_iter200population100.png

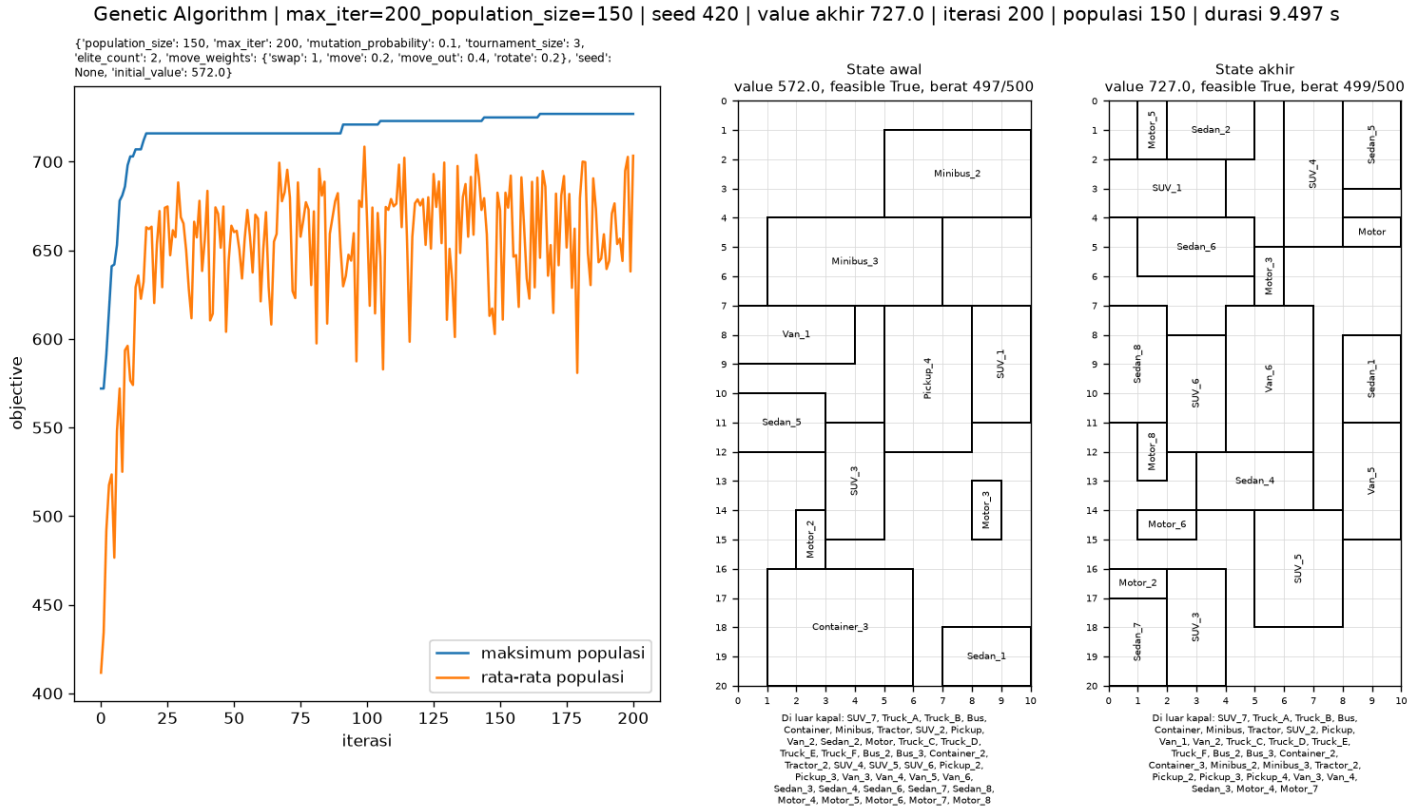
Di bawah merupakan visualisasi hasil eksperimen:



3.6.17 Eksperimen Ketujuh belas

input	seed = 420 population_size = 150 max_iter = 200
Hasil Eksperimen	Iterasi 200 | value 727.0 | feasible | inside: SUV_1, Sedan_1, Sedan_2an_8, Motor_2, Motor_3, Motor_5, Motor_6, Motor_8 Value awal : 572.0 Value akhir : 727.0 Feasible : True Iterasi : 200 Populasi : 150 Durasi : 9.497 s Visualisasi : output/genetic_algorithm/seed420max_iter200population150.png

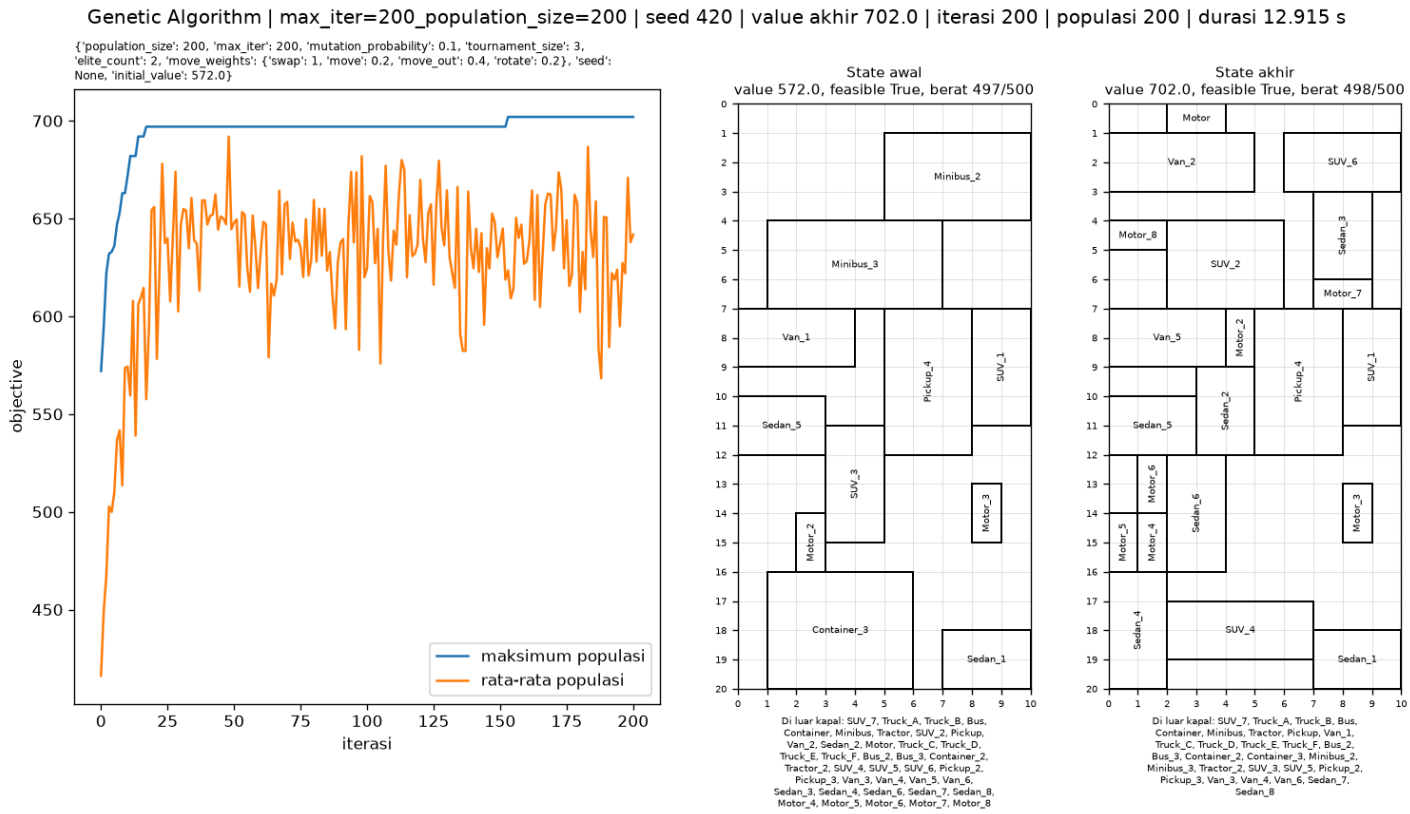
Di bawah merupakan visualisasi hasil eksperimen:



3.6.18 Eksperimen Kedelapan belas

input	seed = 420 population_size = 200 max_iter = 200
Hasil Eksperimen	Iterasi 200 | value 702.0 | feasible | inside: SUV_1, SUV_2, Van_2, Sed Motor_2, Motor_3, Motor_4, Motor_5, Motor_6, Motor_7, Motor_8 Value awal : 572.0 Value akhir : 702.0 Feasible : True Iterasi : 200 Populasi : 200 Durasi : 12.915 s Visualisasi : output/genetic_algorithm/seed420max_iter200population200.png

Di bawah merupakan visualisasi hasil eksperimen:



3.6.19 Analisis Genetic Algorithm

Seluruh eksperimen berhasil meningkatkan nilai objective function dari populasi awal yang acak menjadi hasil yang lebih baik. Seluruh solusi akhir juga memenuhi seluruh constraint, alias feasible.

Pada eksperimen dengan populasi sebagai kontrol, penambahan banyak iterasi hanya memberikan peningkatan yang kecil. Rata-rata nilai akhir naik dari 664,0 pada 100 iterasi menjadi 669,7 pada 200 iterasi dan 671,3 pada 300 iterasi, sedangkan durasinya naik hampir tiga kali lipat. Pada seed 67, misalnya, nilai 690 sudah ditemukan pada generasi ke-129 dan tidak berubah hingga generasi ke-300. Hal ini menunjukkan bahwa populasi berukuran 50 cepat konvergen (menyatu menjdi mirip), sehingga sebagian besar iterasi tambahan tidak lagi menghasilkan perbaikan.

Pada eksperimen dengan iterasi sebagai kontrol, jumlah populasi memberikan pengaruh yang lebih besar. Rata-rata nilai akhir mencapai 704,0 pada populasi 100, 724,0 pada populasi

150, dan 713,0 pada populasi 200, seluruhnya lebih tinggi dibandingkan eksperimen dengan populasi 50. Populasi yang lebih besar memiliki individu yang lebih beragam, sehingga nilai awalnya pun sudah lebih tinggi dan crossover memiliki lebih banyak kombinasi yang dapat digabungkan. Nilai tertinggi diperoleh pada Eksperimen 15 dengan nilai 730. Meskipun demikian, rata-rata populasi 200 sedikit lebih rendah daripada populasi 150 karena seed 67 dan 420 konvergen lebih awal pada nilai 707 dan 702. Durasi pencarian juga naik seiring bertambahnya populasi, yaitu sekitar 6,8, 9,6, dan 12,9 detik.

Elitism selalu membawa dua individu dengan value maksimum, sehingga pada grafik objective function, nilai maksimum cenderung tidak pernah turun. Secara keseluruhan, jumlah populasi lebih berpengaruh terhadap hasil akhir dibandingkan banyak iterasi. Hasil akhir juga lebih konsisten pada populasi yang lebih besar, terlihat dari selisih nilai antar-seed yang lebih kecil pada populasi 100 dan 150 dibandingkan populasi 50.

3.7 Analisis Keseluruhan

Berdasarkan hasil eksperimen dan analisis terhadap keseluruhan implementasi algoritma, berikut hasil analisis yang didapatkan.

1. **Kedekatan Algoritma Global Optimal**

   Global optimum persoalan ini tidak dihitung secara eksak. Nilai 730 dipakai sebagai acuan dan merupakan batas bawah dari global optimum, sehingga persentase pada Tabel 3.7.1 bersifat konservatif. Hasil Simulated Annealing dan Genetic Algorithm menunjukkan bahwa kapasitas berat (500) menjadi kendala yang paling mengikat, sebab seluruh solusi akhir kedua algoritma memuat berat tepat 500 dari 500, sedangkan luas geladak hanya terpakai 114–129 dari 200 sel. Dengan demikian, persoalan ini pada dasarnya mirip knapsack 0/1 atas fee dengan kendala berat. Batas atas knapsack tersebut dapat dihitung dari data instans dan akan menjadi pembanding yang lebih kuat bagi nilai 730.

   Keempat varian Hill-Climbing paling jauh dari acuan (73–82%) karena berhenti pada local optimum. Pada kondisi berat mendekati kapasitas, menambah kendaraan akan terkena penalti overweight

(basis penalti max_fee + 1 ditambah laju per unit berat). Sebagian besar langkah uphill pun hilang, padahal perbaikan yang sebenarnya membutuhkan perubahan komposisi muatan secara serentak, yaitu mengeluarkan beberapa kendaraan kecil dan memasukkan kendaraan yang lebih bernilai. Seluruh run Steepest Ascent berhenti hanya setelah 3 iterasi, yang menandakan bahwa landscape dengan himpunan tetangga satu-langkah ini memiliki banyak puncak lokal dekat titik awal. Sideways Move menambah langkah mendatar, tetapi pada seed 62 dan 420 langkah tersebut tidak membuka jalur perbaikan. Random Restart dan Stochastic memperlebar jangkauan pencarian dan hasilnya naik, tetapi keduanya tetap hanya menerima langkah yang menaikkan nilai sehingga tidak dapat melintasi "lembah" bernilai rendah.

Simulated Annealing mencapai 94,2% karena dapat menerima langkah yang memburuk dengan peluang e^(ΔE/T). Penalti pelanggaran bersifat berhingga, sehingga SA mampu melintasi state yang tidak feasible (nilai sempat negatif pada fase panas) untuk berpindah ke susunan muatan lain. Pemeriksaan tetangga pada solusi akhir SA menunjukkan bahwa ketiga solusi merupakan local optimum terhadap himpunan langkah yang dipakai. Jadi, SA belum menjamin global optimum pada konfigurasi ini, tetapi bergerak ke puncak lokal yang jauh lebih tinggi daripada Hill-Climbing.

Genetic Algorithm paling dekat dengan acuan (92–99%). Populasi yang beragam, crossover yang menggabungkan susunan baik dari dua induk, dan elitisme yang menjaga individu terbaik memungkinkan GA menjelajahi banyak daerah ruang solusi secara paralel. Pada konfigurasi populasi 150, rata-rata hasilnya (724) hanya 6 poin di bawah nilai terbaik.

2. Perbandingan hasil pencarian antaralgoritma

Urutan kualitas rata-rata dari tertinggi ke terendah adalah GA (konfigurasi terbaik 724) > Simulated Annealing (688) > Stochastic (601,33) > Random Restart (584,67) > Sideways Move (536) ≈ Steepest Ascent (534,33). Urutan ini sesuai dengan kemampuan setiap algoritma untuk keluar dari optimum lokal.

Di antara varian Hill-Climbing, selisih Sideways Move terhadap Steepest Ascent hanya 1,67 poin dan seluruhnya berasal dari seed 69

(562 menjadi 567). Pada seed 62 dan 420 hasilnya identik. Setiap run Sideways berakhir karena batas sideways tercapai (13 iterasi = 3 langkah naik + 10 langkah sideways), bukan karena solusi terbaik ditemukan. Plateau di sekitar solusi akhir pada instans ini tidak mengarah ke perbaikan lanjutan. Random Restart selalu tidak lebih buruk daripada Steepest Ascent pada seed yang sama (595 vs 487, 563 vs 562, 596 vs 554) karena restart pertama memakai state awal yang sama. Hal ini menunjukkan bahwa manfaat restart bergantung pada kualitas state awal baru yang kebetulan dijangkau. Stochastic mengungguli Steepest Ascent pada ketiga seed dan merupakan yang tertinggi di antara Hill-Climbing pada seed 69 dan 420. Pemilihan tetangga uphill secara acak menghindari lompatan serakah ke puncak lokal yang rendah. Dengan tiga seed, keunggulan ini masih perlu diuji pada lebih banyak seed sebelum dinyatakan konsisten.

Perlu dicatat bahwa SA dijalankan dengan seed 1, 2, 3 dan GA dengan seed 67, 69, 420, sedangkan Hill-Climbing memakai seed 62, 69, 420. Hanya keempat varian Hill-Climbing yang dibandingkan secara berpasangan pada state awal yang sama. Nilai awal SA (rata-rata 316,67) bahkan lebih rendah daripada Hill-Climbing (441,67), tetapi nilai akhirnya jauh lebih tinggi. Dengan demikian, keunggulan SA berasal dari mekanisme pencariannya, bukan dari state awal yang lebih baik.

3. Perbandingan durasi pencarian

Steepest Ascent merupakan yang tercepat (2,781 s), disusul SA (3,856 s), Stochastic (8,587 s), Sideways Move (10,334 s), dan Random Restart (31,251 s, sekitar 11 kali Steepest Ascent). GA berkisar 6,8–12,9 s pada 200 generasi dengan populasi 100–200.

Durasi tidak sebanding dengan jumlah iterasi karena biaya per iterasi sangat berbeda. Satu iterasi Hill-Climbing mengevaluasi seluruh tetangga (ribuan state, termasuk semua posisi pada move), sekitar 0,9 s per iterasi pada Steepest Ascent (2,781 s / 3 iterasi). SA hanya mengevaluasi satu tetangga acak per iterasi, sehingga sekitar 0,3 ms per iterasi. Dengan $T_0 = 1000$, $\alpha = 0{,}999$, dan $T_{min} = 0{,}01$, pencarian berhenti pada kisaran 11.500 iterasi (perhitungan dari parameter, lebih kecil dari batas max_iter 20.000) dan tetap selesai dalam 3,9 detik. Hal ini menjadikan SA paling efisien dalam rasio kualitas terhadap

waktu: nilai 688 dalam 3,9 detik, sedangkan Random Restart hanya 584,67 dalam 31,3 detik. Durasi GA tumbuh hampir linear terhadap jumlah populasi (sekitar 6,8; 9,6; dan 12,9 detik untuk populasi 100, 150, dan 200) dan terhadap jumlah generasi.

4. Konsistensi hasil antar-run

Rentang hasil dan simpangan baku pada Tabel 3.7.1 menunjukkan bahwa Random Restart paling konsisten di antara varian Hill-Climbing (simpangan baku 18,8; rentang 33), karena hasil akhirnya adalah yang terbaik dari sepuluh pencarian sehingga ketergantungan pada satu state awal berkurang. Steepest Ascent dan Sideways Move paling bergantung pada state awal (simpangan baku sekitar 41–43), dan Stochastic bervariasi karena unsur acaknya (seed 420 menghasilkan 647, jauh di atas dua seed lain). Variasi hasil SA berada pada tingkat menengah (simpangan baku 26,2). Seed 1 dan 2 berhenti membaik pada 660 dan 692, sedangkan seed 3 masih menemukan perbaikan hingga mendekati akhir dan mencapai 712. Jumlah kejadian stuck (K = 500) yang hampir sama (19, 19, 17) tidak memprediksi kualitas solusi akhir, sehingga stuck di sini lebih tepat dibaca sebagai indikator fase pencarian daripada tanda kegagalan.

Konsistensi GA meningkat seiring bertambahnya populasi. Selisih antar-seed pada populasi 100 dan 150 lebih kecil daripada populasi 50. Pada populasi 200, hasil per seed adalah 707, 702, dan 730 (simpangan baku sekitar 14,9), jauh di bawah variasi Hill-Climbing. Karena hanya tiga run per konfigurasi, seluruh nilai simpangan baku bersifat indikatif.

5. Pengaruh banyak iterasi dan jumlah populasi pada Genetic

Eksperimen A (populasi 50). Rata-rata nilai akhir naik dari 664,0 (100 iterasi) ke 669,7 (200 iterasi) dan 671,3 (300 iterasi), sedangkan durasi meningkat hampir tiga kali lipat. Kenaikan nilai sangat kecil dibandingkan kenaikan biaya komputasi. Pada seed 67, nilai 690 sudah ditemukan pada generasi ke-129 dan tidak berubah hingga generasi ke-300. Populasi berukuran 50 konvergen dini ke individu yang mirip (keragaman genetik habis), sehingga iterasi tambahan hampir tidak menghasilkan perbaikan. Elitisme juga membuat nilai maksimum tidak

pernah turun, tetapi sekaligus mempercepat dominasi individu terbaik.

Eksperimen B (200 iterasi). Populasi memberi pengaruh yang jauh lebih besar. Rata-rata nilai akhir mencapai 704,0 (populasi 100), 724,0 (populasi 150), dan 713,0 (populasi 200), seluruhnya lebih tinggi daripada populasi 50 pada iterasi yang sama (669,7). Populasi yang besar membawa lebih banyak variasi sejak awal sehingga nilai awalnya sudah lebih tinggi, dan crossover memiliki lebih banyak kombinasi untuk digabungkan. Nilai tertinggi keseluruhan (730) diperoleh pada populasi 200, seed 69. Rata-rata populasi 200 sedikit lebih rendah daripada populasi 150 karena seed 67 dan 420 konvergen lebih awal pada 707 dan 702. Hasil ini menunjukkan adanya titik jenuh, yaitu penambahan populasi tidak selalu menaikkan hasil sebanding dengan tambahan waktunya (6,8; 9,6; dan 12,9 detik).

Secara keseluruhan, untuk GA pada instans ini, memperbesar populasi lebih efektif daripada memperpanjang iterasi, dan populasi yang lebih besar juga menurunkan variasi antar-seed.

6. Pengaruh bobot langkah move_out pada SA

Uji tambahan pada SA memperlihatkan bahwa komposisi langkah ikut menentukan kualitas hasil. Bobot move_out 0,1 menghasilkan rata-rata 688,0, bobot 1 menghasilkan 568,7, dan bobot 0 hanya 316,7. Pada bobot 0, tidak ada langkah yang dapat mengurangi jumlah kendaraan, sehingga kapal tetap overweight dan solusi akhir sama dengan state awal. Pada bobot 1, kendaraan terlalu sering dikeluarkan, kapal hanya berisi 9–11 kendaraan, dan kendaraan kecil yang kontribusinya besar secara kumulatif terbuang. Karena hanya tiga nilai bobot dan tiga seed yang diuji, bobot 0,1 adalah yang terbaik di antara variasi tersebut, belum tentu yang paling optimal.

7. Keterbatasan dan implikasi

Hasil di atas berasal dari satu instans data (15 kendaraan, geladak 10×20, kapasitas 500) dengan tiga run per konfigurasi, sehingga perbedaan kecil antar-algoritma, misalnya Steepest Ascent dan Sideways Move, belum dapat dinyatakan signifikan secara statistik. Seed yang berbeda antar-algoritma juga membatasi perbandingan langsung. Nilai global optimum belum diverifikasi

secara eksak, sehingga seluruh persentase kedekatan bersifat relatif terhadap nilai terbaik yang ditemukan (730). Meskipun demikian, pola umumnya konsisten: algoritma yang mampu menerima langkah memburuk atau menjaga keragaman solusi (SA dan GA) mengungguli varian Hill-Climbing yang hanya menerima perbaikan, dengan GA memberikan kualitas tertinggi dan SA memberikan keseimbangan terbaik antara kualitas dan waktu.

IV. Kesimpulan dan Saran

Berdasarkan hasil eksperimen pada instans data yang digunakan (15 kendaraan, geladak 10×20, kapasitas berat 500), dapat ditarik kesimpulan sebagai berikut.

Seluruh algoritma menghasilkan solusi akhir yang feasible. Keempat varian Hill-Climbing, Simulated Annealing, dan Genetic Algorithm berhasil meningkatkan nilai objective function dari state awal yang acak, dengan solusi akhir yang tidak mengandung tumpang tindih, tidak keluar batas geladak, dan tidak melebihi kapasitas berat.

Kualitas solusi bergantung pada kemampuan algoritma keluar dari optimum lokal. Urutan rata-rata nilai akhir dari tertinggi ke terendah adalah Genetic Algorithm (724,00 pada konfigurasi populasi 150), Simulated Annealing (688,00), Stochastic Hill-Climbing (601,33), Random Restart Hill-Climbing (584,67), Sideways Move Hill-Climbing (536,00), dan Steepest Ascent Hill-Climbing (534,33). Algoritma yang hanya menerima langkah perbaikan berhenti pada optimum lokal, sedangkan Simulated Annealing dan Genetic Algorithm mampu menjelajahi ruang solusi lebih luas melalui penerimaan langkah yang memburuk dan keragaman populasi.

Terhadap nilai terbaik yang ditemukan (730), Genetic Algorithm dan Simulated Annealing paling mendekati. Genetic Algorithm mencapai 92,0–99,2% dan Simulated Annealing 94,2%, sedangkan varian Hill-Climbing berada pada 73,2–82,4%. Nilai 730 digunakan sebagai acuan karena global optimum belum diverifikasi secara eksak, sehingga persentase tersebut bersifat relatif.

Kapasitas berat merupakan kendala yang paling menentukan pada instans ini. Solusi akhir Simulated Annealing dan Genetic Algorithm memanfaatkan kapasitas berat secara penuh (500 dari 500), sedangkan luas geladak hanya terpakai sekitar 114–129 dari 200 sel. Persoalan ini karenanya berperilaku mirip knapsack 0/1 atas shipping fee.

Durasi pencarian tidak sebanding dengan kualitas solusi. Steepest Ascent tercepat (2,781 detik) tetapi hasilnya terendah. Random Restart paling lama (31,251 detik) dengan hasil yang masih di bawah Simulated Annealing. Simulated Annealing memberikan keseimbangan terbaik antara kualitas dan waktu (688,00 dalam 3,856 detik), karena hanya mengevaluasi satu tetangga acak per iterasi, sedangkan Hill-Climbing mengevaluasi seluruh tetangga pada setiap iterasi.

Konsistensi hasil bervariasi antar-algoritma. Random Restart paling konsisten di antara varian Hill-Climbing (simpangan baku 18,8), sedangkan Steepest Ascent dan Sideways Move paling bergantung pada state awal (simpangan baku sekitar 41–43). Pada Genetic Algorithm, populasi yang lebih besar menurunkan selisih hasil antar-seed.

Pada Genetic Algorithm, jumlah populasi lebih berpengaruh daripada banyak iterasi. Pada populasi 50, menambah iterasi dari 100 ke 300 hanya menaikkan rata-rata dari 664,0 menjadi 671,3 dengan durasi hampir tiga kali lipat, karena populasi konvergen dini. Pada 200 iterasi, populasi 100, 150, dan 200 menghasilkan rata-rata 704,0, 724,0, dan 713,0. Terdapat titik jenuh, yaitu populasi 200 tidak lebih baik daripada populasi 150 meskipun memerlukan waktu lebih lama.

Komposisi langkah neighbor memengaruhi hasil Simulated Annealing. Bobot move_out 0,1 menghasilkan rata-rata 688,0, lebih baik daripada bobot 1 (568,7) dan bobot 0 (316,7). Langkah move_out tetap diperlukan untuk memulihkan kelayakan solusi, tetapi harus dipilih dengan frekuensi rendah agar kendaraan bernilai tidak terbuang

V. Pembagian Tugas

Nama	NIM	Pembagian Tugas
Yumna Fathonah Kautsar	18224063	- Mengerjakan *Steepest Ascent Hill-Climbing Algorithm* - Mengerjakan *Random Restart Hill-Climbing Algorithm* - Mengerjakan *Stochastic Hill-Climbing Algorithm* - Mengerjakan *Sideways Move Hill-Climbing Algorithm*

Rafi Putra Nugraha	18224079	- Mengerjakan *Genetic Algorithm* - Membuat model.py , loader.py,visualicz dan objective.py
Illona Nasywa Hannum	18224081	- Mengerjakan *Simulated Annealing Algorithm* - Mengerjakan neighbors.py

VI. Referensi

Russell, S.J. and Norvig, P. (2021) Artificial Intelligence: A Modern Approach. 4th edn. Harlow: Pearson.

https://www.geeksforgeeks.org/dsa/implement-simulated-annealing-in-python/

https://www.tutorialspoint.com/genetic_algorithms/genetic_algorithms_parent_selection.htm